

O'REILLY®

2nd Edition

Certified Kubernetes Administrator (CKA) Study Guide

In-Depth Guidance and Practice



Early
Release

RAW &
UNEDITED

Benjamin Muschko

Certified Kubernetes Administrator (CKA) Study Guide

SECOND EDITION

In-Depth Guidance and Practice

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Benjamin Muschko

O'REILLY®

Certified Kubernetes Administrator (CKA) Study Guide

by Benjamin Muschko

Copyright © 2025 Automated Ascent LLC. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North,
Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Editors: John Devins and Corbin Collins

Production Editor: Beth Kelly

Copyeditor: FILL IN COPYEDITOR

Proofreader: FILL IN PROOFREADER

Indexer: FILL IN INDEXER

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

June 2022: First Edition

March 2026: Second Edition

Revision History for the Early Release

- 2025-04-04: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9798341608405> for release

details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Certified Kubernetes Administrator (CKA) Study Guide*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-834-1-60835-1

[FILL IN]

Brief Table of Contents (Not Yet Final)

Part 1: Introduction

Chapter 1: Exam Details and Resources (available)

Chapter 2: Kubernetes in a Nutshell (available)

Chapter 3: Interacting with Kubernetes (available)

Part 2: Cluster Architecture, Installation, and Configuration

Chapter 4: Cluster Installation and Upgrade (available)

Chapter 5: Backing Up and Restoring etcd (available)

Chapter 6: Authentication, Authorization, and Admission Control (available)

Chapter 7: Operators and Custom Resource Definitions (CRDs) (available)

Chapter 8: Helm and Kustomize (available)

Appendix A: Answers to Review Questions (available)

Appendix B: Exam Review Guide (available)

Part I. Introduction

The *Introduction* section of the book touches on the most important aspects of the exam and orients beginners to Kubernetes to the lay of the land without introducing too much complexity.

The following chapters cover these concepts:

- **Chapter 1** discusses the exam objectives, curriculum, and tips and tricks for passing the exam.
- **Chapter 2** is a short and sweet overview on Kubernetes. This chapter summarizes the purpose and benefits of Kubernetes and provides an overview of its architecture and components.
- **Chapter 3** discusses how to interact with a Kubernetes cluster using the command line tool `kubectl`. The tool is going to be your only user interface during the exam. We'll compare imperative and declarative commands, their pros and cons, as well as time-saving techniques for the exam.

Chapter 1. Exam Details and Resources

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles. This will be the 1st chapter of the final book. Please note that the GitHub repo will be made active later on. If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

This chapter addresses the most frequently asked questions by candidates preparing to successfully pass the **Certified Kubernetes Administrator (CKA)** exam. Later chapters will give you a summary of **Kubernetes’ benefits and architecture** and how to **interact with a Kubernetes cluster using kubectl**.

Kubernetes Certification Learning Path

The CNCF offers four different Kubernetes certifications. **Figure 1-1** categorizes each of them by target audience.

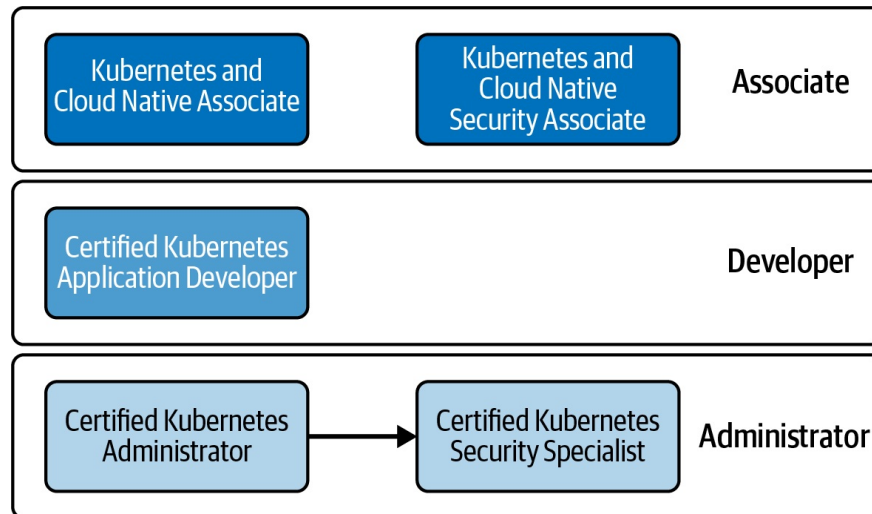


Figure 1-1. Kubernetes certifications learning path

The target audience for associate-level certifications is beginners to the cloud and Kubernetes. Associate-level certification exams use a multiple-choice format. You will not have to interact with a Kubernetes cluster in an interactive environment.

Practitioner-level certifications are meant for developers and administrators with preexisting Kubernetes experience. Exams in this category require you to solve problems in multiple Kubernetes environments hands-on. You will find that the CKA is geared to administrators of a Kubernetes cluster and does not require any other certification as a prerequisite.

If you pass all Kubernetes certifications, you can call yourself a **Kubestronaut**. The Kubestronaut program recognized community members that were able to demonstrate their skills in the most important core areas of Kubernetes.

Let's have a brief look at each certification to see if the CKA is the right fit for you.

Kubernetes and Cloud Native Associate (KCNA)

KCNA is an entry-level certification program for anyone interested in cloud-native application development, runtime environments, and tooling. While the exam does cover Kubernetes, it does *not* expect you to interact with a cluster hands-on. This exam consists of multiple-choice questions and is suitable to candidates interested in the topic with a broad exposure to the ecosystem.

Kubernetes and Cloud Native Security Associate (KCSA)

The **KCSA** verifies your basic knowledge of security concepts and their application in a Kubernetes cluster. The breadth, depth, and format of the program is definitely more advanced than the KCNA. I personally would not categorize this exam as suitable to beginners to security concepts. You may want to take the KCSA after gaining more hands-on and in depth exposure to Kubernetes, as the exam questions can be quite hard to answer.

Certified Kubernetes Application Developer (CKAD)

The **CKAD** exam focuses on verifying your ability to build, configure, and deploy a microservices-based application to Kubernetes. You are not expected to actually implement an application; however, the exam is suitable for developers familiar with topics like application architecture, runtimes, and programming languages.

Certified Kubernetes Administrator (CKA)

The target audience for the **CKA** exam are DevOps practitioners, system administrators, and site reliability engineers. This exam tests your ability to perform in the role of a Kubernetes administrator, which includes tasks like cluster, network, storage, and beginner-level security management, with emphasis on troubleshooting scenarios.

Certified Kubernetes Security Specialist (CKS)

The **CKS** exam expands on the topics verified by the CKA exam. Passing the CKA is a prerequisite before you can sign up for the CKS exam. For this certification, you are expected to have a deeper knowledge of Kubernetes security. The curriculum covers topics like applying best practices for building containerized applications and ensuring a secure Kubernetes runtime environment.

Exam Objectives

Kubernetes clusters need to be installed, configured, and maintained by skilled

professionals. That's the job of a Kubernetes administrator. The CKA certification program verifies a deep understanding of the typical administration tasks encountered on the job, more specifically Kubernetes cluster maintenance, networking, storage solutions, and troubleshooting applications and cluster nodes.

This book focuses on getting you ready for the CKA exam. I will give a little bit of background on why Kubernetes is important to administrators before dissecting the topics important to the exam.

KUBERNETES VERSION USED DURING THE EXAM

At the time of writing, the exam is based on Kubernetes 1.32. All content in this book will follow the features, APIs, and command-line support for that version. It's possible that future versions will break backward compatibility. While preparing for the certification, review the [Kubernetes release notes](#) and practice with the Kubernetes version used during the exam to avoid unpleasant surprises. The exam environment will be aligned with the most recent Kubernetes minor version within approximately four to eight weeks of the Kubernetes release date.

Curriculum

The following overview lists the high-level sections, or domains, of the exam and their scoring weights:

- 25%: [Cluster Architecture, Installation and Configuration](#)
- 15%: Workloads and Scheduling
- 20%: Servicing and Networking
- 10%: Storage
- 30%: Troubleshooting

The next sections detail each domain.

Cluster Architecture, Installation and Configuration

This section of the curriculum dives deep into all aspects of Kubernetes clusters.

It covers the fundamental architecture of Kubernetes, including the distinction between the control plane and worker nodes, high-availability configurations, and the tools needed for installing, upgrading, and maintaining a cluster. Additionally, you'll explore key extension interfaces and learn practical skills such as installing a cluster from scratch, upgrading its version, and backing up/restoring the etcd database.

The Cloud Native Computing Foundation (CNCF) has also incorporated related topics into this domain. For instance, mastering Role-Based Access Control (RBAC) is essential for administrators to effectively manage access to cluster resources. You'll also become proficient in installing Kubernetes operators and using tools like Kustomize and Helm to discover and deploy cluster components efficiently.

Workloads and Scheduling

Administrators must have a solid understanding of Kubernetes concepts essential for managing cloud-native applications effectively. The “Workloads and Scheduling” domain focuses on these critical aspects. It covers key resources such as Deployments, ReplicaSets, and configuration management using ConfigMaps and Secrets.

When a new Pod is created, the Kubernetes scheduler assigns it to an available node based on predefined criteria. Scheduling rules, such as node affinity and taints/tolerations, help fine-tune this process to meet specific requirements. For exam preparation, it's crucial to grasp the various factors and concepts involved in Kubernetes' scheduling algorithm to ensure optimal workload placement and performance.

Servicing and Networking

A cloud-native microservice rarely operates in isolation. More often than not, it interacts with other microservices or external systems. For administrators, understanding Pod-to-Pod communication, exposing applications to external clients, and configuring cluster networking is crucial for maintaining a fully functional system.

This domain of the exam evaluates your knowledge of essential Kubernetes

networking primitives, including Services, Ingress, NetworkPolicy, and the Gateway API. Mastering these components ensures you can effectively manage and secure communication within and outside the cluster.

Storage

This domain focuses on the various types of volumes used for reading and writing data in Kubernetes. As an administrator, you must understand how to create, configure, and manage these volumes effectively.

Persistent Volumes (PVs) play a crucial role in ensuring data persistence, even after a cluster node restarts. You'll need to demonstrate the ability to mount a Persistent Volume to a specific path within a container and understand the underlying mechanics. Additionally, it's essential to grasp the differences between static and dynamic provisioning to manage storage resources efficiently.

Troubleshooting

In production Kubernetes clusters, issues are bound to arise. Applications may misbehave, become unresponsive, or even become completely inaccessible. Additionally, cluster nodes might crash or face configuration problems. Developing effective troubleshooting strategies is critical to quickly identifying and resolving these situations to minimize downtime and disruption.

This domain carries the highest scoring weight in the exam. You will encounter realistic scenarios that require you to diagnose problems and implement the appropriate solutions. Mastering these skills ensures you can maintain the stability and reliability of Kubernetes environments under pressure.

Involved Kubernetes Primitives

Some of the exam objectives can be covered by understanding the relevant core Kubernetes primitives. Be aware that the exam combines multiple concepts in a single problem. Refer to **Figure 1-2** as a guide to the applicable Kubernetes resources and their relationships.

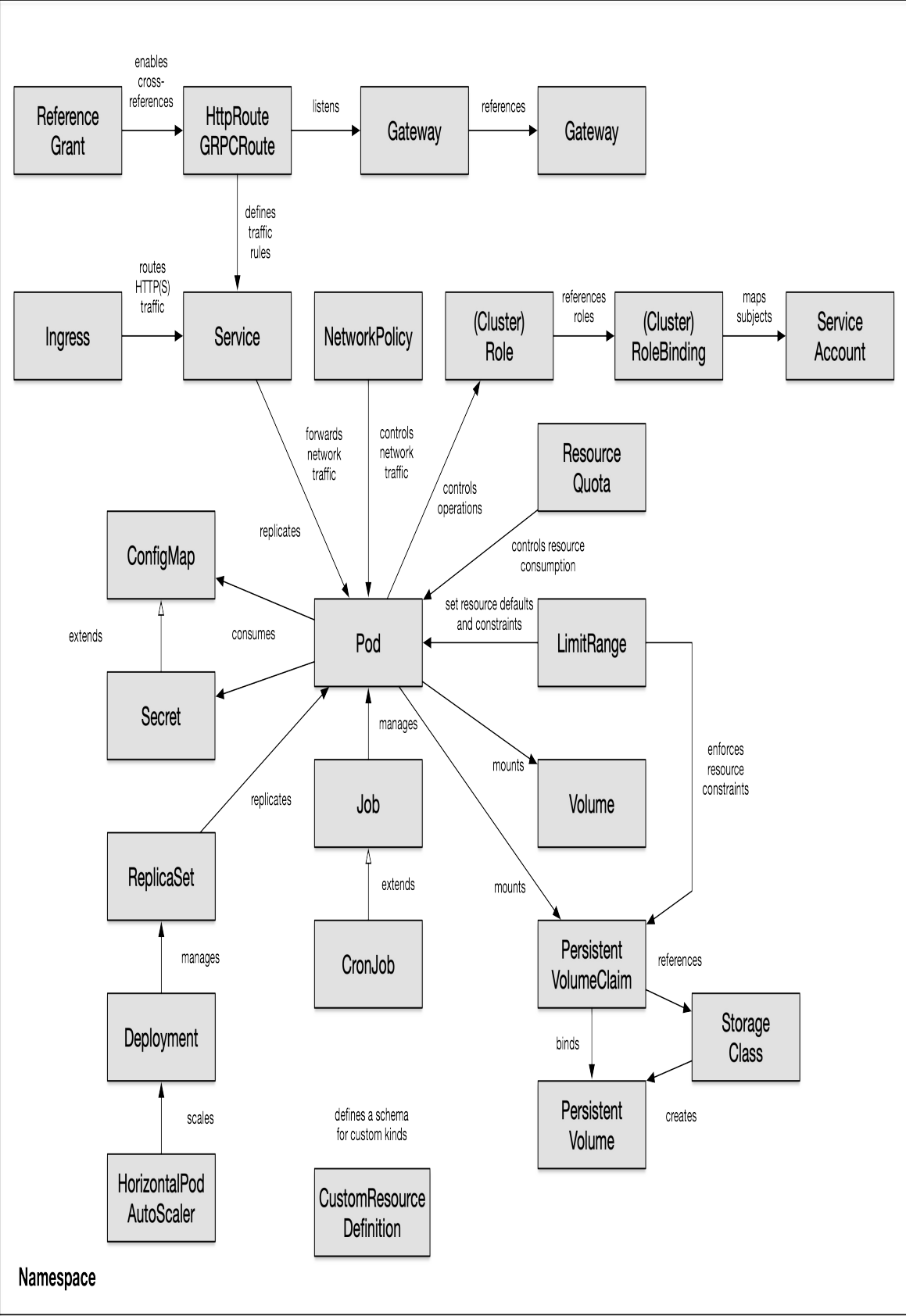


Figure 1-2. Kubernetes primitives relevant to the exam

Documentation

During the exam, you are permitted to open a well-defined list of web pages as a reference. You can freely browse those pages and copy-paste code to the exam terminal.

The official Kubernetes documentation includes the reference manual, and the blog. In addition, you can also browse the Helm documentation.

- Reference manual: <https://kubernetes.io/docs>
- Blog: <https://kubernetes.io/blog>
- Helm: <https://helm.sh/docs>

Having the Kubernetes documentation pages at hand is extremely valuable, but make sure you know *where* to find the relevant information within those pages. In preparation for the test, read all the documentation pages from start to end at least once. Don't forget the search functionality of the official documentation pages. For reference, [Appendix B](#) maps the exam objectives to the book chapters covering the topics and the relevant Kubernetes documentation pages.

USING THE DOCUMENTATION EFFICIENTLY

Using a search term will likely lead you to the right documentation pages quicker than navigating the menu items. Copying and pasting code snippets from the documentation into the console of the exam environment works reasonably well. You may have to adjust the YAML indentation manually as the proper formatting can get lost in the process.

Exam Environment and Tips

To take the exam, you must purchase a registration voucher, which can be acquired on the [CNCF training and certification web page](#). On occasion, the CNCF offers discounts for the voucher (e.g., around the US Thanksgiving holiday). Those discount offers are often announced on the [Linux Foundation LinkedIn page](#).

After you purchase the voucher, you can schedule a time for the exam with **PSI**, the company conducting the test virtually. In-person exams at a testing facility are not available. On the day of your scheduled test, you'll be asked to log into the test platform with a URL provided to you by email. You'll be asked to enable the audio and video feed on your computer to discourage cheating. A proctor will oversee your actions via audio/video feed and terminate the session if they think you are not following the rules.

EXAM ATTEMPTS

The voucher you purchased grants two attempts to pass the exam. I recommend preparing reasonably well before taking the test on the first attempt. It will give you a fair chance to pass the test and provide a good impression of the exam environment and the complexity of the questions. Don't sweat it if you do not pass the test on the first attempt. You've got another free shot.

The exam has a time limit of two hours. During that time, you'll need to solve hands-on problems on a real, predefined Kubernetes cluster. Every question will state the cluster you need to work on. This practical approach to gauge a candidate's skill set is superior to tests with multiple-choice questions, as you can translate the knowledge directly on tasks performed on the job.

I highly recommend reading the **FAQ for the exam**. You will find answers to most of your pressing questions there, including system requirements for your machine, scoring, certification renewal, and retake requirements.

Candidate Skills

The certification assumes that you have a basic understanding of Kubernetes. You should be familiar with Kubernetes internals, its core concepts, and the command-line tool `kubectl`. The CNCF offers a free **"Introduction to Kubernetes" course** for beginners to Kubernetes.

Your background is likely more on the end of an application developer, although it doesn't really matter which programming language you're most accustomed to. Here's a brief overview of the background knowledge you need to increase your likelihood of passing the exam:

Kubernetes architecture and concepts

The exam won't ask you to install a Kubernetes cluster from scratch. Read up on the basics of Kubernetes and its architectural components. Reference [Chapter 2](#) for a jump start on Kubernetes' architecture and concepts.

The `kubectl` CLI tool

The `kubectl` command-line tool is the central tool you will use during the exam to interact with the Kubernetes cluster. Even if you have only a little time to prepare for the exam, it's essential to practice how to operate `kubectl`, as well as its commands and their relevant options. You will have no access to the [web dashboard UI](#) during the exam. [Chapter 3](#) provides a short summary of the most important ways of interacting with a Kubernetes cluster.

Kubernetes cluster maintenance tools

Installing a Kubernetes cluster from scratch and upgrading the Kubernetes version of an existing cluster is performed using the tool `kubeadm`. It's important to understand its usage and the relevant process to walk through the process. Reference [Chapter 4](#) for more information. Additionally, you need to have a good understanding of the tool `etcdctl` and `etcdutl` including its command-line options for backing up and restoring the etcd database covered in [Chapter 5](#).

Working knowledge of a container runtime engine

Kubernetes uses a container runtime engine for managing images. The default container runtime engine in Kubernetes is `containerd`. At a minimum, understand the difference between container images and containers, and their purpose. This topic goes beyond the coverage of this book and isn't directly tested for in the exam.

Other relevant tools

Kubernetes objects are represented by YAML or JSON. The content of this book will use examples in YAML, as it is more commonly used than JSON in the Kubernetes world. You will have to edit YAML during the exam to

create a new object declaratively or when modifying the configuration of a live object. Ensure that you have a good handle on basic YAML syntax, data types, and indentation conforming to the specification. How do you edit the YAML definitions, you may ask? From the terminal, of course. The exam terminal environment comes with the tools `vi` and `vim` preinstalled. Practice the keyboard shortcuts for common operations, (especially how to exit the editor). The last tool I want to mention is GNU Bash. It's imperative that you understand the basic syntax and operators of the scripting language. It's absolutely possible that you may have to read, modify, or even extend a multiline Bash command running in a container.

Time Management

Candidates have two hours to complete the exam, and at least 66% of the answers to the tasks need to be correct to pass. Many tasks consist of multiple steps. Although the Linux Foundation doesn't provide a scoring breakdown, I'd assume that partially correct answers will score a portion of the points.

When taking the test, you will notice that the given time limit will put you under a lot of pressure. That's intentional. The Linux Foundation expects Kubernetes practitioners to be able to apply their knowledge to real-world scenarios by finding solutions to problems in a timely fashion.

The exam will present you with a mix of problems. Some are short and easy to solve; others require more context and take more time. Personally, I tried to tackle the easy problems first to score as many points as possible without getting stuck on the harder questions. I marked any questions I could not solve immediately in the notepad functionality integrated in the exam environment. During the second pass, revisit the questions you skipped and try to solve them as well. Optimally, you will be able to work through all the problems in the allotted time.

Command-Line Tips and Tricks

Given that the command line is your solitary interface to the Kubernetes cluster, it's essential that you become extremely familiar with the `kubectl` tool and its

available options. This section provides tips and tricks for making their use more efficient and productive.

Setting a Context and Namespace

The exam environment comes with multiple Kubernetes clusters already set up for you. Take a look at the **instructions** for a high-level, technical overview of those clusters. Each of the exam tasks need to be solved on a designated cluster, as outlined in its description. Furthermore, the instructions will ask you to work in a namespace other than `default`. Make sure to set the context and namespace as the first course of action before working on a question. The following command sets the context and the namespace as a one-time action:

```
$ kubectl config set-context <context-of-question> \
  --namespace=<namespace-of-question>
$ kubectl config use-context <context-of-question>
```

You can find a more detailed discussion of the context concept and the corresponding `kubectl` commands in “**Authentication with kubectl**”.

For specific tasks, e.g. the upgrade process of a cluster node, you will need to open an interactive shell to the host machine of a cluster node. Use the `ssh <nodename>` command to achieve that.

Using the Alias for kubectl

In the course of the exam, you will have to execute the `kubectl` command tens or even hundreds of times. You might be an extremely fast typist; however, there’s no point in fully spelling out the executable over and over again. The exam environment already sets up the alias `k` for the `kubectl` command.

In preparation for the exam, you can set up the same behavior on your machine. The following `alias` command maps the letter `k` to the full `kubectl` command:

```
$ alias k=kubectl
$ k version
```

Using kubectl Command Auto-Completion

Memorizing `kubectl` commands and command-line options takes a lot of practice. The exam environment comes with auto-completion enabled by default. You can find instructions for setting up auto-completion for the shell on your machine in the [Kubernetes documentation](#).

Internalize Resource Short Names

Many of the `kubectl` commands can be quite lengthy. For example, the command for managing Persistent volume claims is `persistentvolumeclaims`. Spelling out the full command can be error-prone and time-consuming. Thankfully, some of the longer commands come with a short-form usage. The command `api-resources` lists all available commands plus their short names:

```
$ kubectl api-resources
NAME                                SHORTNAMES  APIGROUP  NAMESPACE  KIND
...
persistentvolumeclaims  pvc                                true
PersistentVolumeClaim
...
```

Using `pvc` instead of `persistentvolumeclaims` results in a more concise and expressive command execution, as shown here:

```
$ kubectl describe pvc my-claim
```

Practicing and Practice Exams

Hands-on practice is extremely important when it comes to passing the exam. For that purpose, you'll need a functioning Kubernetes cluster environment. The following options stand out:

- I found it useful to run one or many virtual machines using [Vagrant](#) and [VirtualBox](#). Those tools help with creating an isolated Kubernetes environment that is easy to bootstrap and dispose on demand.
- It is relatively easy to install a simple Kubernetes cluster on your

developer machine. The Kubernetes documentation provides various **installation options**, depending on your operating system. **Minikube** is useful when it comes to experimenting with more advanced features like Ingress or storage classes, as it provides the necessary functionality as add-ons that can be installed with a single command. Alternatively, you can also give **kind** a try, another tool for running local Kubernetes clusters.

- If you're a subscriber to the **O'Reilly Learning Platform**, you have unlimited access to scenarios running a **Kubernetes sandbox environment**. In addition, you can test your knowledge with the help of the **CKA practice test in the form of interactive labs**.
- **Killercodea** hosts interactive scenarios for the exam contributed by the Kubernetes community.

You may also want to try one of the following commercial learning and practice resources:

- **Killer Shell** is a simulator with sample exercises for all Kubernetes certifications. If you purchase a voucher for the exam, you will be allowed two free sessions.
- Other online training providers offer video courses for the exam, some of which include an integrated Kubernetes practice environment. I would like to mention **KodeKloud** and **A Cloud Guru**. You'll need to purchase a subscription to access the content for each course individually.

Summary

The exam is a completely hands-on test that requires you to solve problems in multiple Kubernetes clusters. You're expected to understand, use, and configure the Kubernetes primitives relevant to application developers. The exam curriculum subdivides those focus areas and puts different weights on topics, which determines their contributions to the overall score. Even though focus areas are grouped meaningfully, the curriculum doesn't necessarily follow a

natural learning path, so it's helpful to cross-reference chapters in the book in preparation for the exam.

In this chapter, we discussed the exam environment and how to navigate it. The key to acing the exam is intense practice of `kubectl` to solve real-world scenarios. The next two chapters in **Part I** will provide a jump start to Kubernetes.

All chapters that discuss domain details give you an opportunity to practice hands-on. You will find sample exercises at the end of each chapter.

Chapter 2. Kubernetes in a Nutshell

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles. This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on. If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

It’s helpful to get a quick rundown of what Kubernetes is and how it works if you are new to the space. Many tutorials and 101 courses are available on the web, but I would like to summarize the most important background information and concepts in this chapter. In the course of this book, we’ll reference cluster node components, so feel free to come back to this information at any time.

What Is Kubernetes?

To understand what Kubernetes is, first let’s define microservices and containers.

Microservice architectures call for developing and executing pieces of the application stack as individual services, and those services have to communicate with one another. If you decide to operate those services in containers, you will need to manage a lot of them while at the same time thinking about cross-cutting concerns like scalability, security, persistence, and load balancing.

Tools like **buildkit** and **Podman** package software artifacts into a container image. Container runtime engines like **Docker Engine** and **containerd** use the image to run a container. This works great on developer machines for testing purposes or for ad-hoc executions, e.g., as part of a Continuous Integration

pipeline.

Kubernetes is a container orchestration tool that helps with operating hundreds or even thousands of containers on physical machines, virtual machines, or in the cloud. Kubernetes can also fulfill those cross-cutting concerns mentioned earlier. The container runtime engine integrates with Kubernetes. Whenever a container creation is triggered, Kubernetes will delegate life cycle aspects to the container runtime engine.

The most essential primitive in a Kubernetes is a Pod. The Pod can run one or many containers while at the same time adding cross-cutting concerns like security requirements and resource consumption expectations. Have a look at [\[Link to Come\]](#) to learn about those aspects.

Features

The previous section touched on some features provided by Kubernetes. Here, we are going to dive a little deeper by explaining those features with more detail:

Declarative model

You do not have to write imperative code using a programming language to tell Kubernetes how to operate an application. All you need to do as an end user is to declare a desired state. The desired state can be defined using a YAML or JSON manifest that conforms to an API schema. Kubernetes then maintains the state and recovers it in case of a failure.

Autoscaling

You will want to scale up resources when your application load increases, and scale down when traffic to your application decreases. This can be achieved in Kubernetes by manual or automated scaling. The most practical, optimized option is to let Kubernetes automatically scale resources needed by a containerized application.

Application management

Changes to applications, e.g., new features and bug fixes, are usually baked into a container image with a new tag. You can easily roll out those changes

across all containers running them using Kubernetes' convenient replication feature. If needed, Kubernetes also allows for rolling back to a previous application version in case of a blocking bug or if a security vulnerability is detected.

Persistent storage

Containers offer only a temporary filesystem. Upon restart of the container, all data written to the filesystem is lost. Depending on the nature of your application, you may need to persist data for longer, for example, if your application interacts with a database. Kubernetes offers the ability to mount storage required by application workloads.

Networking

To support a microservices architecture, the container orchestrator needs to allow for communication between containers, and from end users to containers from outside of the cluster. Kubernetes employs internal and external load balancing for routing network traffic.

High-Level Architecture

Architecturally, a Kubernetes cluster consists of control plane nodes and worker nodes, as shown in [Figure 2-1](#). Each node runs on infrastructure provisioned on a physical or virtual machine, or in the cloud. The number of nodes you want to add to the cluster and their topology depends on the application resource needs.

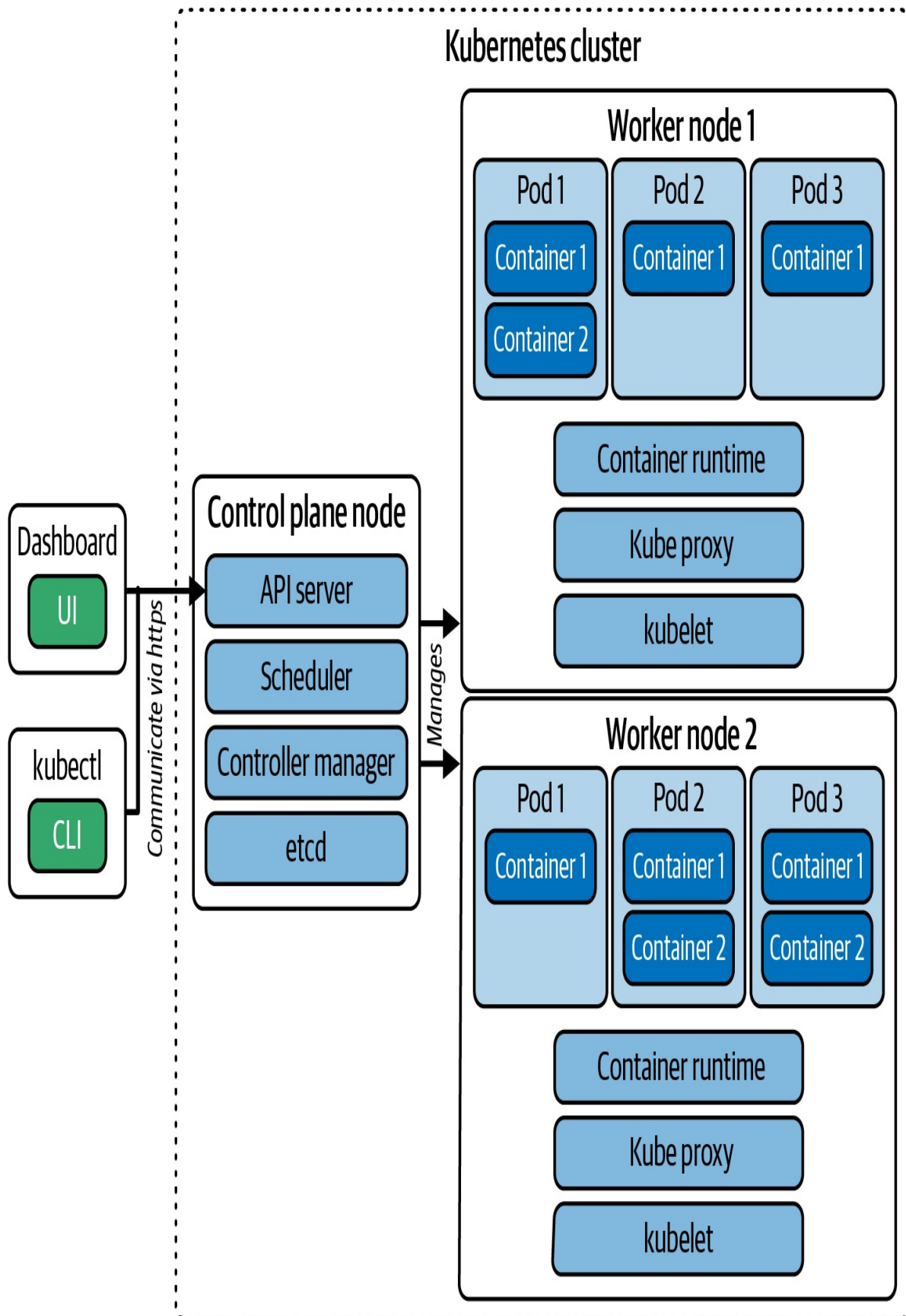


Figure 2-1. Kubernetes cluster nodes and components

Control plane nodes and worker nodes have specific responsibilities:

Control plane node

This node exposes the Kubernetes API through the API server and manages the nodes that make up the cluster. It also responds to cluster events, for example, when the end user requested to scale up the number of Pods to distribute the load for an application. Production clusters employ a **highly available (HA) architecture** that usually involves three or more control plane nodes.

Worker node

The worker node executes workload in containers managed by Pods. Every worker node needs a container runtime engine installed on the host machine to be able to manage containers.

In the next two sections, we'll talk about the essential components embedded in those nodes to fulfill their tasks. Add-ons like cluster DNS are not discussed explicitly here. See the **Kubernetes documentation** for more details.

Control Plane Node Components

The control plane node requires a specific set of components to perform its job. The following list of components will give you an overview:

API server

The API server exposes the API endpoints clients use to communicate with the Kubernetes cluster. For example, if you execute the tool `kubectl`, a command-line based Kubernetes client, you will make a RESTful API call to an endpoint exposed by the API server as part of its implementation. The API processing procedure inside of the API server will ensure aspects like authentication, authorization, and admission control. For more information on that topic, see **Chapter 6**.

Scheduler

The scheduler is a background process that watches for new Kubernetes Pods with no assigned nodes and assigns them to a worker node for execution.

Controller manager

The controller manager watches the state of your cluster and implements changes where needed. For example, if you make a configuration change to an existing object, the controller manager will try to bring the object into the desired state.

Etc

Cluster state data needs to be persisted over time so it can be reconstructed upon a node or even a full cluster restart. That's the responsibility of **etcd**, an open source software Kubernetes integrates with. At its core, etcd is a key-value store used to persist all data related to the Kubernetes cluster.

Common Node Components

Kubernetes employs components that are leveraged by all nodes independent of their specialized responsibility:

Kubelet

The kubelet runs on every node in the cluster; however, it makes the most sense to exist on a worker node. The reason is that the control plane node usually doesn't execute workload, and the worker node's primary responsibility is to run workload. The kubelet is an agent that makes sure that the necessary containers are running in a Pod. You could say that the kubelet is the glue between Kubernetes and the container runtime engine and ensures that containers are running and healthy. We'll have a touch point with the kubelet in [\[Link to Come\]](#).

Kube proxy

The kube proxy is a network proxy that runs on each node in a cluster to maintain network rules and enable network communication. In part, this component is responsible for implementing the Service concept covered in

[Link to Come].

Container runtime

As mentioned earlier, the container runtime is the software responsible for managing containers. Kubernetes can be configured to choose from a range of different container runtime engines. While you can install a container runtime engine on a control plane, it's not necessary as the control plane node usually doesn't handle workload. We'll use a container runtime in [Link to Come] to create a container image and run a container with the produced image.

Advantages

This chapter points out a couple of advantages of Kubernetes, which are summarized here:

Portability

A container runtime engine can manage a container independent of its runtime environment. The container image bundles everything it needs to work, including the application's binary or code, its dependencies, and its configuration. Kubernetes can run applications in a container in on-premise and cloud environments. As an administrator, you can choose the platform you think is most suitable to your needs without having to rewrite the application. Many cloud offerings provide product-specific, opt-in features. While using product-specific features helps with operational aspects, be aware that they will diminish your ability to switch easily between platforms.

Resilience

Kubernetes is designed as a declarative state machine. Controllers are reconciliation loops that watch the state of your cluster, then make or request changes where needed. The goal is to move the current cluster state closer to the desired state.

Scalability

Enterprises run applications at scale. Just imagine how many software

components retailers like Amazon, Walmart, or Target need to operate to run their businesses. Kubernetes can scale the number of Pods based on demand or automatically according to resource consumption or historical trends.

API based

Kubernetes exposes its functionality through APIs. We learned that every client needs to interact with the API server to manage objects. It is easy to implement a new client that can make RESTful API calls to exposed endpoints.

Extensibility

The API aspect stretches even further. Sometimes, the core functionality of Kubernetes doesn't fulfill your custom needs, but you can implement your own extensions to Kubernetes. With the help of specific extension points, the Kubernetes community can build custom functionality according to their requirements, e.g., monitoring or logging solutions.

Summary

Kubernetes is software for managing containerized applications at scale. Every Kubernetes cluster consists of at least a single control plane node and a worker node. The control plane node is responsible for scheduling the workload and acts as the single entrypoint to manage its functionality. Worker nodes handle the workload assigned to them by the control plane node.

Kubernetes is a production-ready runtime environment for companies wanting to operate microservice architectures while also supporting nonfunctional requirements like scalability, security, load balancing, and extensibility.

The next chapter will explain how to interact with a Kubernetes cluster using the command-line tool `kubectl`. You will learn how run it to manage objects, an essential skill for acing the exam.

Chapter 3. Interacting with Kubernetes

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles. This will be the 3rd chapter of the final book. Please note that the GitHub repo will be made active later on. If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

As an application developer, you will want to interact with the Kubernetes cluster to manage objects that operate your application. Every call to the cluster is accepted and processed by the API server component. There are various ways to perform a call to the API server. For example, you can use a **web-based dashboard**, a command-line tool like **kubectl**, or a direct HTTPS request to the RESTful API endpoints.

The exam does not test the use of a visual user interface for interacting with the Kubernetes cluster. Your only client for solving exam questions is **kubectl**. This chapter will touch on the Kubernetes API primitives and objects, as well as the different ways to manage objects with **kubectl**.

API Primitives and Objects

Kubernetes primitives are the basic building blocks anchored in the Kubernetes architecture for creating and operating an application on the platform. Even as a beginner to Kubernetes, you might have heard of the terms Pod, Deployment, and Service, all of which are Kubernetes primitives. There are many more that serve a dedicated purpose in the Kubernetes architecture.

To draw an analogy, think back to the concepts of object-oriented programming. In object-oriented programming languages, a class defines the blueprint of a real-world functionality: its properties and behavior. A Kubernetes primitive is the equivalent of a class. The instance of a class in object-oriented programming is an object, managing its own state and having the ability to communicate with other parts of the system. Whenever you create a Kubernetes object, you produce such an instance.

For example, a Pod in Kubernetes is the class of which there can be many instances with their own identity. Every Kubernetes object has a system-generated unique identifier (also known as UID) to clearly distinguish between the entities of a system. Later, we'll look at the properties of a Kubernetes object. **Figure 3-1** illustrates the relationship between a Kubernetes primitive and an object.

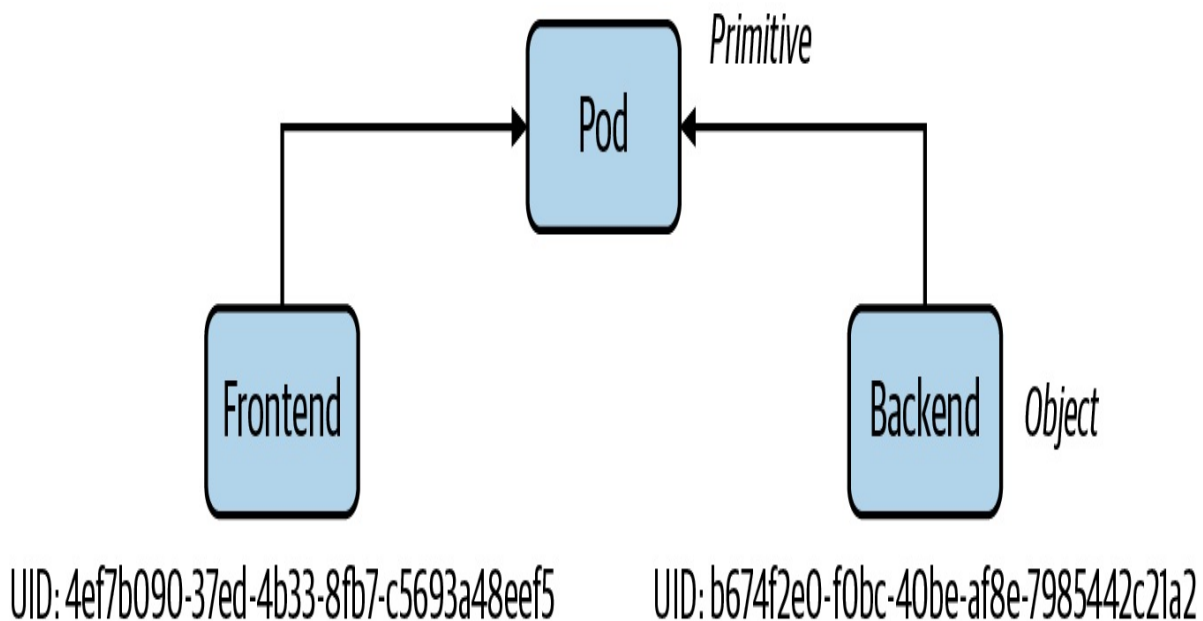


Figure 3-1. Kubernetes object identity

Every Kubernetes primitive follows a general structure, which you can observe if you look deeper at a manifest of an object, as shown in **Figure 3-2**. The primary markup language used for a Kubernetes manifest is YAML.

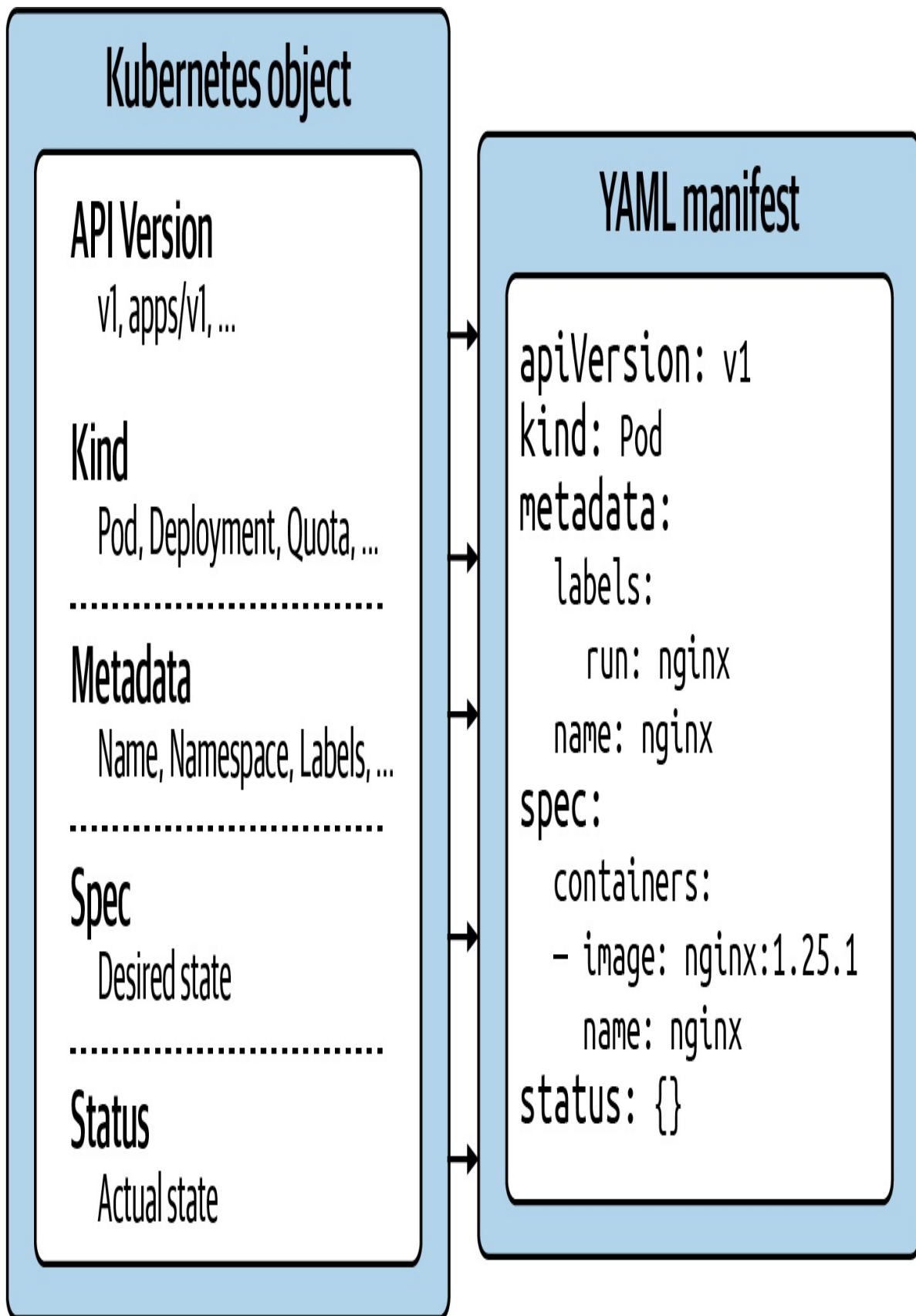


Figure 3-2. Kubernetes object structure

Let's look at each section and its relevance within the Kubernetes system:

API version

The Kubernetes API version defines the structure of a primitive and uses it to validate the correctness of the data. The API version serves a similar purpose as XML schemas to an XML document or JSON schemas to a JSON document. The version usually undergoes a maturity process—for example, from alpha to beta to final. Sometimes you see different prefixes separated by a slash (**apps**). You can list the API versions compatible with your cluster version by running the command `kubectl api-versions`.

Kind

The kind defines the type of primitive—e.g., a Pod or a Service. It ultimately answers the question, “What kinds of resource are we dealing with here?”

Metadata

Metadata describes higher-level information about the object—e.g., its name, what namespace it lives in, or whether it defines labels and annotations. This section also defines the UID.

Spec

The specification (“spec” for short) declares the desired state—e.g., how should this object look after it has been created? Which image should run in the container, or which environment variables should be set?

Status

The status describes the actual state of an object. The Kubernetes controllers and their reconciliation loops constantly try to transition a Kubernetes object from the desired state into the actual state. The object has not yet been materialized if the YAML status shows the value `{}`.

With this basic structure in mind, let's look at how to create a Kubernetes object with the help of `kubectl`.

Using kubectl

`kubectl` is the primary tool for interacting with the Kubernetes clusters from the command line. The exam is exclusively focused on the use of `kubectl`. Therefore, it's paramount to understand its ins and outs and practice its use heavily.

This section provides you with a brief overview of its typical usage pattern. Let's start by looking at the syntax for running commands. A `kubectl` execution consists of a command, a resource type, a resource name, and optional command line flags:

```
$ kubectl [command] [TYPE] [NAME] [flags]
```

The command specifies the operation you're planning to run. Typical commands are verbs like `create`, `get`, `describe`, or `delete`. Next, you'll need to provide the resource type you're working on, either as a full resource type or its short form. For example, you could work on a `service` here or use the short form, `svc`.

The name of the resource identifies the user-facing object identifier, effectively the value of `metadata.name` in the YAML representation. Be aware that the object name is not the same as the UID. The UID is an autogenerated, Kubernetes-internal object reference that you usually don't have to interact with. The name of an object has to be unique across all objects of the same resource type within a namespace.

Finally, you can provide zero to many command line flags to describe additional configuration behavior. A typical example of a command-line flag is the `--port` flag, which exposes a Pod's container port.

Figure 3-3 shows a full `kubectl` command in action.

The diagram illustrates the `kubectl` usage pattern. It shows the command `kubectl` followed by four components, each in square brackets and connected by spaces: `[command]`, `[TYPE]`, `[NAME]`, and `[flags]`. Below each bracketed component is a curly brace that groups the component with an example below it. The examples are: `get` under `[command]`, `pod` under `[TYPE]`, `app` under `[NAME]`, and `-o yaml` under `[flags]`.

Figure 3-3. `kubectl` usage pattern

Over the course of this book, we'll explore the `kubectl` commands that will make you the most productive during the exam. There are many more, however, and they usually go beyond the ones you'd use on a day-to-day basis as an application developer. Next up, we'll have a deeper look at the `create` command, the imperative way to create a Kubernetes object. We'll also compare the imperative object creation approach with the declarative approach.

Managing Objects

You can create objects in a Kubernetes cluster in two ways: imperatively or declaratively. The following sections describe each approach, including their benefits, drawbacks, and use cases.

Imperative Object Management

Imperative object management does not require a manifest definition. You'll use `kubectl` to drive the creation, modification, and deletion of objects with a single command and one or many command-line options. See the [Kubernetes documentation](#) for a more detailed description of imperative object management.

Creating objects

Use the `run` or `create` command to create an object on the fly. Any configuration needed at runtime is provided by command-line options. The benefit of this approach is the fast turnaround time without the need to wrestle with YAML structures. The following `run` command creates a Pod named `frontend` that executes the container image `nginx:1.24.0` in a container with the exposed port 80:

```
$ kubectl run frontend --image=nginx:1.24.0 --port=80
pod/frontend created
```

Updating objects

The configuration of live objects can still be modified. `kubectl` supports this use case by providing the `edit` or `patch` command.

The `edit` command opens an editor with the raw configuration of the live

object. Changes to the configuration will be applied to the live object after exiting the editor. The command will open the editor defined by the `KUBE_EDITOR`, or `EDITOR` environment variables, or fall back to `vi` for Linux or `notepad` for Windows. This command demonstrates the use of the `edit` command for the Pod live object named `frontend`:

```
$ kubectl edit pod frontend
```

The `patch` command allows for fine-grained modification of a live object on an attribute level using a JSON merge patch. The following example illustrates the use of `patch` command to update the container image tag assigned to the Pod created earlier. The `-p` flag defines the JSON structure used to modify the live object:

```
$ kubectl patch pod frontend -p '{"spec":{"containers":\n[{"name":"frontend",\n"image":"nginx:1.25.1"}]}}'\npod/frontend patched
```

Deleting objects

You can delete a Kubernetes object at any time. During the exam, the need may arise if you made a mistake while solving a problem and want to start from scratch to ensure a clean slate. In a production Kubernetes environment, you'll want to delete objects that are no longer needed. The following `delete` command deletes the Pod object by its name `frontend`:

```
$ kubectl delete pod frontend\npod "frontend" deleted
```

Upon execution of the `delete` command, Kubernetes tries to delete the targeted object gracefully so that there's minimal impact on the end user. If the object cannot be deleted within the default grace period (30 seconds), the kubelet attempts to forcefully kill the object.

During the exam, end user impact is not a concern. The most important goal is to complete all tasks in the time granted to the candidate. Therefore, waiting on an object to be deleted gracefully is a waste of time. You can force an immediate

deletion of an object with the command-line option with the `--now` option. The following command kills the Pod named `nginx` using a `SIGKILL` signal:

```
$ kubectl delete pod nginx --now
```

Declarative Object Management

Declarative object management requires one or several manifests in the format of YAML or JSON describing the desired state of an object. You create, update, and delete objects using this approach.

The benefit of using the declarative method is reproducibility and improved maintenance, as the file is checked into version control in most cases. The declarative approach is the recommended way to create objects in production environments.

More information on declarative object management can be found in the [Kubernetes documentation](#).

Creating objects

The declarative approach creates objects from a manifest (in most cases, a YAML file) using the `apply` command. The command works by pointing to a file, a directory of files, or a file referenced by an HTTP(S) URL using the `-f` option. If one or more of the objects already exist, the command will synchronize the changes made to the configuration with the live object.

To demonstrate the functionality, we'll assume the following directories and configuration files. The following commands create objects from a single file, from all files within a directory, and from all files in a directory recursively. Refer to files in the book's GitHub repository if you want to give it a try. Later chapters will explain the purpose of the primitives used here:

```
.
├── app-stack
│   ├── mysql-pod.yaml
│   ├── mysql-service.yaml
│   ├── web-app-pod.yaml
│   └── web-app-service.yaml
├── nginx-deployment.yaml
└── web-app
```

```

├── config
│   ├── db-configmap.yaml
│   └── db-secret.yaml
└── web-app-pod.yaml

```

Creating an object from a single file:

```

$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created

```

Creating objects from multiple files within a directory:

```

$ kubectl apply -f app-stack/
pod/mysql-db created
service/mysql-service created
pod/web-app created
service/web-app-service created

```

Creating objects from a recursive directory tree containing files:

```

$ kubectl apply -f web-app/ -R
configmap/db-config configured
secret/db-creds created
pod/web-app created

```

Creating objects from a file referenced by an HTTP(S) URL:

```

$ kubectl apply -f https://raw.githubusercontent.com/bmuschko/\
cka-study-guide/master/ch03/object-management/nginx-deployment.yaml
deployment.apps/nginx-deployment created

```

The `apply` command keeps track of the changes by adding or modifying the annotation with the key `kubectl.kubernetes.io/last-applied-configuration`. Here's an example of the annotation in the output of the `get pod` command:

```

$ kubectl get pod web-app -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |

```

```

        {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{}, \
        "labels":{"app":"web-app"},"name":"web-
app","namespace":"default"}, \
        "spec":{"containers":[{"envFrom":[{"configMapRef":{"name":"db-
config"}}, \
        {"secretRef":{"name":"db-creds"}}],"image":"bmuschko/web-
app:1.0.1", \
        "name":"web-app","ports":
[{"containerPort":3000,"protocol":"TCP"}]}, \
        "restartPolicy":"Always"}}
...

```

Updating objects

Updating an existing object is done with the same **apply** command. All you need to do is to change the configuration file and then run the command against it. **Example 3-1** modifies the existing configuration of a Deployment in the file `nginx-deployment.yaml`. We added a new label with the key `team` and changed the number of replicas from 3 to 5.

Example 3-1. Modified configuration file for a Deployment

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
    team: red
spec:
  replicas: 5
...

```

The following command applies the changed configuration file. As a result, the number of Pods controlled by the underlying ReplicaSet is 5:

```

$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment configured

```

The Deployment's `kubectl.kubernetes.io/last-applied-configuration` annotation reflects the latest change to the configuration:

```

$ kubectl get deployment nginx-deployment -o yaml
apiVersion: apps/v1
kind: Deployment

```

```

metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment","metadata":
{"annotations":{}, \
  "labels":{"app":"nginx","team":"red"},"name":"nginx-deployment",
\
  "namespace":"default"},"spec":{"replicas":5,"selector":
{"matchLabels": \
  {"app":"nginx"}}, "template":{"metadata":{"labels":
{"app":"nginx"}}, \
  "spec":{"containers":[{"image":"nginx:1.14.2","name":"nginx", \
    "ports":[{"containerPort":80}]}]}}}
...

```

Deleting objects

While you can delete objects using the `apply` command by providing the options `--prune -l <labels>`, it is recommended to delete an object using the `delete` command and point it to the configuration file. The following command deletes a Deployment and the objects it controls (ReplicaSet and Pods):

```

$ kubectl delete -f nginx-deployment.yaml
deployment.apps "nginx-deployment" deleted

```

You can use the `--now` option to forcefully delete Pods, as described in [“Deleting objects”](#).

Hybrid Approach

Sometimes you may want to go with a hybrid approach. You can start by using the imperative method to produce a manifest file without actually creating an object. You do so by executing the `run` or `create` command with the command-line options `-o yaml` and `--dry-run=client`:

```

$ kubectl run frontend --image=nginx:1.25.1 --port=80 \
  -o yaml --dry-run=client > pod.yaml

```

You can now use the generate YAML manifest as a starting point to make further modifications before creating the object. Simply open the file with an editor,

change the content, and execute the declarative `apply` command:

```
$ vim pod.yaml
$ kubectl apply -f pod.yaml
pod/frontend created
```

Which Approach to Use?

During the exam, using imperative commands is the most efficient and quick way to manage objects. Not all configuration options are exposed through command-line flags, which may force you into using the declarative approach. The hybrid approach can help here.

GITOPS AND KUBERNETES

GitOps is a practice that leverages source code checked into Git repositories to automate infrastructure management, specifically in cloud-native environments powered by Kubernetes. Tools such as **Argo CD** and **Flux** implement GitOps principles to deploy applications to Kubernetes through a declarative approach. Teams responsible for overseeing real-world Kubernetes clusters and the applications within them are highly likely to adopt the declarative approach.

While creating objects imperatively can optimize the turnaround time, in a real-world Kubernetes environment you'll most certainly want to use the declarative approach. A YAML manifest file represents the ultimate source of truth of a Kubernetes object. Version-controlled files can be audited and shared, and they store a history of changes in case you need to revert to a previous revision.

Summary

Kubernetes represents its functionality for deploying and operating a cloud-native application with the help of primitives. Each primitive follows a general structure: the API version, the kind, the metadata, and the desired state of the resources, also called the spec. Upon creation or modification of the object, the Kubernetes scheduler automatically tries to ensure that the actual state of the object follows the defined specification. Every live object can be inspected, edited, and deleted.

`kubectl` acts as a CLI-based client to interact with the Kubernetes cluster. You can use its commands and flags to manage Kubernetes objects. The imperative approach provides a fast turnaround time for managing objects with a single command, as long as you memorize the available flags. More complex configuration calls for the use of a YAML manifest to define a primitive. Use the declarative command to instantiate objects from that definition. The YAML manifest is usually checked into version control and offers a way to track changes to the configuration.

Part II. Cluster Architecture, Installation and Configuration

This domain explains the concepts you'd expect every Kubernetes administrator to be familiar with: the installation, maintenance, and configuration of the cluster. We'll discuss the processes, tools, and Kubernetes primitives and concepts involved by example.

The following chapters cover these concepts:

- **Chapter 4** expands on the discussion of architectural aspects applicable to a Kubernetes cluster. The content then provides an overview on the installation and update process for a Kubernetes cluster using `kubeadm`.
- **Chapter 5** focuses on a specific aspect of cluster maintenance: ensuring the data stored in `etcd` is backed up periodically and can be restored in case of a disaster that makes the data unreadable.
- **Chapter 6** first sets the stage on explaining the processing phases every API request has to go through before being handled. The chapter will then primarily concentrate on the authorization phase by explaining Role-Based Access Control (RBAC).
- **Chapter 7** touches on the Operator pattern, a concept used to extend the core functionality of the Kubernetes functionality. You will learn how to inspect and interact with existing Custom Resource Definitions (CRDs) as a schema for instantiating objects to implement custom requirements not natively covered by Kubernetes.
- **Chapter 8** discusses the use of the open source tools Helm and Kustomize for deploying more complex application stacks or cluster components.

Chapter 4. Cluster Installation and Upgrade

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles. This will be the 4th chapter of the final book. Please note that the GitHub repo will be made active later on. If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

According to the name of the chapter, the first section of the curriculum refers to typical tasks you’d expect of a Kubernetes administrator. Those tasks include understanding the architectural components of a Kubernetes cluster, setting up a cluster from scratch, and maintaining a cluster going forward. At the end of this chapter, you will understand the tools and procedures for installing and maintaining a Kubernetes cluster.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Prepare underlying infrastructure for installing a Kubernetes cluster
- Understand extension interfaces (CNI, CSI, CRI, etc.)
- Create and manage Kubernetes clusters using kubeadm
- Manage the lifecycle of Kubernetes clusters
- Implement and configure a highly-available control plane

Provisioning the Infrastructure

The infrastructure required by Kubernetes cluster nodes consist of servers, networks, and the storage. Provisioning of the infrastructure is done by configuring those resources on-premises or in the cloud optimally in an automated fashion. That's the purpose of infrastructure automation tools like Ansible and Terraform.

Even though “provisioning the infrastructure” has been mentioned in the curriculum exam, a detailed discussion of this topic would go beyond the scope of this book and isn't directly related to Kubernetes. If you are more interested in the topic, have a look at the book “**Infrastructure as Code**” by Kief Morris (O'Reilly Media).

Understanding Extension Interfaces

Chapter 2 laid the foundation for understanding the Kubernetes architecture, node types, and their cluster components. Kubernetes' architecture has been design to be flexible, modular, and extensible so that the core functionality of the platform can be expanded. You can extend the cluster using so-called extensions. Extensions are components that can be plugged into the platform to support custom functionality not native to Kubernetes. Those plugins usually include functions like network plugins, device plugins, and storage plugins.

In this section, we are going to talk about critical interfaces, you will need to understand as a Kubernetes administrator:

- **Container Network Interface (CNI)**
- **Container Runtime Interface (CRI)**
- **Container Storage Interface (CSI)**

Container Network Interface (CNI)

The CNI is a specification and the corresponding libraries for writing plugins to configure network interfaces in Linux containers. In Kubernetes, it is responsible for establishing network connections between Pods running in the cluster. You

will need to install a CNI plugin in the control plane node(s) for them to function properly. “[Installing a Pod Network Add-on](#)” demonstrates the installation process of a CNI.

Container Runtime Interface (CRI)

The container runtime is responsible for managing the lifecycle of containers and ensuring that containers receive the resources they requested. Kubernetes uses containerd as its default container runtime, however, you can swap it out with a different container runtime if needed. The CRI is an API facilitating the interaction between Kubernetes and different container runtimes.

Container Storage Interface (CSI)

Although Kubernetes provides a Volume plugin system, it was hard to integrate third-party storage solutions. In response, CSI was developed to as a standard to implement plugins for integrating arbitrary block and file storage systems to containerized workloads.

Using kubeadm

The low-level command-line tool for performing cluster bootstrapping operations is called kubeadm. It is not meant for provisioning the underlying infrastructure.

To install kubeadm, follow the [installation instructions](#) in the official Kubernetes documentation. While not explicitly stated in the CKA frequently asked questions (FAQ) page, you can assume that the kubeadm executable has been preinstalled for you.

The following sections describe the processes for creating and managing a Kubernetes cluster on a high level and will use kubeadm heavily. For more detailed information, see the step-by-step Kubernetes reference documentation I will point out for each of the tasks.

Installing a Cluster

The most basic topology of a Kubernetes cluster consists of a single node that acts as the control plane and the worker node at the same time. By default, many developer-centric Kubernetes installations like minikube or Docker Desktop start with this configuration. While a single-node cluster may be a good option for a Kubernetes playground, it is not a good foundation for scalability and high-availability reasons. At the very least, you will want to create a cluster with a single control plane and one or many nodes handling the workload.

This section explains how to install a cluster with a single control plane and one worker node. You can repeat the worker node installation process to add more worker nodes to the cluster. You can find a full description of the **installation steps** in the official Kubernetes documentation. **Figure 4-1** illustrates the installation process.

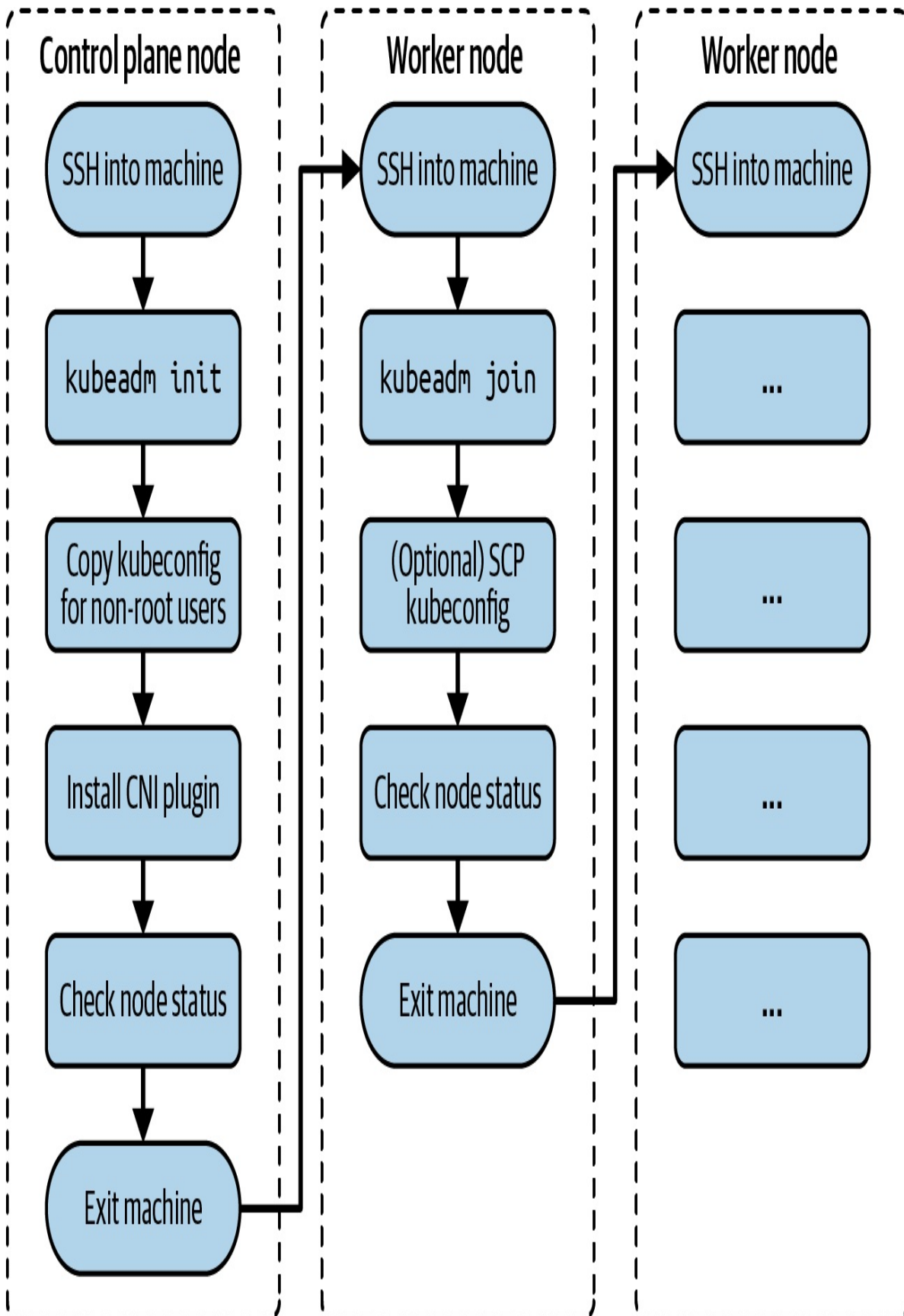


Figure 4-1. Process for a cluster installation process

Initializing the Control Plane Node

Start by initializing the control plane on the control plane node. The control plane is the machine responsible for hosting the API server, etcd, and other components important to managing the Kubernetes cluster.

Open an interactive shell to the control plane node using the `ssh` command. The following command targets the control plane node named `kube-control-plane` running Ubuntu 24.10:

```
$ ssh kube-control-plane
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-8-generic aarch64)
...
```

Initialize the control plane using the `kubeadm init` command. You will need to add the following two command-line options: provide the IP addresses for the Pod network with the option `--pod-network-cidr`. With the option `--apiserver-advertise-address`, you can declare the IP address the API Server will advertise to listen on.

By default, `kubeadm` uses its own version to determine the control plane node's version. For example, if you use `kubeadm` version 1.31.1 then it will use version 1.31.1 for the initialized node. You can provide the desired Kubernetes version by providing the `--kubernetes-version` option, though it is recommended to use the `kubeadm` version you want to use for the nodes.

The console output renders a `kubeadm join` command. Keep that command around for later. It is important for joining worker nodes to the cluster in a later step.

RETRIEVING THE JOIN COMMAND FOR WORKER NODES

You can always retrieve the `join` command by running `kubeadm token create --print-join-command` on the control plane node should you lose it.

The following command uses `10.244.0.0/16` for the Classless Inter-Domain Routing (CIDR):

```
$ sudo kubeadm init --pod-network-cidr=10.244.0.0/16
```

```
...
```

To start using your cluster, you need to run the following as a regular user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

You should now deploy a pod network to the cluster.

Run `"kubectl apply -f [podnetwork].yaml"` with one of the options listed at:

<https://kubernetes.io/docs/concepts/cluster-administration/addons/>

Then you can join any number of worker nodes by running the following on \ each as root:

```
kubeadm join 172.16.0.5:6443 --token fi8io0.dtkzsy9kws56dmsp \
--discovery-token-ca-cert-hash \
sha256:cc89ea1f82d5ec460e21b69476e0c052d691d0c52cce83fbd7e403559c1ebda
c
```

After the `init` command has finished, run the necessary commands from the console output to start the cluster as nonroot user:

```
$ mkdir -p $HOME/.kube
$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Installing a Pod Network Add-on

You must deploy a **Container Network Interface (CNI) plugin** so that Pods can communicate with each other. You can pick from a wide range of networking plugins listed in the **Kubernetes documentation**. Popular plugins include Flannel, Calico, and Cilium. Sometimes you will see the term “add-ons” in the documentation, which is synonymous with plugin.

The exam will most likely ask you to install a specific add-on. Most of the installation instructions live on external web pages, not permitted to be used

during the exam. Make sure that you search for the relevant instructions in the official Kubernetes documentation. For example, you can find the installation instructions for Flannel [here](#).

Installing Flannel using kubectl

The following command installs the Flannel objects via the released YAML manifest.

```
$ kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
```

The YAML manifest hard-codes the Pod CIDR to the value `10.244.0.0/16`. If you want to change that value, you'd have to download the YAML manifest first, edit the corresponding value, and then apply the manifest.

Installing Flannel using Helm

You may decide to install Flannel using Helm instead, either because you prefer the installation method or because you want to more conveniently provide a custom Pod CIDR. Helm offers the `--set` option to inject the custom value. The following command shows the use of Helm to install Flannel. Refer to [“Working with Helm”](#) to learn more about the use of Helm.

```
$ kubectl create ns kube-flannel
$ kubectl label --overwrite ns kube-flannel pod-security.kubernetes.io/enforce=privileged
$ helm repo add flannel https://flannel-io.github.io/flannel/
$ helm install flannel --set podCidr="10.244.0.0/16" --namespace kube-flannel \
  flannel/flannel
namespace/kube-flannel created
namespace/kube-flannel labeled
"flannel" has been added to your repositories
NAME: flannel
```

LAST DEPLOYED: Wed Jan 29 23:03:33 2025
NAMESPACE: kube-flannel
STATUS: deployed
REVISION: 1
TEST SUITE: None

Both installation methods, the YAML manifest and the Helm chart, create the Flannel Pod into the namespace `kube-flannel`. Wait until the Pod observes the “Running” status. You can check on the Pod’s details with the command below.

```
$ kubectl get pods -n kube-flannel
```

NAME	READY	STATUS	RESTARTS	AGE
kube-flannel-ds-h6455	1/1	Running	0	25s

Verify that the control plane node indicates the “Ready” status using the command `kubectl get nodes`. It might take a couple of seconds before the node transitions from the “NotReady” status to the “Ready” status. You have an issue with your node installation in case the status transition does not occur. Refer to [Link to Come] for debugging strategies:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kube-control-plane	Ready	control-plane	5m31s	v1.31.1

Exit the control plane node using the `exit` command:

```
$ exit  
logout  
...
```

Joining the Worker Nodes

Worker nodes are responsible for handling the workload scheduled by the control plane. Examples of workloads are Pods, Deployments, Jobs, and CronJobs. To add a worker node to the cluster so that it can be used, you will have to run a couple of commands, as described next.

Open an interactive shell to the worker node using the `ssh` command. The following command targets the worker node named `kube-worker-1` running

Ubuntu Ubuntu 24.10:

```
$ ssh kube-worker-1
```

```
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-8-generic aarch64)
```

```
...
```

Run the `kubeadm join` command provided by the `kubeadm init` console output on the control plane node. The following command shows an example. Remember that the token and SHA256 hash will be different for you:

```
$ sudo kubeadm join 172.16.0.5:6443 --token fi8io0.dtkzsy9kws56dmsp \  
--discovery-token-ca-cert-hash \
```

```
sha256:cc89ea1f82d5ec460e21b69476e0c052d691d0c52cce83fbd7e403559c1ebda
```

```
[preflight] Running pre-flight checks  
[preflight] Reading configuration from the cluster...  
[preflight] FYI: You can look at this config file with \  
'kubectl -n kube-system get cm kubeadm-config -o yaml'  
[kubelet-start] Writing kubelet configuration to file \  
"/var/lib/kubelet/config.yaml"  
[kubelet-start] Writing kubelet environment file with \  
flags to file "/var/lib/kubelet/kubeadm-flags.env"  
[kubelet-start] Starting the kubelet  
[kubelet-start] Waiting for the kubelet to perform the TLS  
Bootstrap...
```

```
This node has joined the cluster:
```

```
* Certificate signing request was sent to apiserver and a response was  
received.
```

```
* The Kubelet was informed of the new secure connection details.
```

```
Run 'kubectl get nodes' on the control plane to see this node join the  
cluster.
```

You won't be able to run the `kubectl get nodes` command from the worker node without copying the administrator kubeconfig file from the control plane node. Follow the [instructions](#) in the Kubernetes documentation to do so or log back into the control plane node. Here, we are just going to log back into the control plane node. You should see that the worker node has joined the cluster and is in a "Ready" status:

```
$ ssh kube-control-plane
```

```
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-8-generic aarch64)
```

```
...
$ kubectl get nodes
NAME                STATUS    ROLES    AGE     VERSION
kube-control-plane  Ready    control-plane  2m14s   v1.31.1
kube-worker-1       Ready    <none>       6m43s   v1.31.1
```

You can repeat the process for any other worker node you want to add to the cluster.

Managing a Highly Available Cluster

Single control plane clusters are easy to install; however, they present an issue when the node is lost. Once the control plane node becomes unavailable, any ReplicaSet running on a worker node cannot re-create a Pod due to the inability to talk back to the scheduler running on a control plane node. Moreover, clusters cannot be accessed externally anymore (e.g., via `kubectl`), as the API server cannot be reached.

High-availability (HA) clusters help with scalability and redundancy. For the exam, you will need to have a basic understanding about configuring them and their implications. Given the complexity of standing up an HA cluster, it's unlikely that you'll be asked to perform the steps during the exam. For a full discussion on setting up HA clusters, see the [relevant page](#) in the Kubernetes documentation.

Stacked etcd Topology

The *stacked etcd topology* involves creating two or more control plane nodes where etcd is colocated on the node. [Figure 4-2](#) shows a representation of the topology with three control plane nodes.

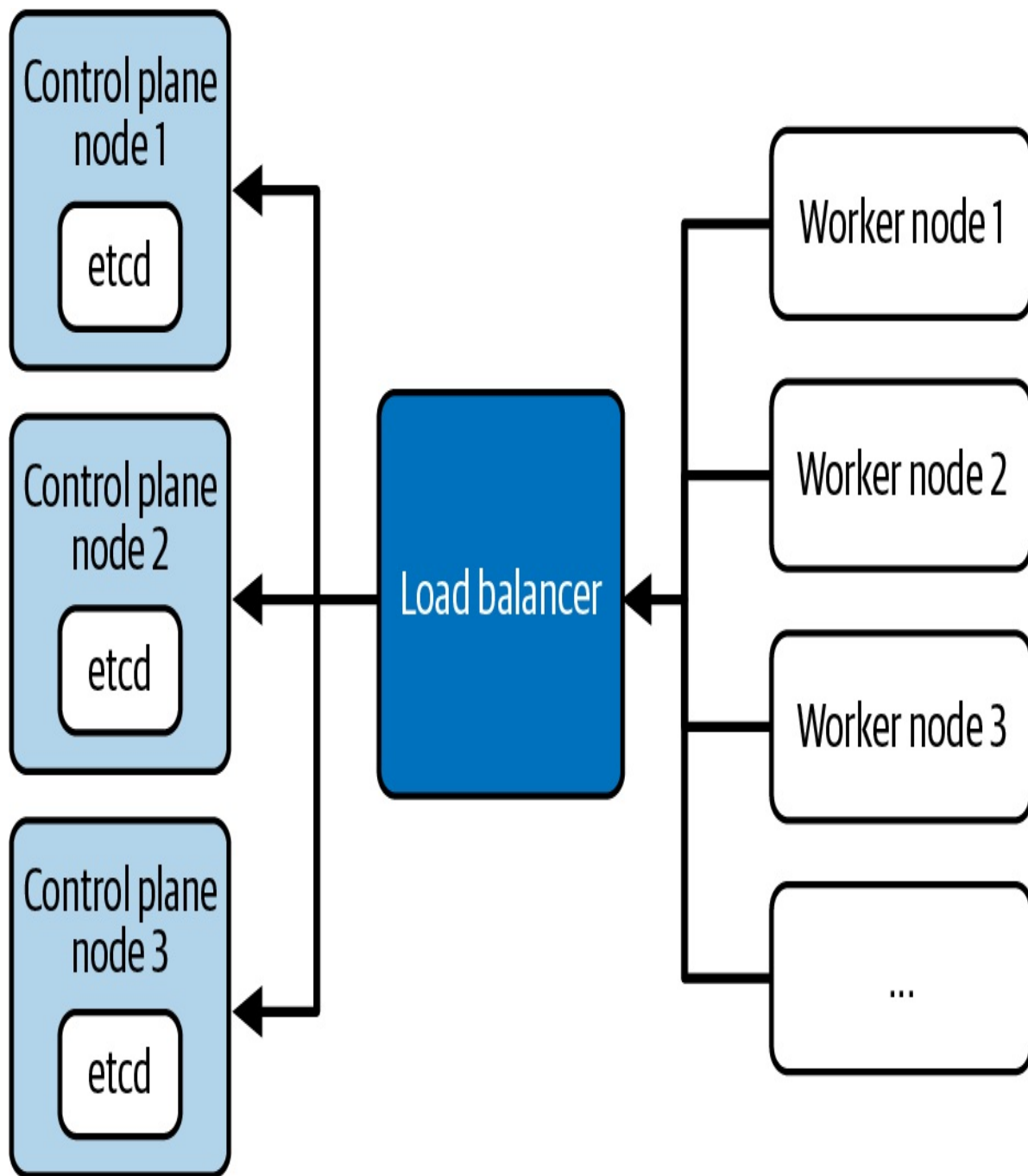


Figure 4-2. Stacked etcd topology with three control plane nodes

Each of the control plane nodes hosts the API server, the scheduler, and the controller manager. Worker nodes communicate with the API server through a load balancer. It is recommended to operate this cluster topology with a minimum of three control plane nodes for redundancy reasons due to the tight coupling of etcd to the control plane node. By default, `kubeadm` will create an etcd instance when joining a control plane node to the cluster.

External etcd Node Topology

The *external etcd node* topology separates etcd from the control plane node by running it on a dedicated machine. **Figure 4-3** shows a setup with three control plane nodes, each of which run etcd on a different machine.

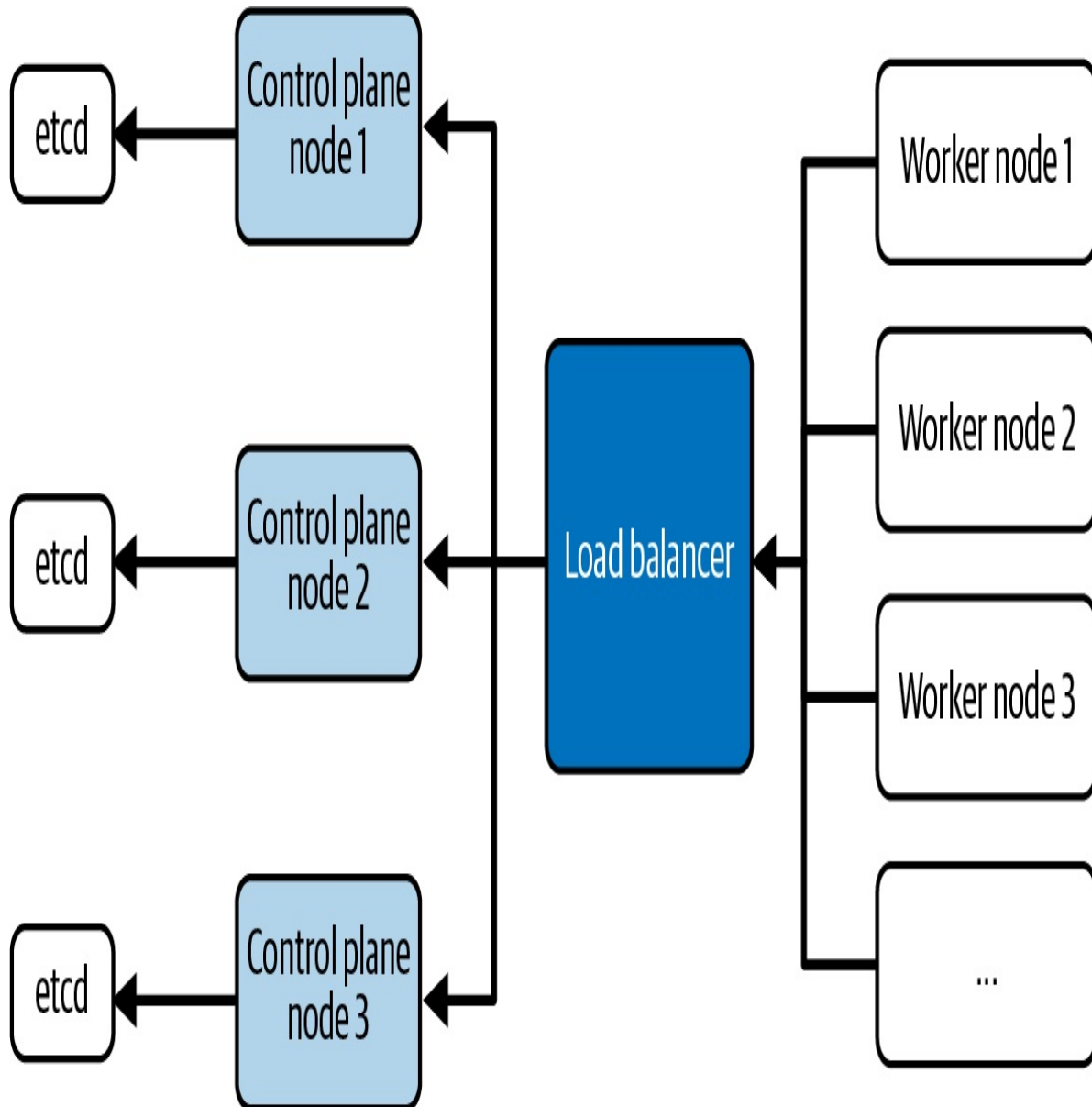


Figure 4-3. External etcd node topology

Similar to the stacked etcd topology, each control plane node hosts the API server, the scheduler, and the controller manager. The worker nodes communicate with them through a load balancer. The main difference here is that the etcd instances run on a separate host. This topology decouples etcd from

other control plane functionality and therefore has less of an impact on redundancy when a control plane node is lost. As you can see in the illustration, this topology requires twice as many hosts as the stacked etcd topology.

Upgrading a Cluster Version

Over time, you will want to upgrade the Kubernetes version of an existing cluster to pick up bug fixes and new features. The upgrade process has to be performed in a controlled manner to avoid the disruption of workload currently in execution and to prevent the corruption of cluster nodes.

It is recommended to upgrade from a minor version to a next higher one (e.g., from 1.18.0 to 1.19.0), or from a patch version to a higher one (e.g., from 1.18.0 to 1.18.3). Abstain from jumping up multiple minor versions to avoid unexpected side effects. You can find a full description of the **upgrade steps** in the official Kubernetes documentation. **Figure 4-4** illustrates the upgrade process.

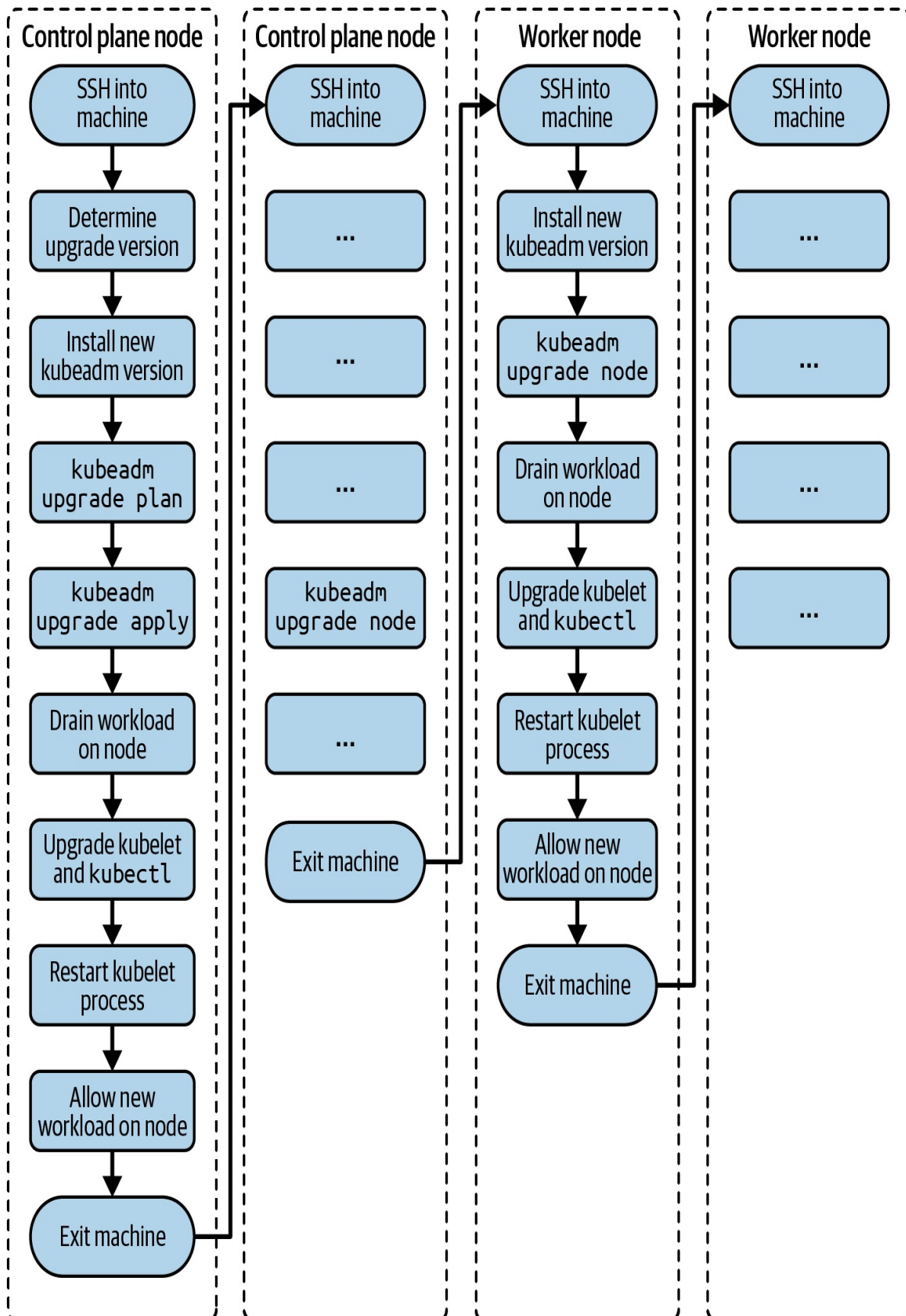


Figure 4-4. Process for a cluster version upgrade

Upgrading control plane nodes

As explained earlier, a Kubernetes cluster may employ one or many control plane nodes to better support high-availability and scalability concerns. When upgrading a cluster version, this change needs to happen for control plane nodes one at a time.

Pick one of the control plane nodes that contains the kubeconfig file (located at `/etc/kubernetes/admin.conf`), and open an interactive shell to the control plane node using the `ssh` command. The following command targets the control plane node named `kube-control-plane` running Ubuntu 24.10:

```
$ ssh kube-control-plane
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-8-generic aarch64)
...
```

First, check the nodes and their Kubernetes versions. In this setup, all nodes run on version 1.31.1. We are dealing with only a single control plane node and a single worker node:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kube-control-plane	Ready	control-plane	4m54s	v1.31.1
kube-worker-1	Ready	<none>	3m18s	v1.31.1

Start by upgrading the `kubeadm` version. Identify the version you'd like to upgrade to. On Ubuntu machines, you can use the following `apt-get` command. The version format usually includes a patch version (e.g., `1.20.7-00`). Check the Kubernetes documentation if your machine is running a different operating system:

```
$ sudo apt update
...
$ sudo apt-cache madison kubeadm
kubeadm | 1.31.5-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb
Packages
kubeadm | 1.31.4-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb
Packages
```

```

    kubeadm | 1.31.3-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb
Packages
    kubeadm | 1.31.2-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb
Packages
    kubeadm | 1.31.1-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb
Packages
    kubeadm | 1.31.0-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb
Packages

```

Upgrade kubeadm to a target version. Say you'd want to upgrade to version **1.31.5-1.1**. The following series of commands installs kubeadm with that specific version and checks the currently installed version to verify:

```

$ sudo apt-mark unhold kubeadm && sudo apt-get update && sudo apt-get
install \
    -y kubeadm=1.31.5-1.1 && sudo apt-mark hold kubeadm
Canceled hold on kubeadm.
...
Unpacking kubeadm (1.31.5-1.1) over (1.31.1-1.1) ...
Setting up kubeadm (1.31.5-1.1) ...
kubeadm set on hold.
$ sudo apt-get update && sudo apt-get install -y --allow-change-held-
packages \
    kubeadm=1.31.5-1.1
...
kubeadm is already the newest version (1.31.5-1.1).
0 upgraded, 0 newly installed, 0 to remove and 94 not upgraded.
$ kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"31",
GitVersion:"v1.31.5", \
GitCommit:"af64d838aacd9173317b39cf273741816bd82377",
GitTreeState:"clean", \
BuildDate:"2025-01-15T14:39:21Z", GoVersion:"go1.22.10",
Compiler:"gc", \
Platform:"linux/arm64"}

```

Check which versions are available to upgrade to and validate whether your current cluster is upgradable. You can see in the output of the following command that we could upgrade to version **1.31.5**.

```

$ sudo kubeadm upgrade plan
...
[upgrade] Fetching available versions to upgrade to
[upgrade/versions] Cluster version: 1.31.5
[upgrade/versions] kubeadm version: v1.31.5

```

```
I0130 22:26:53.887541 13574 version.go:261] remote version is \
much newer: v1.32.1; falling back to: stable-1.31
[upgrade/versions] Target version: v1.31.5
[upgrade/versions] Latest version in the v1.31 series: v1.31.
```

As described in the console output, we'll start the upgrade for the control plane. The process may take a couple of minutes. You may have to upgrade the CNI plugin as well. Follow the provider instructions for more information:

```
$ sudo kubeadm upgrade apply v1.31.5
...
[upgrade/version] You have chosen to change the cluster version to
"v1.31.5"
[upgrade/versions] Cluster version: v1.31.5
[upgrade/versions] kubeadm version: v1.31.5
...
[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.31.5".
Enjoy!

[upgrade/kubelet] Now that your control plane is upgraded, please
proceed \
with upgrading your kubelets if you haven't already done so.
```

Drain the control plane node by evicting the workload. Any new workload won't be schedulable on the node until uncordoned:

```
$ kubectl drain kube-control-plane --ignore-daemonsets
node/kube-control-plane cordoned
WARNING: ignoring DaemonSet-managed Pods: kube-system/calico-node-
qndb9, \
kube-system/kube-proxy-vpvm5
evicting pod kube-system/calico-kube-controllers-65f8bc95db-krp72
evicting pod kube-system/coredns-f9fd979d6-2brkq
pod/calico-kube-controllers-65f8bc95db-krp72 evicted
pod/coredns-f9fd979d6-2brkq evicted
node/kube-control-plane evicted
```

Upgrade the kubelet and the kubectl tool to the same version:

```
$ sudo apt-mark unhold kubelet kubectl && sudo apt-get update && sudo \
apt-get install -y kubelet=1.31.5-1.1 kubectl=1.31.5-1.1 && sudo \
apt-mark \
hold kubelet kubectl
...
```

```
Setting up kubelet (1.31.5-1.1) ...
Setting up kubect1 (1.31.5-1.1) ...
kubelet set on hold.
kubect1 set on hold.
```

Restart the kubelet process:

```
$ sudo systemctl daemon-reload
$ sudo systemctl restart kubelet
```

Reenable the control plane node back so that the new workload can become schedulable:

```
$ kubectl uncordon kube-control-plane
node/kube-control-plane uncordoned
```

The control plane nodes should now show the usage of Kubernetes 1.31.5:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kube-control-plane	Ready	control-plane	21h	v1.31.5
kube-worker-1	Ready	<none>	21h	v1.31.1

Exit the control plane node using the `exit` command:

```
$ exit
logout
...
```

Upgrading worker nodes

Pick one of the worker nodes, and open an interactive shell to the node using the `ssh` command. The following command targets the worker node named `kube-worker-1` running Ubuntu 24.10:

```
$ ssh kube-worker-1
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-8-generic aarch64)
...
```

Upgrade `kubeadm` to a target version. This is the same command you used for the control plane node, as explained earlier:

```

$ sudo apt-mark unhold kubeadm && sudo apt-get update && sudo apt-get
install \
  -y kubeadm=1.31.5-1.1 && sudo apt-mark hold kubeadm
Canceled hold on kubeadm.
...
Unpacking kubeadm (1.31.5-1.1) over (1.31.1-1.1) ...
Setting up kubeadm (1.31.5-1.1) ...
kubeadm set on hold.
$ kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"31",
GitVersion:"v1.31.5", \
GitCommit:"af64d838aacd9173317b39cf273741816bd82377",
GitTreeState:"clean", \
BuildDate:"2025-01-15T14:39:21Z", GoVersion:"go1.22.10",
Compiler:"gc", \
Platform:"linux/arm64"}

```

Upgrade the kubelet configuration:

```

$ sudo kubeadm upgrade node
[upgrade] Reading configuration from the cluster...
[upgrade] FYI: You can look at this config file with \
'kubectl -n kube-system get cm kubeadm-config -o yaml'
[preflight] Running pre-flight checks
[preflight] Skipping prepull. Not a control plane node.
[upgrade] Skipping phase. Not a control plane node.
[upgrade] Skipping phase. Not a control plane node.
[upgrade] Backing up kubelet config file to \
/etc/kubernetes/tmp/kubeadm-kubelet-config3058962439/config.yaml
[kubelet-start] Writing kubelet configuration to file \
"/var/lib/kubelet/config.yaml"
[upgrade] The configuration for this node was successfully updated!
[upgrade] Now you should go ahead and upgrade the kubelet package \
using your package manager.

```

Drain the worker node by evicting the workload. Any new workload won't be schedulable on the node until uncordoned:

```

$ kubectl drain kube-worker-1 --ignore-daemonsets
node/kube-worker-1 cordoned
WARNING: ignoring DaemonSet-managed Pods: kube-system/calico-node-
2hrxg, \
kube-system/kube-proxy-qf6n1
evicting pod kube-system/calico-kube-controllers-65f8bc95db-kggbr
evicting pod kube-system/coredns-f9fd979d6-7zm4q
evicting pod kube-system/coredns-f9fd979d6-tlmhq
pod/calico-kube-controllers-65f8bc95db-kggbr evicted

```

```
pod/coredns-f9fd979d6-7zm4q evicted
pod/coredns-f9fd979d6-tlmhq evicted
node/kube-worker-1 evicted
```

Upgrade the kubelet and the `kubect1` tool with the same command used for the control plane node:

```
$ sudo apt-mark unhold kubelet kubect1 && sudo apt-get update && sudo
apt-get \
  install -y kubelet=1.31.5-1.1 kubect1=1.31.5-1.1 && sudo apt-mark
hold kubelet \
  kubect1
...
Setting up kubelet (1.31.5-1.1) ...
Setting up kubect1 (1.31.5-1.1) ...
kubelet set on hold.
kubect1 set on hold.
```

Restart the kubelet process:

```
$ sudo systemctl daemon-reload
$ sudo systemctl restart kubelet
```

Reenable the worker node so that the new workload can become schedulable:

```
$ kubect1 uncordon kube-worker-1
node/kube-worker-1 uncordoned
```

Listing the nodes should now show version 1.31.5 for the worker node. You won't be able to run `kubect1 get nodes` from the worker node without copying the administrator kubeconfig file from the control plane node. Follow the [instructions](#) in the Kubernetes documentation to do so or log back into the control plane node:

```
$ kubect1 get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kube-control-plane	Ready	control-plane	24h	v1.31.5
kube-worker-1	Ready	<none>	24h	v1.31.5

Exit the worker node using the `exit` command:

```
$ exit
```

logout

...

Summary

As a Kubernetes administrator, you need to be familiar with typical tasks involving the management of the cluster nodes. The primary tool for installing new nodes and upgrading a node version is `kubeadm`. The cluster topology of such a cluster can vary. For optimal results with redundancy and scalability, consider configuring the cluster with a high-availability setup that uses three or more control plane nodes and dedicated etcd hosts.

Exam Essentials

Understand why you need to provision infrastructure

Every cluster node needs to run on a physical or virtual machine. Provisioning the hardware is the job of an administrator, though you will not have to have hands-on experience for the exam. Explore the manual and automated approaches for provisioning hardware as you will need it for installing a cluster.

Know how to create a Kubernetes cluster from scratch

Installing new cluster nodes and upgrading the version of an existing cluster node are typical tasks performed by a Kubernetes administrator. You do not need to memorize all the steps involved. The documentation provides a step-by-step, easy-to-follow manual for those operations. During the exam, pull the relevant documentation and copy-paste the commands.

Practice the cluster upgrade process

The cluster upgrade process involves executing more commands than the installation process. It's important to remember that you only jump up by a single minor version or multiple patch versions before tackling the next higher version. I'd suggest you open the upgrade documentation page, and walk through the process a couple of times.

Have a theoretical understanding of high-availability cluster topologies

High-availability clusters help with redundancy and scalability. For the exam, you will need to understand the different HA topologies though it's unlikely that you'll have to configure one of them as the process would involve a suite of different hosts.

Sample Exercises

Solutions to these exercises are available in the [Appendix A](#).

1. Create a cluster with four nodes, one control plane node, and three worker nodes.

Create a Pod named `nginx` that uses the container image `nginx:1.27.4-alpine`.

Identify the node the Pod has been scheduled on.

Evict all Pods from the node that runs the Pod at once. Do not use the `kubectl delete pod` command to perform the operation. Ensure that the Pod is not running anymore.

Prerequisite: To create a cluster with 4 nodes using minikube, run the command `minikube start --nodes 4`.

2. Navigate to the directory `app-a/ch04/upgrade-cluster-version` of the checked-out GitHub repository [bmuschko/cka-study-guide](#).

Start up the VMs running the cluster using the command `vagrant up`. The cluster consists of a single control plane node named `kube-control-plane`, and one worker node named `kube-worker-1`. Open an interactive shell into the control plane node and inspect the currently-used Kubernetes version by listing all the nodes.

Upgrade all nodes of the cluster from Kubernetes 1.32.1 to 1.32.2.

Once done, shut down the cluster using `vagrant destroy -f`.

Prerequisite: This exercise requires the installation of the tools [Vagrant](#) and a [VMware provider](#).

Chapter 5. Backing Up and Restoring etcd

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles. This will be the 5th chapter of the final book. Please note that the GitHub repo will be made active later on. If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Kubernetes stores both the declared and observed states of the cluster in the distributed etcd key-value store. It’s important to have a backup plan in place that can help you with restoring the data in case of data corruption. Backing up the data should happen periodically in short time frames to avoid losing as little historical data as possible.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Manage the lifecycle of Kubernetes clusters

Using the etcd Administration Utilities

The backup process stores the etcd data in a so-called snapshot file. This snapshot file can be used to restore the etcd data at any given time. You can encrypt the snapshot file to protect sensitive information. The tool `etcdctl` is used to create a backup snapshot file. Restoring the etcd data from the snapshot

file requires to the use of the tool `etcdctl`.

As an administrator, you will need to understand how to use the tools for both operations. You may need to install `etcdctl` and `etcdutl` if they are not available on the control plane node yet. You can find [installation instructions](#) in the etcd GitHub repository. [Figure 5-1](#) visualizes the etcd backup and restoration process.

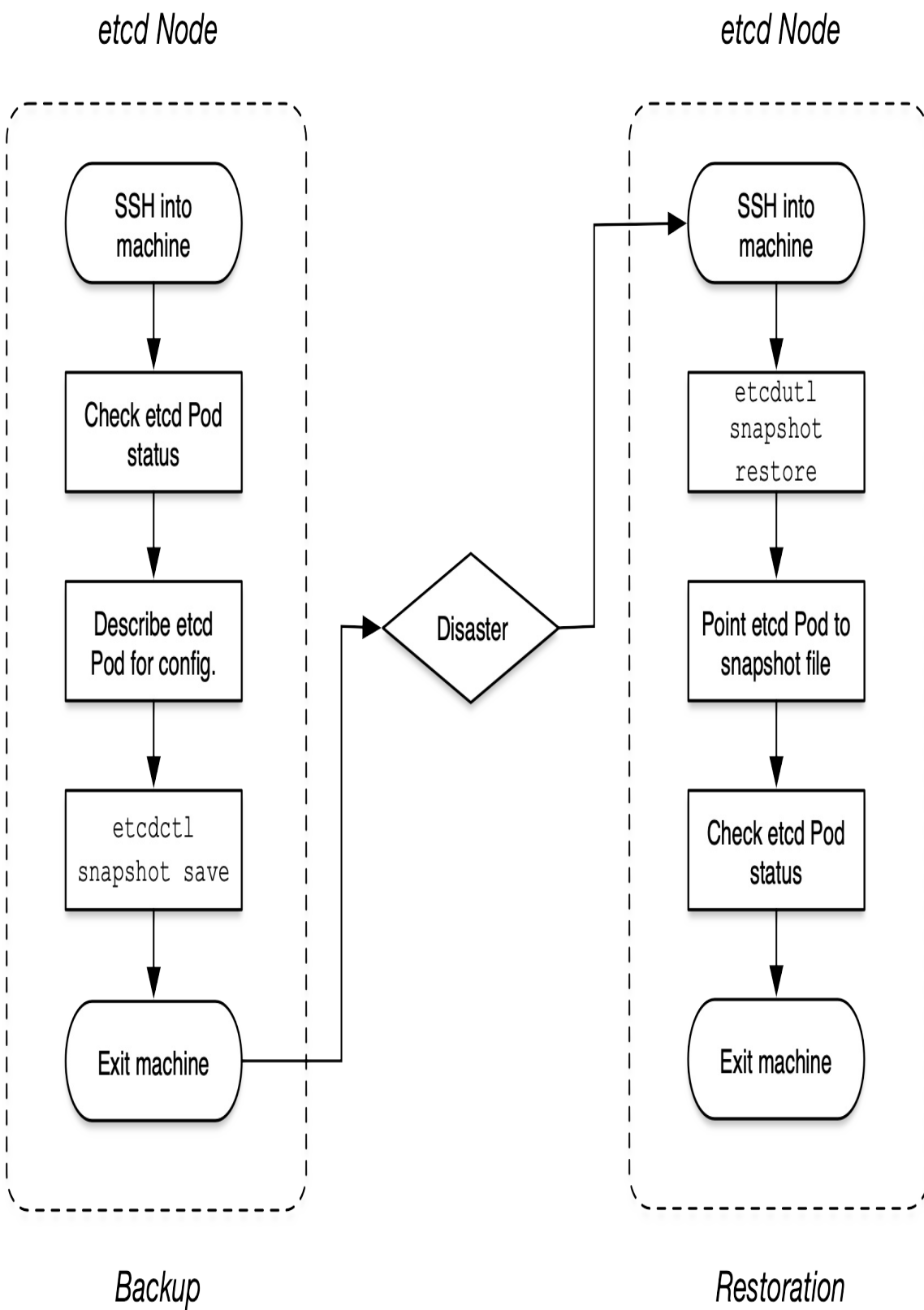


Figure 5-1. Process for a backing up and restoring etcd

Depending on your cluster topology, your cluster may consist of one or many etcd instances. Refer to “[Managing a Highly Available Cluster](#)” for more information on how to set it up. The following sections explain a single-node etcd cluster setup. You can find [additional instructions](#) on the backup and restoration process for multinode etcd clusters in the official Kubernetes documentation.

Backing Up etcd

Open an interactive shell to the machine hosting etcd using the `ssh` command. The following command targets the control plane node named `kube-control-plane` running Ubuntu 24.04 LTS:

```
$ ssh kube-control-plane
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-51-generic x86_64)
...
```

Check the installed version of `etcdctl` to verify that the tool has been installed. On this node, the version is 3.5.15:

```
$ etcdctl version
etcdctl version: 3.5.15
API version: 3.5
```

Etcd is deployed as a Pod in the `kube-system` namespace. Inspect the version by describing the Pod. In the following output, you will find that the version is 3.5.15-0:

```
$ kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS
AGE
...
etcd-kube-control-plane            1/1     Running   0
33m
...
$ kubectl describe pod etcd-kube-control-plane -n kube-system
...
Containers:
```

```

etcd:
  Container ID:
containerd://47a6cf3ed27d455be6c9b782d2e35ee77b429ee5c0b \
3c6c3d6282628f6492b15
  Image:          registry.k8s.io/etcd:3.5.15-0
  Image ID:
registry.k8s.io/etcd@sha256:a6dc63e6e8cfa0307d7851762fa6 \
b629afb18f28d8aa3fab5a6e91b4af60026a
...

```

The same `describe` command reveals the configuration of the etcd service. Look for the value of the option `--listen-client-urls` for the endpoint URL. In the following output, the host is `localhost`, and the port is `2379`. The server certificate is located at `/etc/kubernetes/pki/etcd/server.crt` defined by the option `--cert-file`. The CA certificate can be found at `/etc/kubernetes/pki/etcd/ca.crt` specified by the option `--trusted-ca-file`:

```

$ kubectl describe pod etcd-kube-control-plane -n kube-system
...
Containers:
  etcd:
    ...
    Command:
      etcd
    ...
    --cert-file=/etc/kubernetes/pki/etcd/server.crt
    --key-file=/etc/kubernetes/pki/etcd/server.key
    --listen-client-urls=/etc/kubernetes/pki/etcd/server.key
    --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
...

```

Use the `etcdctl` command to create the backup with version 3 of the tool. For a good starting point, copy the command from the [official Kubernetes documentation](#). Provide the mandatory command-line options `--cacert`, `--cert`, and `--key`. The option `--endpoints` is not needed as we are running the command on the same server as etcd. After running the command, the file `/opt/etcd-backup.db` has been created:

```

$ sudo ETCDCTL_API=3 etcdctl --cacert=/etc/kubernetes/pki/etcd/ca.crt \
\

```

```
--cert=/etc/kubernetes/pki/etcd/server.crt \  
--key=/etc/kubernetes/pki/etcd/server.key \  
snapshot save /opt/etcd-backup.db  
...  
Snapshot saved at /opt/etcd-backup.db
```

Exit the node using the `exit` command:

```
$ exit  
logout  
...
```

Restoring etcd

You created a backup of etcd and stored it in a safe space. There's nothing else to do at this time. Effectively, it's your insurance policy that becomes relevant when disaster strikes. In the case of a disaster scenario, the data in etcd gets corrupted or the machine managing etcd experiences a physical storage failure. That's the time when you want to pull out the etcd backup for restoration.

To restore etcd from the backup, use the `etcdctl snapshot restore` command. At a minimum, provide the `--data-dir` command-line option.

The `snapshot restore` command for the `etcdctl` executable has been deprecated and will be **removed** with the release of etcd 3.6. Use the `etcdctl` executable instead.

Here, we are using the data directory `/tmp/from-backup`. After running the command, you should be able to find the restored backup in the directory `/var/lib/from-backup`:

```
$ sudo ETCDCTL_API=3 etcdctl --data-dir=/var/lib/from-backup snapshot  
restore \  
/opt/etcd-backup.db  
...  
$ sudo ls /var/lib/from-backup  
member
```

Edit the YAML manifest of the etcd Pod, which can be found at `/etc/kubernetes/manifests/etcd.yaml`. Change the value of the attribute `spec.volumes.hostPath` with the name `etcd-data` from the

original value `/var/lib/etcd` to `/var/lib/from-backup`:

```
$ cd /etc/kubernetes/manifests/
$ sudo vim etcd.yaml
...
spec:
  volumes:
    ...
    - hostPath:
        path: /var/lib/from-backup
        type: DirectoryOrCreate
        name: etcd-data
    ...
```

The `etcd-kube-control-plane` Pod will be re-created, and it points to the restored backup directory:

```
$ kubectl get pod etcd-kube-control-plane -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
etcd-kube-control-plane	1/1	Running	0	5m1s

In case the Pod doesn't transition into the "Running" status, try to delete it manually with the command `kubectl delete pod etcd-kube-control-plane -n kube-system`.

Exit the node using the `exit` command:

```
$ exit
logout
...
```

Summary

Backing up the etcd database should be performed as a periodic process to prevent the loss of crucial data in the event of a node or storage corruption. You can use the tool `etcdctl` to back up and the tool `etcdctl` restore etcd from the control plane node or via an API endpoint.

Exam Essentials

Practice backing up etcd

Backing up etcd requires the installation of a compatible version of the `etcdctl` executable. You can identify the version of etcd by inspecting the container image tag used to run the etcd Pod (assuming we are talking about a cluster without **high-availability characteristics**). The etcd process run in the container of the Pod also list the command line flags needed to drive the backup process. In the exam, you can assume that the `etcdctl` executable has been preinstalled.

Know how to restore etcd

Restoring etcd requires the use of the executable `etcdctl`. You will need to point the command to the snapshot file created in the backup process, and a target directory used to extract the etcd data in. Just extracting the etcd data into a directory doesn't tell the etcd process to use it. You need to configure the host path to the directory in the configuration for etcd.

Sample Exercises

Solutions to these exercises are available in **Appendix A**.

1. Navigate to the directory `app-a/ch05/etcd-backup-restore` of the checked-out GitHub repository ***bmuschko/cka-study-guide***. Start up the VMs running the cluster using the command `vagrant up`. The cluster consists of a single control plane node named `kube-control-plane`, and one worker node named `kube-worker-1`. Open an interactive shell into the control plane node and inspect the currently-used Kubernetes version by listing all the nodes.

SSH into the control plane node host machine. Identify the Pod that runs the etcd executable. Inspect the details of the Pod to find out which version of etcd it is running. Write the version to the file `etcd-version.txt`.

Once done, shut down the cluster using `vagrant destroy -f`.

Prerequisite: This exercise requires the installation of the tools **Vagrant**

and a **VMware provider**.

2. Navigate to the directory *app-a/ch05/etcd-backup-restore* of the checked-out GitHub repository *bmuschko/cka-study-guide*. Start up the VMs running the cluster using the command `vagrant up`. The cluster consists of a single control plane node named `kube-control-plane`, and one worker node named `kube-worker-1`. Open an interactive shell into the control plane node and inspect the currently-used Kubernetes version by listing all the nodes.

The `etcdctl` and `etcdutl` tool have been preinstalled on the node `kube-control-plane`. Back up etcd to the snapshot file `/opt/etcd.bak`. Restore etcd from the snapshot file. Use the data directory `/var/bak`.

Once done, shut down the cluster using `vagrant destroy -f`.

Prerequisite: This exercise requires the installation of the tools **Vagrant** and a **VMware provider**.

Chapter 6. Authentication, Authorization, and Admission Control

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles. This will be the 6th chapter of the final book. Please note that the GitHub repo will be made active later on. If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

The API server is the gateway to the Kubernetes cluster. Any human user, client (e.g., `kubectl`), cluster component, or service account will access the API server by making a RESTful API call via HTTPS. It is *the* central point for performing operations like creating a Pod or deleting a Service.

In this chapter, we’ll focus on the security-specific aspects relevant to the API server. For a detailed discussion on the inner workings of the API server and use of the Kubernetes API, refer to *Managing Kubernetes* by Brendan Burns and Craig Tracey (O’Reilly).

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Manage role based access control (RBAC)

Processing an API Request

Figure 6-1 illustrates the stages a request goes through when a call is made to the API server. For reference, you can find more information in the [Kubernetes documentation](#).

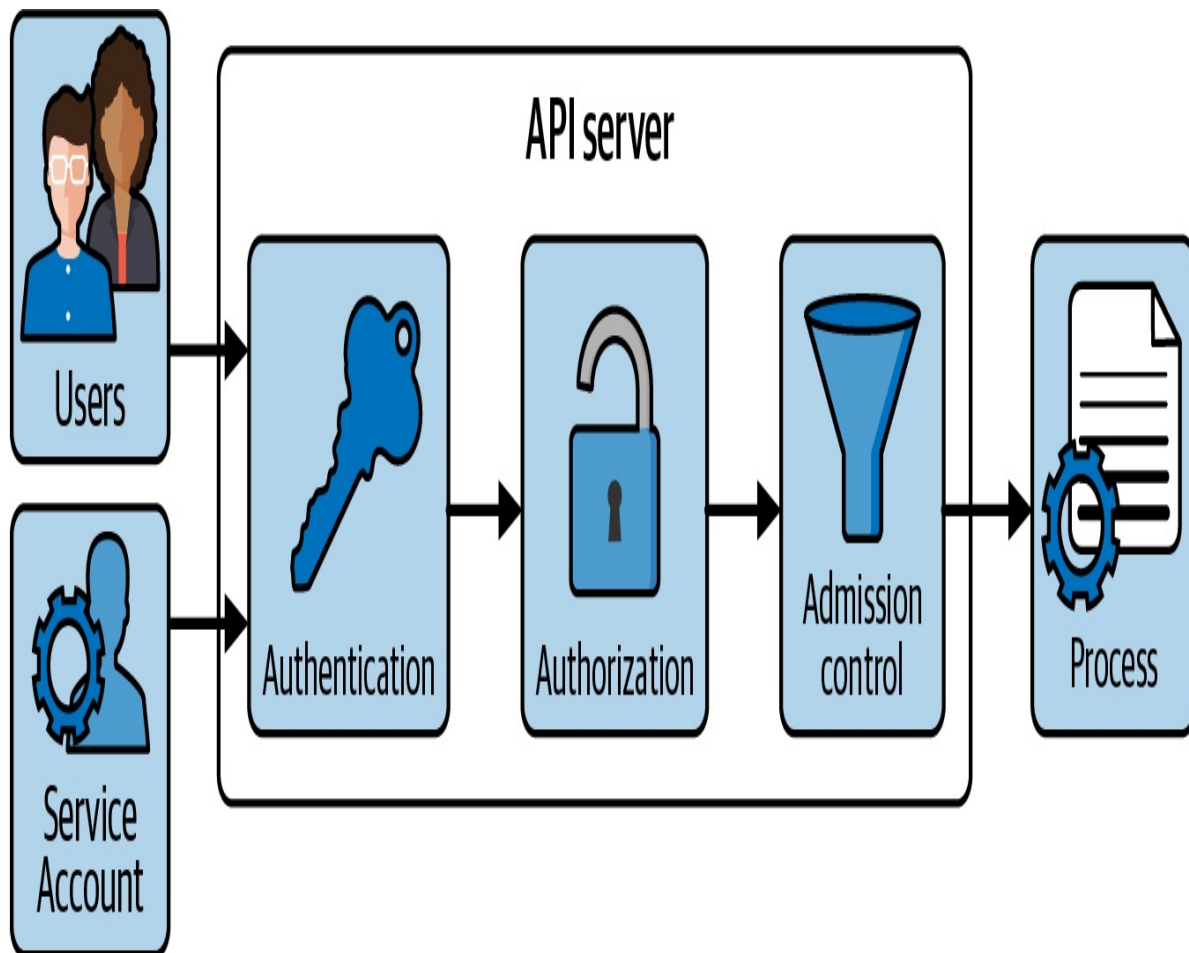


Figure 6-1. API server request processing

The first stage of request processing is *authentication*. Authentication validates the identity of the caller by inspecting the client certificates or bearer tokens. If the bearer token is associated with a service account, then it will be verified here.

The second stage determines if the identity provided in the first stage can access the verb and HTTP path request. Therefore, stage two deals with *authorization* of the request, which is implemented with the standard Kubernetes RBAC model. Here, we ensure that the service account is allowed to list Pods or create a new Service object if that has been requested.

The third stage of request processing deals with *admission control*. Admission control verifies if the request is well formed or potentially needs to be modified before the request is processed. An admission control policy could, for example, ensure that the request for creating a Pod includes the definition of a specific label. If the request doesn't define the label, then it is rejected.

Authentication with `kubectl`

Developers interact with the Kubernetes API by running the `kubectl` command line tool. Whenever you execute a command with `kubectl`, the underlying HTTPS call to the API server needs to authenticate.

The Kubeconfig

Credentials for the use of `kubectl` are stored in the file `$HOME/.kube/config`, also known as the *kubeconfig file*. The kubeconfig file defines the API server endpoints of the clusters we want to interact with, as well as a list of users registered with the cluster, including their credentials in the form of client certificates. The mapping between a cluster and user for a given namespace is called a *context*. `kubectl` uses the currently selected context to know which cluster to talk to and which credentials to use.

NOTE

You can point the environment variable `KUBECONFIG` to a set of kubeconfig files. At runtime, `kubectl` will merge the contents of the set of defined kubeconfig files and use them. By default, `KUBECONFIG` is not set and falls back to `$HOME/.kube/config`.

Example 6-1 shows a kubeconfig file. Be aware that file paths assigned in the example are user-specific and may differ in your own environment. You can find a detailed description of all configurable attributes in the [Config resource type](#) API documentation.

Example 6-1. A kubeconfig file

```
apiVersion: v1
kind: Config
```

```

clusters:                                ❶
- cluster:
  certificate-authority: /Users/bmuschko/.minikube/ca.crt
  extensions:
  - extension:
    last-update: Mon, 09 Oct 2023 07:33:01 MDT
    provider: minikube.sigs.k8s.io
    version: v1.30.1
    name: cluster_info
    server: https://127.0.0.1:63709
  name: minikube
contexts:                                ❷
- context:
  cluster: minikube
  user: bmuschko
  name: bmuschko
- context:
  cluster: minikube
  extensions:
  - extension:
    last-update: Mon, 09 Oct 2023 07:33:01 MDT
    provider: minikube.sigs.k8s.io
    version: v1.30.1
    name: context_info
    namespace: default
    user: minikube
  name: minikube
current-context: minikube                ❸
preferences: {}
users:                                    ❹
- name: bmuschko
  user:
    client-key-data: <REDACTED>
- name: minikube
  user:
    client-certificate:
/Users/bmuschko/.minikube/profiles/minikube/client.crt
    client-key: /Users/bmuschko/.minikube/profiles/minikube/client.key

```

- ❶ A list of referential names to clusters and their API server endpoints.
- ❷ A list of referential names to contexts (a combination of cluster and user).
- ❸ The currently selected context.
- ❹ A list of referential names to users and their credentials.

User management is handled by the cluster administrator. The administrator creates a user representing the developer and hands the relevant information (username and credentials) to the human wanting to interact with the cluster via

`kubectl`. Alternatively, it is also possible to integrate with external identity providers for authentication purposes, e.g., via [OpenID Connect](#).

Creating a new user manually consists of multiple steps, as described in the [Kubernetes documentation](#). The developer would then add the user to the kubeconfig file on the machine intended to interact with the cluster.

Managing Kubeconfig Using `kubectl`

You do not have to manually edit the kubeconfig file(s) to change or add configuration. `Kubectl` provides commands for reading and modifying its contents. The following commands provide an overview. You can find additional examples for commands in the [kubectl cheatsheet](#).

To view the merged contents of the kubeconfig file(s), run the following command:

```
$ kubectl config view
apiVersion: v1
kind: Config
clusters:
...
```

To render the currently selected context, use the `current-context` subcommand. The context named `minikube` is the active one:

```
$ kubectl config current-context
minikube
```

To change the context, provide the name with the `use-context` subcommand. Here, we are switching to the context `bmuschko`:

```
$ kubectl config use-context bmuschko
Switched to context "bmuschko".
```

To register a user with the kubeconfig file(s), use the `set-credentials` subcommand. We are choosing to assign the username `myuser` and point to the client certificate by providing the corresponding CLI flags:

```
$ kubectl config set-credentials myuser \
```

```
--client-key=myuser.key --client-certificate=myuser.crt \  
--embed-certs=true
```

For the exam, familiarize yourself with the `kubectl config` command. Every task in the exam will require you to work with a specific context and/or namespace.

Authorization with Role-Based Access Control

We've learned that the API server will try to authenticate any request sent using `kubectl` by verifying the provided credentials. An authenticated request will then need be checked against the permissions assigned to the requestor. The authorization phase of the API processing workflow checks if the operation is permitted against the requested API resource.

In Kubernetes, those permissions can be controlled using Role-Based Access Control (RBAC). In a nutshell, RBAC defines policies for users, groups, and service accounts by allowing or disallowing access to manage API resources. Enabling and configuring RBAC is mandatory for any organization with a strong emphasis on security.

Setting permissions is the responsibility of a cluster administrator. In the following sections, we'll briefly talk about the effects of RBAC on requests from users and service accounts.

RBAC Overview

RBAC helps with implementing a variety of use cases:

- Establishing a system for users with different roles to access a set of Kubernetes resources
- Controlling processes (associated with a service account) running in a Pod and performing operations against the Kubernetes API
- Limiting the visibility of certain resources per namespace

RBAC consists of three key building blocks, as shown in [Figure 6-2](#). Together, they connect API primitives and their allowed operations to the so-called

subject, which is a user, a group, or a service account.

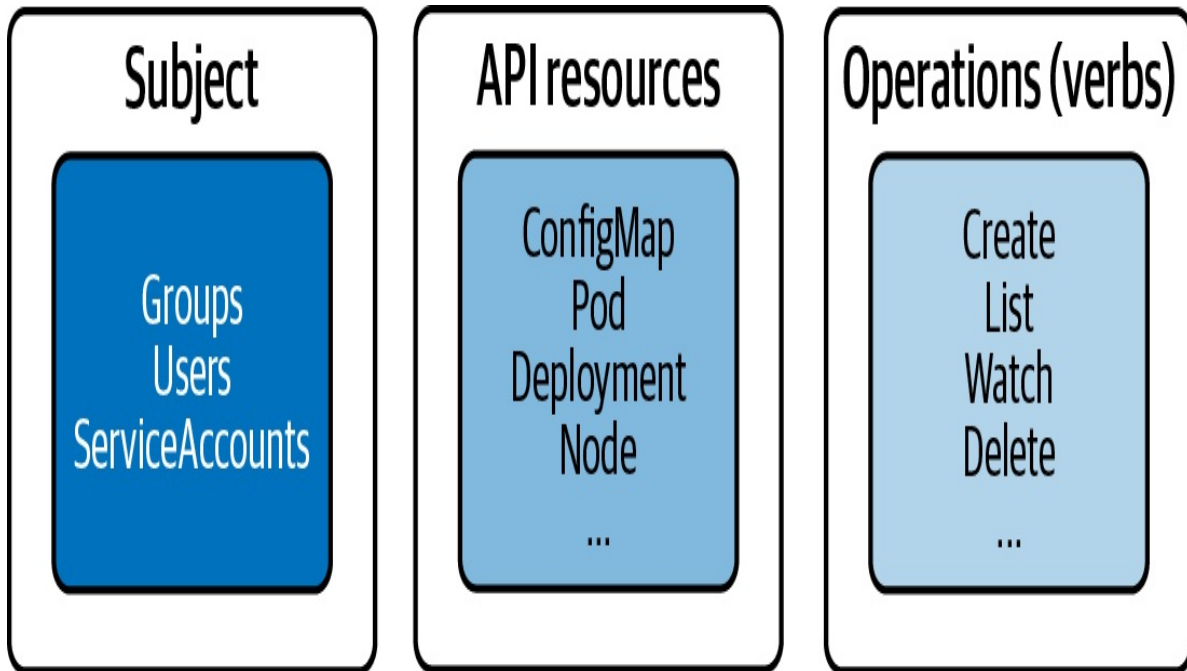


Figure 6-2. RBAC key building blocks

Each block's responsibilities are as follows:

Subject

The user or service account that wants to access a resource

Resource

The Kubernetes API resource type (e.g., a Deployment or node)

Verb

The operation that can be executed on the resource (e.g., creating a Pod or deleting a Service)

Understanding RBAC API Primitives

With these key concepts in mind, let's look at the Kubernetes API primitives that implement the RBAC functionality:

Role

The Role API primitive declares the API resources and their operations this

rule should operate on in a specific namespace. For example, you may want to say “allow listing and deleting of Pods,” or you may express “allow watching the logs of Pods,” or even both with the same Role. Any operation that is not spelled out explicitly is disallowed as soon as it is bound to the subject.

RoleBinding

The RoleBinding API primitive *binds* the Role object to the subject(s) in a specific namespace. It is the glue for making the rules active. For example, you may want to say “bind the Role that permits updating Services to the user John Doe.”

Figure 6-3 shows the relationship between the involved API primitives. Keep in mind that the image renders only a selected list of API resource types and operations.

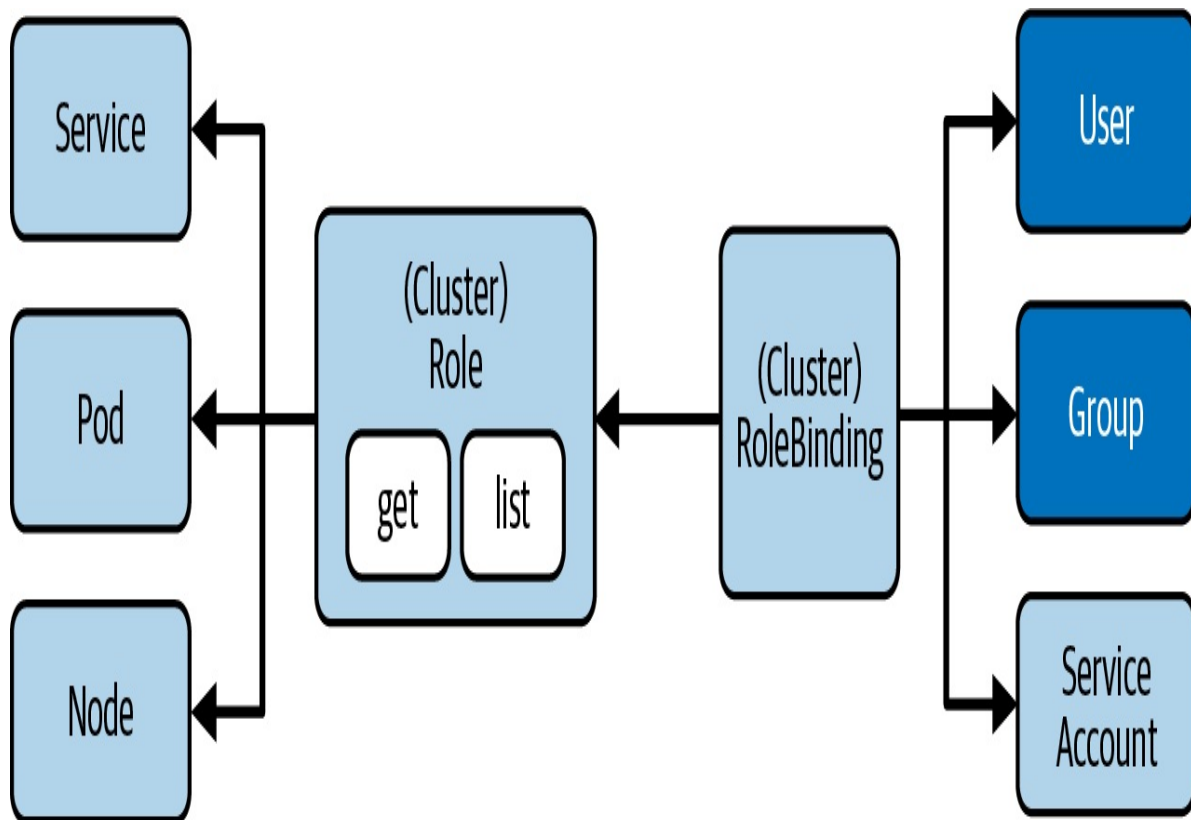


Figure 6-3. RBAC primitives

The following sections demonstrate the namespace-wide usage of Roles and

RoleBindings, but the same operations and attributes apply to cluster-wide Roles and RoleBindings, discussed in “**Namespace-Wide and Cluster-Wide RBAC**”.

Default User-Facing Roles

Kubernetes defines a set of default Roles. You can assign them to a subject via a RoleBinding or define your own, custom Roles depending on your needs.

Table 6-1 describes the default user-facing Roles.

Table 6-1. Default user-facing Roles

Default ClusterRole	Description
cluster-admin	Allows read and write access to resources across all namespaces.
admin	Allows read and write access to resources in namespace including Roles and RoleBindings.
edit	Allows read and write access to resources in namespace except Roles and RoleBindings. Provides access to Secrets.
view	Allows read-only access to resources in namespace except Roles, RoleBindings, and Secrets.

To define new Roles and RoleBindings, you will have to use a context that allows for creating or modifying them, that is, cluster-admin or admin.

Creating Roles

Roles can be created imperatively with the `create role` command. The most important options for the command are `--verb` for defining the verbs, aka operations, and `--resource` for declaring a list of API resources (core primitives as well as CRDs). The following command creates a new Role for the

resources Pod, Deployment, and Service with the verbs `list`, `get`, and `watch`:

```
$ kubectl create role read-only --verb=list,get,watch \
  --resource=pods,deployments,services
role.rbac.authorization.k8s.io/read-only created
```

Declaring multiple verbs and resources for a single imperative `create role` command can be declared as a comma-separated list for the corresponding command-line option or as multiple arguments. For example, `--verb=list,get,watch` and `--verb=list --verb=get --verb=watch` carry the same instructions. You also can use the wildcard “*” to refer to all verbs or resources.

The command-line option `--resource-name` spells out one or many object names that the policy rules should apply to. A name of a Pod could be `nginx` and listed here with its name. Providing a list of resource names is optional. If no names have been provided, then the provided rules apply to all objects of a resource type.

The declarative approach can become a little lengthy. As you can see in [Example 6-2](#), the section `rules` lists the resources and verbs. Resources with an API group, like Deployments that use the API version `apps/v1`, need to explicitly declare it under the attribute `apiGroups`. All other resources (e.g., Pods and Services), simply use an empty string, as their API version doesn’t contain a group. Be aware that the imperative command for creating a Role automatically determines the API group.

Example 6-2. A YAML manifest defining a Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: read-only
rules:
- apiGroups:
  - ""
  resources:
  - pods
  - services
  verbs:
  - list
```

- get
- watch
- **apiGroups:** ❶
 - apps
- resources:**
 - deployments
- verbs:**
 - list
 - get
 - watch

- ❶ Any resource that belongs to an API group need to be listed as an explicit rule in addition to the API resources that do not belong to an API group.

Listing Roles

Once the Role has been created, its object can be listed. The list of Roles renders only the name and the creation timestamp. Each of the listed roles does not give away any of its details:

```
$ kubectl get roles
NAME          CREATED AT
read-only     2021-06-23T19:46:48Z
```

Rendering Role Details

You can inspect the details of a Role using the `describe` command. The output renders a table that maps a resource to its permitted verbs:

```
$ kubectl describe role read-only
Name:          read-only
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources          Non-Resource URLs  Resource Names  Verbs
  -----
  pods               []                 []              [list get
watch]
  services           []                 []              [list get
watch]
  deployments.apps   []                 []              [list get
watch]
```

This cluster has no resources created, so the list of resource names in the

following console output is currently empty.

Creating RoleBindings

The imperative command creating a RoleBinding object is `create rolebinding`. To bind a Role to the RoleBinding, use the `--role` command-line option. The subject type can be assigned by declaring the options `--user`, `--group`, or `--serviceaccount`. The following command creates the RoleBinding with the name `read-only-binding` to the user called `bmuschko`:

```
$ kubectl create rolebinding read-only-binding --role=read-only --
user=bmuschko
rolebinding.rbac.authorization.k8s.io/read-only-binding created
```

Example 6-3 shows a YAML manifest representing the RoleBinding. You can see from the structure that a role can be mapped to one or many subjects. The data type is an array indicated by the dash character under the attribute `subjects`. At this time, only the user `bmuschko` has been assigned.

Example 6-3. A YAML manifest defining a RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-only-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: read-only
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: bmuschko
```

Listing RoleBindings

The most important information the list of RoleBindings displays is the associated Role. The following command shows that the RoleBinding `read-only-binding` has been mapped to the Role `read-only`:

```
$ kubectl get rolebindings
```

NAME	ROLE	AGE
read-only-binding	Role/read-only	24h

The output does not provide an indication of the subjects. You will need to render the details of the object for more information, as described in the next section.

Rendering RoleBinding Details

RoleBindings can be inspected using the `describe` command. The output renders a table of subjects and the assigned role. The following example renders the descriptive representation of the RoleBinding named `read-only-binding`:

```
$ kubectl describe rolebinding read-only-binding
Name:          read-only-binding
Labels:        <none>
Annotations:   <none>
Role:
  Kind:  Role
  Name:  read-only
Subjects:
  Kind  Name      Namespace
  ----  -
  User  bmuschko
```

Seeing the RBAC Rules in Effect

Let's see how Kubernetes enforces the RBAC rules for the scenario we set up so far. First, we'll create a new Deployment with the `cluster-admin` permissions. In Minikube, those permissions are available to the context `minikube` by default:

```
$ kubectl config current-context
minikube
$ kubectl create deployment myapp --image=:1.25.2 --port=80 --
replicas=2
deployment.apps/myapp created
```

Now, we'll switch the context for the user `bmuschko`:

```
$ kubectl config use-context bmuschko-context
Switched to context "bmuschko-context".
```

Remember that the user `bmuschko` is permitted to list Deployments. We'll verify that by using the `get deployments` command:

```
$ kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
myapp     2/2     2            2           8s
```

The RBAC rules allow listing Deployments, Pods, and Services only. The following command tries to list the ReplicaSets, which results in an error:

```
$ kubectl get replicaset
Error from server (Forbidden): replicaset.apps is forbidden: User
"bmuschko" \
cannot list resource "replicaset" in API group "apps" in the
namespace "default"
```

A similar behavior can be observed when trying to use verbs other than `list`, `get`, or `watch`. The following command tries to delete a Deployment:

```
$ kubectl delete deployment myapp
Error from server (Forbidden): deployment.apps "myapp" is forbidden:
User \
"bmuschko" cannot delete resource "deployment" in API group "apps" in
the \
namespace "default"
```

At any given time, you can check a user's permissions with the `auth can-i` command. The command gives you the option to list all permissions or check a specific permission:

```
$ kubectl auth can-i --list --as bmuschko
Resources          Non-Resource URLs   Resource Names       Verbs
...
pods               []                  []                   [list get
watch]
services          []                  []                   [list get
watch]
deployments.apps   []                  []                   [list get
watch]
$ kubectl auth can-i list pods --as bmuschko
```


yes

Namespace-Wide and Cluster-Wide RBAC

Roles and RoleBindings apply to a particular namespace. You will have to specify the namespace when creating both objects. Sometimes, a set of Roles and RoleBindings needs to apply to multiple namespaces or even to the whole cluster. For a cluster-wide definition, Kubernetes offers the API resource types ClusterRole and ClusterRoleBinding. The configuration elements are effectively the same. The only difference is the value of the `kind` attribute:

- To define a cluster-wide Role, use the imperative subcommand `clusterrole` or the kind `ClusterRole` in the YAML manifest.
- To define a cluster-wide RoleBinding, use the imperative subcommand `clusterrolebinding` or the kind `ClusterRoleBinding` in the YAML manifest.

ClusterRoles and ClusterRoleBindings not only set up cluster-wide permissions to a namespaced resource, but they can also be used to set up permissions for non-namespaced resources like CRDs and nodes.

Aggregating RBAC Rules

Existing ClusterRoles can be aggregated to avoid having to redefine a new, composed set of rules that likely leads to duplication of instructions. For example, say you wanted to combine a user-facing role with a custom Role. An aggregated ClusterRule can merge rules via label selection without having to copy-paste the existing rules into one.

Say we defined two ClusterRoles shown in [Example 6-4](#) and [Example 6-5](#). The ClusterRole `list-pods` allows for listing Pods and the ClusterRole `delete-services` allows for deleting Services.

Example 6-4. A YAML manifest defining a ClusterRole for listing Pods

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: list-pods
  namespace: rbac-example
```

```

    labels:
      rbac-pod-list: "true"
rules:
- apiGroups:
  - ""
  resources:
  - pods
  verbs:
  - list

```

Example 6-5. A YAML manifest defining a ClusterRole for deleting Services

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: delete-services
  namespace: rbac-example
  labels:
    rbac-service-delete: "true"
rules:
- apiGroups:
  - ""
  resources:
  - services
  verbs:
  - delete

```

To aggregate those rules, ClusterRoles can specify an `aggregationRule`. This attribute describes the label selection rules. [Example 6-6](#) shows an aggregated ClusterRole defined by an array of `matchLabels` criteria. The ClusterRole does not add its own rules as indicated by `rules: []`; however, there's no limiting factor that would disallow it.

Example 6-6. A YAML manifest defining a ClusterRole with aggregated rules

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: pods-services-aggregation-rules
  namespace: rbac-example
aggregationRule:
  clusterRoleSelectors:
  - matchLabels:
      rbac-pod-list: "true"
  - matchLabels:
      rbac-service-delete: "true"
rules: []

```

We can verify the proper aggregation behavior of the ClusterRole by describing

the object. You can see in the following output that both ClusterRoles, `list-pods` and `delete-services`, have been taken into account:

```
$ kubectl describe clusterroles pods-services-aggregation-rules -n
rbac-example
Name:          pods-services-aggregation-rules
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources      Non-Resource URLs  Resource Names  Verbs
  -----
  services      []                  []              [delete]
  pods          []                  []              [list]
```

For more information on ClusterRole label selection rules, see the [official documentation](#). The page also explains how to aggregate the default user-facing ClusterRoles.

Working with Service Accounts

We've been using the `kubectl` executable to run operations against a Kubernetes cluster. Under the hood, its implementation calls the API server by making an HTTP call to the exposed endpoints. Some applications running inside of a Pod may have to communicate with the API server as well. For example, the application may ask for specific cluster node information or available namespaces.

Pods can use a service account to authenticate with the API server through an authentication token. A Kubernetes administrator assigns rules to a service account via RBAC to authorize access to specific resources and actions as illustrated in [Figure 6-4](#).

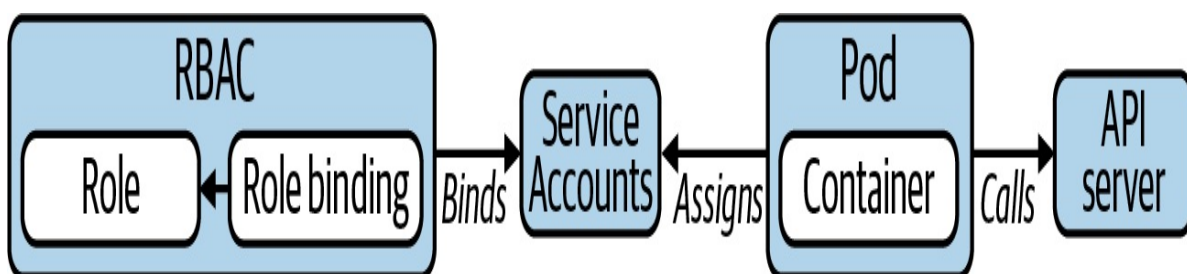


Figure 6-4. Using a service account to communicate with an API server

A Pod doesn't necessarily need to be involved in the process. Other use cases call for leveraging a service account outside of a Kubernetes cluster. For example, you may want to communicate with the API server as part of CI/CD pipeline automation step. The service account can provide the credentials to authenticate with the API server.

The Default Service Account

So far, we haven't defined a service account for a Pod. If not assigned explicitly, a Pod uses the **default service account**, which has the same permissions as an unauthenticated user. This means that the Pod cannot view or modify the cluster state or list or modify any of its resources. The **default** service account can however request basic cluster information via the assigned **system:discovery** Role.

You can query for the available service accounts with the subcommand **serviceaccounts**. You should see only the **default** service account listed in the output:

```
$ kubectl get serviceaccounts
NAME      SECRETS  AGE
default   0        4d
```

While you can execute the **kubectl** operation to delete the **default** service account, Kubernetes will reinstantiate the service account immediately.

Creating a Service Account

You can create a custom service account object using the imperative and declarative approach. This command creates a service account object with the name **cicd-bot**. The assumption here is to use the service account for calls to the API server made by a CI/CD pipeline:

```
$ kubectl create serviceaccount cicd-bot
serviceaccount/cicd-bot created
```

You can also represent the service account in the form of a manifest. In its simplest form, the definition assigns the kind **ServiceAccount** and a name,

as shown in [Example 6-7](#).

Example 6-7. YAML manifest for a service account

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cicd-bot
```

You can set a couple of [configuration options](#) for a service account. For example, you may want to disable automounting of the authentication token when assigning the service account to a Pod. Although you will not need to understand those configuration options for the exam, it makes sense to dive deeper into security best practices by reading up on them in the Kubernetes documentation.

Setting Permissions for a Service Account

It's important to limit the permissions to only the service accounts that are necessary for the application to function. The next sections will explain how to achieve this to minimize the potential attack surface.

For this scenario to work, you'll need to create a ServiceAccount object and assign it to the Pod. Service accounts can be tied in with RBAC and assigned a Role and RoleBinding to define which operations they should be allowed to perform.

Binding the service account to a Pod

As a starting point, we will set up a Pod that lists all Pods and Deployments in the namespace `k97` by calling the Kubernetes API. The call is made as part of an infinite loop every ten seconds. The response from the API call will be written to standard output accessible via the Pod's logs.

ACCESSING THE API SERVER ENDPOINT

Accessing the Kubernetes API from a Pod is straightforward. Instead of using the IP address and port for the API server Pod, you can simply refer to a Service named `kubernetes.default.svc` instead. This special Service lives in the `default` namespace and is stood up by the cluster automatically.

To authenticate against the API server, we'll send a bearer token associated with the service account used by the Pod. The default behavior of a service account is to auto-mount API credentials on the path

`/var/run/secrets/kubernetes.io/serviceaccount/token`. We'll simply get the contents of the file using the `cat` command-line tool and send them along as a header for the HTTP request. **Example 6-8** defines the namespace, the service account, and the Pod in a single YAML manifest file: *setup.yaml*.

Example 6-8. YAML manifest for assigning a service account to a Pod

```
apiVersion: v1
kind: Namespace
metadata:
  name: k97
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: sa-api
  namespace: k97
---
apiVersion: v1
kind: Pod
metadata:
  name: list-objects
  namespace: k97
spec:
  serviceAccountName: sa-api ❶
  containers:
  - name: pods
    image: alpine/curl:3.14
    command: ['sh', '-c', 'while true; do curl -s -k -m 5 -H \
      "Authorization: Bearer $(cat
/var/run/secrets/kubernetes.io/ \
      serviceaccount/token)"
https://kubernetes.default.svc.cluster. \
      local/api/v1/namespaces/k97/pods; sleep 10; done'] ❷
  - name: deployments
    image: alpine/curl:3.14
    command: ['sh', '-c', 'while true; do curl -s -k -m 5 -H \
      "Authorization: Bearer $(cat
/var/run/secrets/kubernetes.io/ \
      serviceaccount/token)"
https://kubernetes.default.svc.cluster. \
      local/apis/apps/v1/namespaces/k97/deployments;
      sleep 10; done'] ❸
```

- ❶ The service account referenced by name used for communicating with the Kubernetes API.
- ❷ Performs an API call to retrieve the list of Pods in the namespace k97.
- ❸ Performs an API call to retrieve the list of Deployments in the namespace k97.

Create the objects from the YAML manifest with the following command:

```
$ kubectl apply -f setup.yaml
namespace/k97 created
serviceaccount/sa-api created
pod/list-objects created
```

Verifying the default permissions

The Pod named `list-objects` makes a call to the API server to retrieve the list of Pods and Deployments in dedicated containers. The container `Pods` performs the call to list Pods. The container `deployments` sends a request to the API server to list Deployments.

As explained in the [Kubernetes documentation](#), the default RBAC policies do not grant any permissions to service accounts outside of the `kube-system` namespace. The logs of the containers `Pods` and `deployments` return an error message indicating that the service account `sa-api` is not authorized to list the resources:

```
$ kubectl logs list-objects -c pods -n k97
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "pods is forbidden: User \"system:serviceaccount:k97:sa-api\" \
      cannot list resource \"pods\" in API group \"\" in the \
      namespace \"k97\"",
  "reason": "Forbidden",
  "details": {
    "kind": "pods"
  },
  "code": 403
}
```

```
$ kubectl logs list-objects -c deployments -n k97
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "deployments.apps is forbidden: User \
    \"system:serviceaccount:k97:sa-api\" cannot list
resource \
    \"deployments\" in API group \"apps\" in the namespace \
    \"k97\"",
  "reason": "Forbidden",
  "details": {
    "group": "apps",
    "kind": "deployments"
  },
  "code": 403
}
```

Next up, we'll stand up a Role and RoleBinding object with the required API permissions to perform the necessary calls.

Creating the Role

Start by defining the Role named `list-pods-role` shown in [Example 6-9](#) in the file `role.yaml`. The set of the rules adds only the Pod resource and the verb `list`.

Example 6-9. YAML manifest for a Role that allows listing Pods

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: list-pods-role
  namespace: k97
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["list"]
```

Create the object by pointing to its corresponding YAML manifest file:

```
$ kubectl apply -f role.yaml
role.rbac.authorization.k8s.io/list-pods-role created
```

Creating the RoleBinding

Example 6-10 defines the YAML manifest for the RoleBinding in the file `rolebinding.yaml`. The RoleBinding maps the Role `list-pods-role` to the service account named `sa-pod-api` and applies it only to the namespace `k97`.

Example 6-10. YAML manifest for a RoleBinding attached to a service account

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: serviceaccount-pod-rolebinding
  namespace: k97
subjects:
- kind: ServiceAccount
  name: sa-api
roleRef:
  kind: Role
  name: list-pods-role
  apiGroup: rbac.authorization.k8s.io
```

Create both RoleBinding objects using the `apply` command:

```
$ kubectl apply -f rolebinding.yaml
rolebinding.rbac.authorization.k8s.io/serviceaccount-pod-rolebinding
created
```

Verifying the granted permissions

With the granted `list` permissions, the service account can now properly retrieve all the Pods in the `k97` namespace. The `curl` command in the `Pods` container succeeds, as shown in the following output:

```
$ kubectl logs list-objects -c pods -n k97
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "resourceVersion": "628"
  },
  "items": [
    {
      "metadata": {
        "name": "list-objects",
        "namespace": "k97",
        ...
      }
    }
  ]
}
```

```
}  
}
```

We did not grant any permissions to the service account for other resources. Listing the Deployments in the k97 namespace still fails. The following output shows the response from the `curl` command in the `deployments` namespace:

```
$ kubectl logs list-objects -c deployments -n k97  
{  
  "kind": "Status",  
  "apiVersion": "v1",  
  "metadata": {},  
  "status": "Failure",  
  "message": "deployments.apps is forbidden: User \\  
            \\\"system:serviceaccount:k97:sa-api\\\" cannot list  
resource \\  
            \\\"deployments\\\" in API group \\\"apps\\\" in the namespace \\  
            \\\"k97\\\"",  
  "reason": "Forbidden",  
  "details": {  
    "group": "apps",  
    "kind": "deployments"  
  },  
  "code": 403  
}
```

Feel free to modify the Role object to allow listing Deployment objects as well.

Admission Control

The last phase of processing a request to the API server is admission control. Admission control is implemented by admission controllers. An admission controller provides a way to approve, deny, or mutate a request before it takes effect.

Admission controllers can be registered with the configuration of the API server. By default, the configuration file can be found at `/etc/kubernetes/manifests/kube-apiserver.yaml`. It is the cluster administrator's job to manage the API server configuration. The following command-line invocation of the API server enables the **admission control plugins** named `NamespaceLifecycle`, `PodSecurity` and `LimitRanger`:

```
$ kube-apiserver --enable-admission-  
plugins=NamespaceLifecycle,PodSecurity,\  
LimitRanger
```

Developers will inadvertently use admission control plugins that have been configured by the administrator. One example is the **LimitRanger** and the **ResourceQuota**, we'll discuss in [Link to Come] and [Link to Come].

Summary

The API server processes requests to the Kubernetes API. Every request has to go through three phases: authentication, authorization, and admission control. Every phase can short-circuit the processing. For example, if the credentials sent with the request cannot be authenticated, then the request will be dropped immediately.

We looked at examples of all phases. The authentication phase covered `kubectl` as the client making a call to the Kubernetes API. The `kubeconfig` file serves as configuration source for named cluster, users, and their credentials. In Kubernetes, authorization is handled by RBAC. We learned the Kubernetes primitives that let you configure permissions for API resources tied to one or many subjects.

Finally, we briefly covered the purpose of admission control and listed some plugins that act as controllers for validating or mutating a request to the Kubernetes API.

Exam Essentials

Practice interacting with the Kubernetes API

This chapter demonstrated some ways to communicate with the Kubernetes API. We performed API requests by switching to a user context and with the help of a RESTful API call using `curl`. Explore the **Kubernetes API** and its endpoints on your own for broader exposure.

Understand the implications of defining RBAC rules for users and service

accounts

Anonymous user requests to the Kubernetes API will not allow any substantial operations. For requests coming from a user or a service account, you will need to carefully analyze permissions granted to the subject. Learn the ins and outs of defining RBAC rules by creating the relevant objects to control permissions. Service accounts automount a token when used in a Pod. Expose the token as a volume only if you are intending to make API calls from the Pod.

Be aware of the purpose of admission control

The API server comes with preconfigured admission control plugins that support the functionality of Kubernetes primitives like the LimitRange. For the exam, you will not have to have a deep understanding of enabling or configuring admission control plugins.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Navigate to the directory *app-a/ch06/rbac-aggregation* of the checked-out GitHub repository *[bmuschko/cka-study-guide](#)*.

Create a ClusterRole with the name `service-view` to the API resources `services` with the operations `get` and `list`. Create the RoleBinding named `ellasmith-service-view` in the `development` namespace. Map the user `ellasmith` to the ClusterRole `service-view`.

Create a ClusterRole with the name `combined`. Aggregate cluster roles based on the matching label key-value pair `rbac.cka.cncf.com/aggregate: "true"`. Render the selected rules of the ClusterRole `combined`. How many rules do you see?

Create a ClusterRole with the name `deployment-modify` to the API resources `deployments` with the operations `create`, `delete`,

patch, and update. Assign the label key-value pair `rbac.cka.cncf.com/aggregate: "true"`. Render the selected rules of the ClusterRole combined. How many rules do you see?

Run a command to figure out if the user `ellasmith` can list Services in the namespace `development`. Write the output of the command to the file `list-services-ellasmith.txt`. The output is either “no” or “yes.”

Run a command to figure out if the user `ellasmith` can watch Deployments in the namespace `production`. Write the output of the command to the file `watch-deployments-ellasmith.txt`. The output is either “no” or “yes.”

Prerequisite: This exercise requires the installation of the tools **Vagrant** and a **VMware provider**.

2. Create the ServiceAccount named `api-access` in a new namespace called `apps`.

Create a ClusterRole with the name `api-clusterrole`, and create a ClusterRoleBinding named `api-clusterrolebinding`. Map the ServiceAccount from the previous step to the API resources `pods` with the operations `watch`, `list`, and `get`.

Create a Pod named `operator` with the image `nginx:1.21.1` in the namespace `apps`. Expose the container port 80. Assign the ServiceAccount `api-access` to the Pod. Create another Pod named `disposable` with the image `nginx:1.21.1` in the namespace `rm`. Do not assign the ServiceAccount to the Pod.

Open an interactive shell to the Pod named `operator`. Use the command-line tool `curl` to make an API call to list the Pods in the namespace `rm`. What response do you expect? Use the command-line tool `curl` to make an API call to delete the Pod `disposable` in the namespace `rm`. Does the response differ from the first call? You can find information about how to interact with Pods using the API via HTTP in the **reference guide**.

Chapter 7. Operators and Custom Resource Definitions (CRDs)

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles. This will be the 7th chapter of the final book. Please note that the GitHub repo will be made active later on. If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Kubernetes comes with core feature set to fulfill the basic needs for running application stacks with a standard set of primitives. For custom use cases, Kubernetes allows for installing extensions to the platform, so-called operators.

A Custom Resource Definition (CRD) is a Kubernetes extension mechanism (often bundled with an operator) for introducing custom API primitives to fulfill requirements not covered by built-in primitives.

This chapter will focus the installation and configuration of operators, as well the interaction with provided CRDs.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Understand CRDs, install and configure operators

Working with Operators

Operators extend the core behavior of the Kubernetes cluster without actually change the Kubernetes code. You can think of the operator as a plugin to the platform. Operators typically automate tasks that would have to be performed by humans, such as deploying, configuring, scaling, upgrading, and managing applications.

The Operator Pattern

An operator usually consists of two components, one or more Custom Resource Definitions (CRD) and a controller. CRDs can be understood as the schema that defines the blueprint for a custom object, and then the instantiation of those objects with the newly introduced type, also called the Custom Resources (CR).

For a CRD to be useful, it has to be backed by a controller. Controllers interact with the Kubernetes API and implement the reconciliation logic that interacts with CRD objects.

The combination of CRDs and controllers is commonly referred to as the *operator pattern*. The exam does not require you to have an understanding of controllers; therefore, their implementation won't be covered in this chapter.

Figure 7-1 shows the operator patterns with all its moving parts.

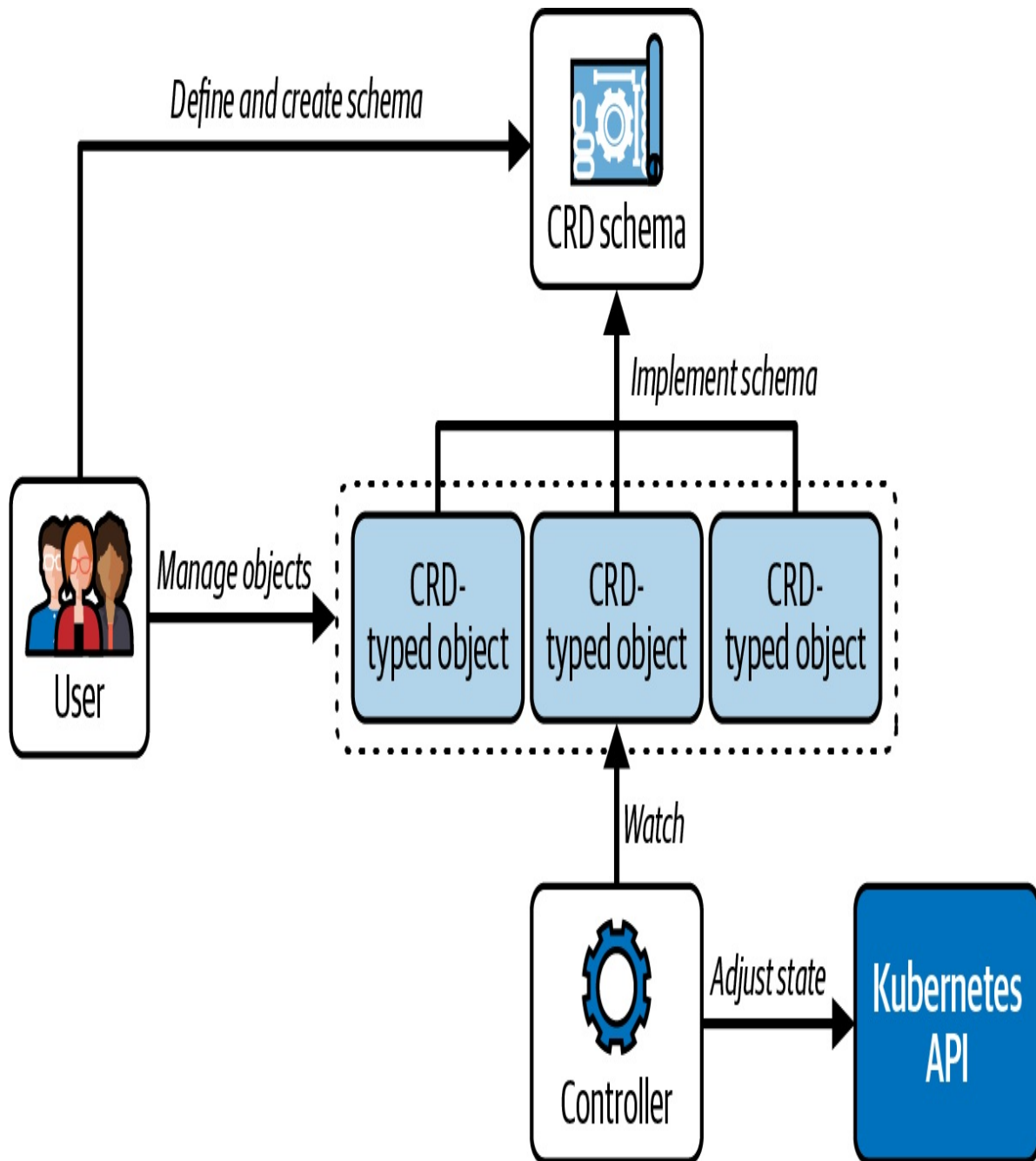


Figure 7-1. The Kubernetes operator pattern

Discovering Operators

The Kubernetes community has implemented many useful operators discoverable on [Operator Hub](#) or [Artifact Hub](#).

A prominent operator is the [External Secrets Operator](#) that helps with integrating external Secret managers, like AWS Secrets Manager and HashiCorp Vault, with

Kubernetes. Another is the **Crossplane Operator** which helps with creating and managing cloud resources using declarative syntax.

To demonstrate the functionality of Operator Hub, we are search and install the popular **Argo CD Operator**, a declarative, GitOps continuous delivery tool for Kubernetes that automates deployments by continuously monitoring applications and synchronizing them with the desired state defined in Git repositories.

In the browser, open the URL for **Operator Hub** and enter the term “argo cd” into the search box named “Search OperatorHub...”. You will receive a result, shown in **Figure 7-2**.

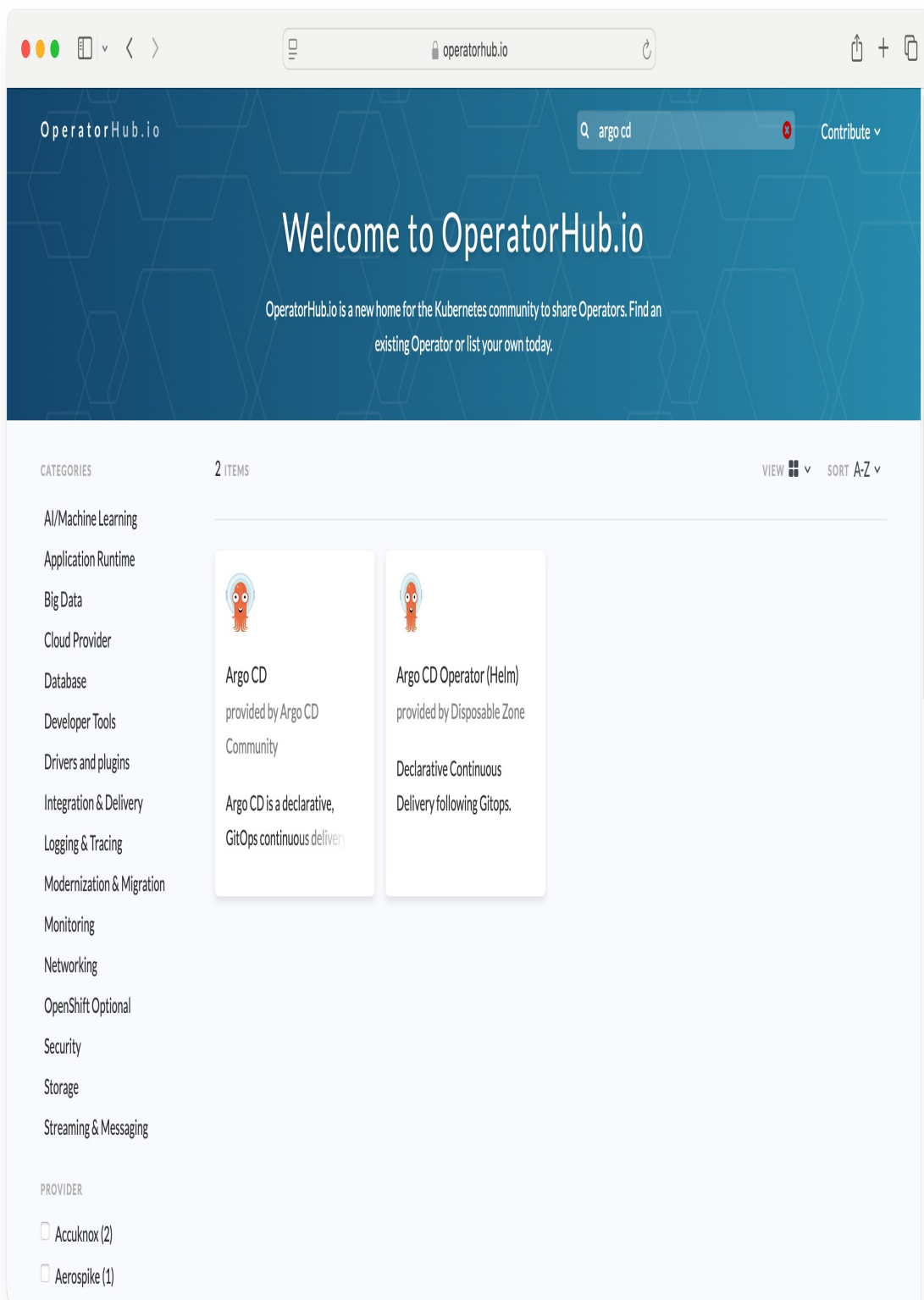


Figure 7-2. A search result on Operator Hub

Clicking on the panel named “Argo CD” will bring you to the details of the **Argo CD Operator**.

OperatorHub.io

Search OperatorHub...

Contribute



Argo CD

Argo CD is a declarative, GitOps continuous delivery tool for Kubernetes.

[Home](#) > [Argo CD](#)

Argo CD

Install

Overview

The Argo CD Operator manages the full lifecycle for [Argo CD](#) and it's components. The operator's goal is to automate the tasks required when operating an Argo CD cluster.

Beyond installation, the operator helps to automate the process of upgrading, backing up and restoring as needed and remove the human as much as possible. In addition, the operator aims to provide deep insights into the Argo CD environment by configuring Prometheus and Grafana to aggregate, visualize and expose the metrics already exported by Argo CD.

The operator aims to provide the following, and is a work in progress.

- Easy configuration and installation of the Argo CD components with sane defaults to get up and running quickly.
- Provide seamless upgrades to the Argo CD components.
- Ability to back up and restore an Argo CD cluster from a point in time or on a recurring schedule.
- Aggregate and expose the metrics for Argo CD and the operator itself using Prometheus and Grafana.
- Autoscale the Argo CD components as necessary to handle variability in demand.

CHANNEL

alpha

VERSION

0.13.0 (Current)

CAPABILITY LEVEL

Basic Install

Seamless Upgrades

Full Lifecycle

Deep Insights

Auto Pilot

PROVIDER

Argo CD Community

LINKS

Figure 7-3. The Argo CD Operator on Operator Hub

The page describes the functionality of the operator including a high-level overview on the provided CRDs. To be able to create CRs from the CRDs, we'll first need to install the operator.

Installing Operators

You can install many of those operators with a single `kubectl` command execution or by using the Helm executable. For more information on using Helm, see [Chapter 8](#).

Clicking on the “Install” button on Operator Hub brings up the installation instructions, as shown in [Figure 7-4](#).

OperatorHub.io

Search OperatorHub

Contribute

Home > Argo CD

Argo CD

Overview

The Argo CD Operator is a Kubernetes operator that manages the lifecycle of Argo CD clusters. It provides a simple and easy way to install and manage Argo CD clusters on Kubernetes.

Beyond installation, the operator provides a number of features to help you manage your Argo CD clusters as much as possible. In addition, it provides a number of features to help you manage your Argo CD clusters as much as possible. In addition, it provides a number of features to help you manage your Argo CD clusters as much as possible.

The operator aims to provide a number of features to help you manage your Argo CD clusters as much as possible. In addition, it provides a number of features to help you manage your Argo CD clusters as much as possible.

- Easy configuration and installation of the Argo CD components with sane defaults to get up and running quickly.
- Provide seamless upgrades to the Argo CD components.
- Ability to back up and restore an Argo CD cluster from a point in time or on a recurring schedule.
- Aggregate and expose the metrics for Argo CD and the operator itself using Prometheus and Grafana.
- Autoscale the Argo CD components as necessary to handle variability in demand.

Argo CD

0.13.0 provided by Argo CD Community

Install on Kubernetes

1. Install Operator Lifecycle Manager (OLM), a tool to help manage the Operators running on your cluster.

```
$ curl -sL https://github.com/operator-framework/operator-lifecycle-manager/releases/download/v0.31.0/install.sh | bash -s v0.31.0
```
2. Install the operator by running the following command:

[What happens when I execute this command?](#)

```
$ kubectl create -f https://operatorhub.io/install/argocd-operator.yaml
```

This Operator will be installed in the "operators" namespace and will be usable from all namespaces in the cluster.
3. After install, watch your operator come up using next command.

```
$ kubectl get csv -n operators
```

To use it, checkout the custom resource definitions (CRDs) introduced by this operator to start using it.

Deep Insights

Auto Pilot

PROVIDER

Argo CD Community

LINKS

Figure 7-4. The Argo CD Operator installation instructions

As shown on the installation page for the operator, we'll use the **Operator Lifecycle Manager (OLM)**, a tool to help manage the operators running on your cluster. This is a one-time operation.

```
$ curl -sL https://github.com/operator-framework/operator-lifecycle-manager/releases/download/v0.31.0/install.sh | bash -s v0.31.0
```

Next, we'll install the Argo CD Operator which will place the operator objects into the `operators` namespace.

```
$ kubectl create -f https://operatorhub.io/install/argocd-operator.yaml
subscription.operators.coreos.com/my-argocd-operator created
```

You can follow the installation process by running the following command. A valid installation finishes with the “Succeeded” phase in the rendered output of the command.

```
$ kubectl get csv -n operators
```

NAME	DISPLAY	VERSION	REPLACES
argocd-operator.v0.13.0	Argo CD	0.13.0	argocd-operator.v0.12.0
Succeeded			

You are now ready to use the operator. Please refer to the next sections in the chapter to interact with the installed CRDs.

Working with Custom Resource Definitions (CRDs)

For the exam, you will need to understand how to discover CRD schemas provided by external operators and how to interact with objects that follow the CRD schema.

Discovering CRDs

The Argo CD Operator provides a couple of CRDs like the Application, ApplicationSet, AppProject, and some more. You can find their high-level purpose described below.

- **Application:** An Application is a group of Kubernetes resources as defined by a manifest.
- **ApplicationSet:** An ApplicationSet is a group or set of Application resources.
- **AppProject:** An AppProject is a logical grouping of Argo CD Applications.

Run the following command to list all installed CRDs. You will find the Argo CD CRDs in the command output.

```
$ kubectl get crds
NAME                                     CREATED AT
applications.argoproj.io                2025-03-21T23:02:40Z
applicationsets.argoproj.io            2025-03-21T23:02:39Z
appprojects.argoproj.io                2025-03-21T23:02:39Z
argocdexports.argoproj.io              2025-03-21T23:02:39Z
argocds.argoproj.io                    2025-03-21T23:02:39Z
notificationsconfigurations.argoproj.io 2025-03-21T23:02:39Z
```

As with any other Kubernetes object, you can render the details. The details of the CRD will reveal the kind, API group and version, and its properties. The command below inspects the Application CRD.

```
$ kubectl describe crd applications.argoproj.io
```

For brevity, I do not show the lengthy output. In the next section, we are going to create a CR for the Application CRD.

Instantiating a CR for one the CRDs

The Application CRD describes a schema for defining an application representation that lives in a Git repository so that it can be deployed to a Kubernetes cluster.

For that purpose, create a new YAML manifest of kind Application in the file

`nginx-application.yaml`. as shown in [Example 7-1](#). You may recognize some of the properties used when you rendered the CRD schema.

Example 7-1. Instantiation of CR for the Application CRD

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: nginx
spec:
  project: default
  source:
    repoURL: https://github.com/bmuschko/cka-study-guide.git
    targetRevision: HEAD
    path: ./ch07/nginx
  destination:
    server: https://kubernetes.default.svc
    namespace: default
```

Create a new object from the YAML manifest.

```
kubectl apply -f nginx-application.yaml
application.argoproj.io/nginx created
```

Interacting with a CR

You can interact with the CR like any other object in Kubernetes. All `kubectl` create, read, update, and delete (CRUD) functions are available. For example, to list the object use the `describe` command. The following command show the operations in action:

```
$ kubectl describe application nginx
Name:          nginx
Namespace:     default
Labels:        <none>
Annotations:   <none>
API Version:   argoproj.io/v1alpha1
Kind:          Application
...
```

To delete the object, use the `delete` command, as shown below.

```
$ kubectl delete application nginx
application.argoproj.io "nginx" deleted
```

We demonstrated that an operator can install CRDs into the cluster, and that we can create CR from the CRDs and interact with them.

Going deeper using Argo CD would require a chapter on its own and is beyond the coverage of the certification. For more information on Argo CD, check out the book *Argo CD: Up and Running* by Andrew Block and Christian Hernandez (O'Reilly).

Inspecting the controller

The Argo CD CRs just represent data and won't be useful by themselves. A controller acts as a reconciliation process by inspecting the state of CR objects via calls to the Kubernetes API to perform a deployment process to the cluster.

The Argo CD Operator runs the controller logic within a Pod managed by a Deployment. You can discover the Argo CD controller objects, as follows.

```
$ kubectl get deployments,pods -n operators
```

NAME	READY	UP-TO-
DATE ...		
deployment.apps/argocd-operator-controller-manager	1/1	1
...		

NAME	READY
STATUS ...	
pod/argocd-operator-controller-manager-6998544bff-zx8bg	1/1
Running ...	

Because we installed the operator using OLM, the controller Pod would be placed into the `operators` namespace to keep them separate from other objects in the cluster. You can scale the number of replicas as needed by changing the configuration of the Deployment.

Summary

Kubernetes refers to the CRD and the corresponding controller as the operator pattern. The Kubernetes community has implemented many operators to fulfill custom requirements. You can install them into your cluster to reuse the functionality.

A CRD schema defines the structure of a custom resource. The schema includes the group, name, version, and its configurable attributes. New objects of this kind, so-called CRs, can be created after registering the schema. You can interact with a custom object using `kubectl` with the same CRUD commands used by any other primitive.

CRDs realize their full potential when combined with a controller implementation. The controller implementation inspects the state of specific custom objects and reacts based on their discovered state.

Exam Essentials

Know how to install operators

Operators built and managed by the Kubernetes community are available on searchable sites like Artifact Hub and Operator Hub. You will find installation instructions on the correspond webpages. You do not have to memorize them for the exam. If you want to explore further, install an open source operator, such as the **Prometheus operator** or the **Jaeger operator**.

Acquire a high-level understanding of configurable options for a CRD schema

You are not expected to implement a custom CRD schema. All you need to know is how to discover and interact with them using `kubectl`. Practice the definition of a CR in the form of a YAML manifest, and create the objects for it. Controller implementations are definitely outside the scope of the exam.

Sample Exercises

Solutions to these exercises are available in **Appendix A**.

1. You decide to manage a **MongoDB** installation in Kubernetes with the help of the **official community operator**. This operator provides a CRD. After installing the operator, you will interact with the CRD.

Navigate to the directory `app-a/ch07/mongodb-operator` of the

checked-out GitHub repository *[bmuschko/cka-study-guide](#)*. Install the operator using the following command: `kubectl apply -f mongodbcommunity.mongodb.com_mongodbcommunity.yaml`.

List all CRDs using the appropriate `kubectl` command. Can you identify the CRD that was installed by the installation procedure?

Inspect the schema of the CRD. What are the type and property names of this CRD?

2. Navigate to the directory *[app-a/ch07/backup-crd](#)* of the checked-out GitHub repository *[bmuschko/cka-study-guide](#)*.

Create the CRD from the file `backup-resource.yaml`. Retrieve the details for the `Backup` custom resource created in the previous step.

Create a CR named `nginx-backup` for the CRD in the `default` namespace. Provide the following property values:

- `cronExpression: 0 0 * * *`
- `podName: nginx`
- `path: /usr/local/nginx`

Retrieve the details for the `nginx-backup` object created in the previous step.

Chapter 8. Helm and Kustomize

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles. This will be the 8th chapter of the final book. Please note that the GitHub repo will be made active later on. If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at ccollins@oreilly.com.

Kubernetes objects can be created, modified, and deleted by using imperative `kubectl` commands or by running a `kubectl` command against a configuration file declaring the desired state of an object, a so-called manifest. The primary definition language of a manifest is YAML, though you can opt for JSON, which is the less widely adopted format among the Kubernetes community. It’s recommended that development teams commit and push those configuration files to version control repositories as it will help with tracking and auditing changes over time.

Modeling an application in Kubernetes often requires a set of supporting objects, each of which can have their own manifest. For example, you may want to create a Deployment that runs the application on five Pods, a ConfigMap to inject configuration data as environment variables, and a Service for exposing network access.

It is not practical to manage a full application stack by running individual `kubectl` commands. That is where open source tools like Helm and Kustomize come into play. They allow you conveniently manage the lifecycle of application stacks and cluster components as one single unit, while at the same time allow for customization at the time of deployment.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Use Helm and Kustomize to install cluster components

Working with Helm

Helm is a templating engine and package manager for a set of Kubernetes manifests. At runtime, it replaces placeholders in YAML template files with actual, end-user-defined values. The artifact produced by the Helm executable is a so-called *chart file* bundling the manifests that comprise the API resources of an application. You can upload the chart file to a *chart repository* so that other teams can be use it to deploy the bundled manifests. The Helm ecosystem offers a wide range of reusable charts for common use cases searchable on **Artifact Hub** (for example, for running Grafana or PostgreSQL).

Due to the wealth of functionality available to Helm, we'll discuss only the basics. The exam does not expect you be a Helm expert; rather, it wants you to be familiar with the workflow of installing existing packages with Helm. Building and publishing your own charts is outside the scope of the exam. For more detailed information on Helm, see the **user documentation**. The version of Helm used to describe the functionality here is 3.17.2.

Managing an Existing Chart

As a developer, you want to reuse existing functionality instead of putting in the work to define and configure it yourself. For example, you may want to install the open source monitoring service Prometheus on your cluster.

Prometheus requires the installation of multiple Kubernetes primitives. Thankfully, the Kubernetes community provided a Helm chart making it very easy to install and configure all the moving parts in the form of a **Kubernetes operator**. Revisit **Chapter 7** to refresh your memory on the moving parts of the operator pattern.

The following list shows the typical workflow for consuming and managing a Helm chart. Most of those steps need to use the `helm` executable:

1. Identifying the chart you'd like to install
2. Adding the repository containing the chart
3. Installing the chart from the repository
4. Verifying the Kubernetes objects that have been installed by the chart
5. Rendering the list of installed charts
6. Upgrading an installed chart
7. Uninstalling a chart if its functionality is no longer needed

The following sections will explain each of the steps.

Identifying a Chart

Over the years, the Kubernetes community implemented and published thousands of Helm charts. Artifact Hub provides a web-based search capability for discovering charts by keyword.

Say you wanted to find a chart that installs the Continuous Integration solution Jenkins. All you'd need to do is to enter the term "jenkins" into the search box and press the enter key. **Figure 8-1** shows the list of results in Artifact Hub.

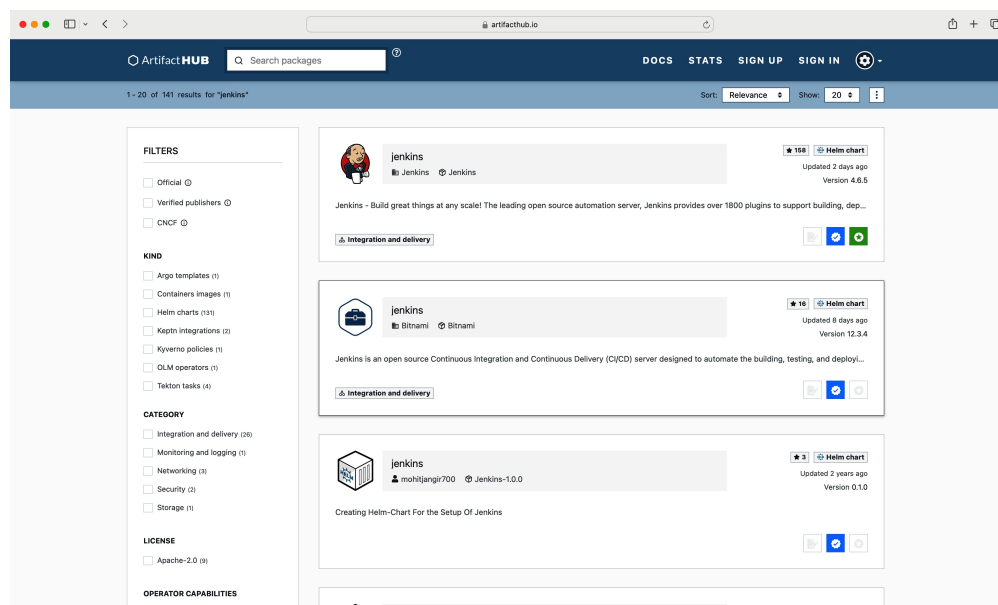


Figure 8-1. Searching for a Jenkins chart on Artifact Hub

At the time of writing, there are 141 matches for the search term. You will be able to inspect details about the chart by clicking on one of the search results, which includes a high-level description and the repository that the chart file resides in. Moreover, you can inspect the templates bundled with the chart file, indicating the objects that will be created upon installation and their configuration options. **Figure 8-2** shows the page for the official Jenkins chart.



jenkins

Helm chart

Jenkins Jenkins

Helm chart Integration and delivery

☆ Star 158

Jenkins - Build great things at any scale! The leading open source automation server, Jenkins provides over 1800 plugins to support building, deploying and automating any project.

 SUBSCRIPTIONS: 42 & WEBHOOKS: 3 & PRODUCTION USERS: 4

Jenkins

 Report issue

↓ **INSTALL**

TEMPLATES

⌘ DEFAULT VALUES

CHANGELOG

Artifact Hub jenkins License Apache 2.0 downloads 7.6M glitter join chat

Jenkins is the leading open source automation server, Jenkins provides over 1800 plugins to support building, deploying and automating any project.

This chart installs a Jenkins server which spawns agents on [Kubernetes](#) utilizing the [Jenkins Kubernetes plugin](#).

Inspired by the awesome work of [Carlos Sanchez](#).

Get Repository Info

```
helm repo add jenkins https://charts.jenkins.io
helm repo update
```

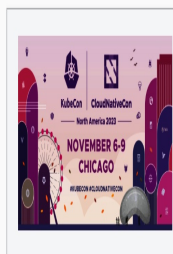
See `helm repo` for command documentation.

Install Chart

```
# Helm 3
$ helm install [RELEASE NAME] jenkins/jenkins [flags]
```

See [configuration below](#).

See [helm install](#) for command documentation.



APPLICATION VERSION

2.414.2

[CHART VERSIONS](#) [RSS](#)

4.6.5 (26 Sep, 2023)

4.6.4 (8 Sep, 2023)

4.6.3 (7 Sep, 2023)

[+ See all \(530\)](#)

LAST YEAR ACTIVITY



Figure 8-2. Jenkins chart details

You cannot install a chart directly from Artifact Hub. You must install it from the repository hosting the chart file.

Adding a Chart Repository

The chart description may mention the repository that hosts the chart file. Alternatively, you can click on the “Install” button to render repository details and the command for adding it. **Figure 8-3** shows the contextual pop-up that appears after clicking the “Install” button.

By default, a Helm installation defines no external repositories. The following command shows how to list all registered repositories. No repositories have been registered yet:

```
$ helm repo list  
Error: no repositories to show
```

ArtifactHUB

Search packages

DOCSSTATSSIGN UPSIGN IN



jenkins

Helm chart

Integration and delivery

Jenkins

Jenkins

Star

158

Jenkins - Build great things at any scale! The leading open source automation server, Jenkins provides over 1800 plugins to support building, deploying and automating any project.

SUBSCRIPTION

Jenkins

Artifact HubjenkinsLicenseApache 2

Jenkins is the leading open source automation server project.

This chart installs a Jenkins server with all the plugins you need.

Inspired by the awesome work of Carl.

Get Repository Info

```
helm repo add jenkins https://charts.jenkins.io/
helm repo update
```

See [helm repo](#) for command documentation.

Install Chart

```
# Helm 3
$ helm install [RELEASE_NAME] jenkins/jenkins [flags]
```

See configuration below.

See [helm install](#) for command documentation.

jenkins

Helm v3

Add repository

```
helm repo add jenkinsci https://charts.jenkins.io/
```

Install chart

```
helm install my-jenkins jenkinsci/jenkins --version 4.6.5
```

my-jenkins corresponds to the release name, feel free to change it to suit your needs. You can also add additional flags to the helm install command if you need to.

Need Helm?

You can also download this package's content directly using [this link](#).

CLOSE

INSTALL

TEMPLATES

DEFAULT VALUES

CHANGELOG



APPLICATION VERSION

2.414.2

CHART VERSIONS

RSS

4.6.5 (26 Sep, 2023)

4.6.4 (8 Sep, 2023)

4.6.3 (7 Sep, 2023)

See all (530)

LAST YEAR ACTIVITY

Figure 8-3. Jenkins chart installation instructions

As you can see from the screenshot, the chart file lives in the repository with the URL <https://charts.jenkins.io>. We will need to add this repository. This is an one-time operation. You can install other charts from that repository or you can update a chart that originated from that repository with commands we'll discuss in a later section.

You need to provide a name for the repository when registering one. Make the repository name as descriptive as possible. The following command registers the repository with the name `jenkinsci`:

```
$ helm repo add jenkinsci https://charts.jenkins.io/
"jenkinsci" has been added to your repositories
```

Listing the repositories now shows the mapping between name and URL:

```
$ helm repo list
NAME      URL
jenkinsci https://charts.jenkins.io/
```

You permanently added the repository to the Helm installation.

Searching for a Chart in a Repository

The “Install” pop-up window already provided the command to install the chart. You can also search the repository for available charts in case you do not know their names or latest versions. Add the `--versions` flag to list all available versions:

```
$ helm search repo jenkinsci
NAME                CHART VERSION  APP VERSION  DESCRIPTION
jenkinsci/jenkins  5.8.26         2.492.2      ...
```

At the time of writing, the latest version available is 5.8.26. This may be different if you run the command on your machine, given that the Jenkins project may have released a newer version.

Installing a Chart

Let's assume that the latest version of the Helm chart contains a security vulnerability. Therefore, we decide to install the Jenkins chart with the previous version, 5.8.25. You need to assign a name to be able to identify an installed chart. The name we'll use here is `my-jenkins`:

```
$ helm install my-jenkins jenkinsci/jenkins --version 5.8.25
NAME: my-jenkins
LAST DEPLOYED: Wed Mar 26 13:48:50 2025
NAMESPACE: default
STATUS: deployed
REVISION: 1
...
```

The chart automatically created the Kubernetes objects in the `default` namespace. You can use the following command to discover the most important resource types:

```
$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/my-jenkins-0	2/2	Running	0	12m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
...			
service/my-jenkins	ClusterIP	10.99.166.189	<none>
...			
service/my-jenkins-agent	ClusterIP	10.110.246.141	<none>
...			

NAME	READY	AGE
statefulset.apps/my-jenkins	1/1	12m

The chart has been installed with the default configuration options. You can inspect those default values by clicking on the “Default Values” button on the chart page, as shown in [Figure 8-4](#).

Default values

Compare to version ...

Search path



```
1 # Default values for jenkins.
2 # This is a YAML-formatted file.
3 # Declare name/value pairs to be passed into your templates.
4 # name: value
5
6 ## Overrides for generated resource names
7 # See templates/_helpers.tpl
8 # nameOverride:
9 # fullnameOverride:
10 # namespaceOverride:
11
12 # For FQDN resolving of the controller service. Change this value to match your existing configuration.
13 # ref: https://github.com/kubernetes/dns/blob/master/docs/specification.md
14 clusterZone: "cluster.local"
15
16 # The URL of the Kubernetes API server
17 kubernetesURL: "https://kubernetes.default"
18
19 renderHelmLabels: true
20
```



X CLOSE

Helm 3

\$ helm install [RELEASE_NAME] jenkins/jenkins [flags]

4.6.4 (8 Sep, 2023)

4.6.3 (7 Sep, 2023)

See all (530)

Figure 8-4. Jenkins chart default values

You can also discover those configuration options using the following command. The output shown renders only a subset of values, the admin username and its password, represented by `controller.adminUser` and `controller.adminPassword`:

```
$ helm show values jenkinsci/jenkins
...
controller:
  # When enabling LDAP or another non-Jenkins identity source, the
  built-in \
  # admin account will no longer exist.
  # If you disable the non-Jenkins identity store and instead use the
  Jenkins \
  # internal one,
  # you should revert controller.adminUser to your preferred admin
  user:
    adminUser: "admin"
    # adminPassword: <defaults to random>
  ...
```

You can customize any configuration value when installing the chart. To pass configuration data during the install processing use one of the following flags:

- `--values`: Specifies the overrides in the form of a pointer to a YAML manifest file.
- `--set`: Specifies the overrides directly from the command line.

For more information, see [“Customizing the Chart Before Installing”](#) in the Helm documentation.

You can decide to install the chart into a custom namespace. Use the `-n` flag to provide the name of an existing namespace. Add the flag `--create-namespace` to automatically create the namespace if it doesn’t exist yet.

The following command shows how to customize some of the values and the namespace used during the installation process:

```
$ helm install my-jenkins jenkinsci/jenkins --version 4.6.4 \
--set controller.adminUser=boss --set
controller.adminPassword=password \
```

```
-n jenkins --create-namespace
```

We specifically set the username and the password for the admin user. Helm created the objects controlled by the chart into the `jenkins` namespace.

Listing Installed Charts

Charts can live in the `default` namespace or a custom namespace. You can inspect the list of installed charts using the `helm list` command. If you do not know which namespace, simply add the `--all-namespaces` flag to the command:

```
$ helm list --all-namespaces
NAME          NAMESPACE  REVISION  UPDATED           STATUS  CHART
my-jenkins    default    1         2023-09-28...    deployed
jenkins-4.6.4
```

The output of the command includes the column `NAMESPACE` that shows the namespace used by a particular chart. Similar to the use of `kubectl`, the `helm list` command provides the option `-n` for spelling out a namespace. Providing no flag(s) with the command will return the result for the `default` namespace.

Upgrading an Installed Chart

Upgrading an installed chart usually means moving to a new chart version. You can poll for new versions available in the repository by running this command:

```
$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "jenkinsci" chart repository
Update Complete. *Happy Helming!*
```

What if you want to upgrade your existing chart installation to a newer chart version? Run the following command to upgrade the chart to that specific version with the default configuration:

```
$ helm upgrade my-jenkins jenkinsci/jenkins --version 5.8.26
Release "my-jenkins" has been upgraded. Happy Helming!
...
```


As with the `install` command, you will have to provide custom configuration values if you want to tweak the chart's runtime behavior when upgrading a chart.

Uninstalling a Chart

Sometimes you no longer need to run a chart. The command for uninstalling a chart is straightforward, as shown here. It will delete all objects controlled by the chart. Don't forget to provide the `-n` flag if you previously installed the chart into a namespace other than `default`:

```
$ helm uninstall my-jenkins
release "my-jenkins" uninstalled
```

Executing the command may take up to 30 seconds, as Kubernetes needs to wait for the workload grace period to end.

Working with Kustomize

Kustomize is a tool introduced with Kubernetes 1.14 that aims to make manifest management more convenient. It supports three different use cases:

- Generating manifests from other sources. For example, creating a ConfigMap and populating its key-value pairs from a properties file.
- Adding common configuration across multiple manifests. For example, adding a namespace and a set of labels for a Deployment and a Service.
- Composing and customizing a collection of manifests. For example, setting resource boundaries for multiple Deployments.

The central file needed for Kustomize to work is the *kustomization file*. The standardized name for the file is `kustomization.yaml` and cannot be changed. A kustomization file defines the processing rules Kustomize works upon.

Kustomize is fully integrated with `kubectl` and can be executed in two modes: rendering the processing output on the console or creating the objects. Both modes can operate on a directory, tarball, Git archive, or URL as long as they

contain the kustomization file and referenced resource files:

Rendering the produced output

The first mode uses the `kustomize` subcommand to render the produced result on the console but does not create the objects. This command works similar to the dry-run option you might know from the `run` command:

```
$ kubect1 kustomize <target>
```

Creating the objects

The second mode uses the `apply` command in conjunction with the `-k` command-line option to apply the resources processed by Kustomize, as explained in the previous section:

```
$ kubect1 apply -k <target>
```

The following sections demonstrate each of the use cases by a single example. For a full coverage on all possible scenarios, refer to the [documentation](#) or the [Kustomize GitHub repository](#).

Composing Manifests

One of the core functionalities of Kustomize is to create a composed manifest from other manifests. Combining multiple manifests into a single one may not seem that useful by itself, but many of the other features described later will build upon this capability. Say you wanted to compose a manifest from a Deployment and a Service resource file. All you need to do is to place the resource files into the same folder as the kustomization file:

```
.  
├── kustomization.yaml  
├── web-app-deployment.yaml  
└── web-app-service.yaml
```

The kustomization file lists the resources in the `resources` section, as shown

in **Example 8-1**.

Example 8-1. A kustomization file combining two manifests

resources:

- web-app-deployment.yaml
- web-app-service.yaml

As a result, the `kustomize` subcommand renders the combined manifest containing all of the resources separated by three hyphens (`- - -`) to denote the different object definitions:

```
$ kubectl kustomize ./
apiVersion: v1
kind: Service
metadata:
  labels:
    app: web-app-service
    name: web-app-service
spec:
  ports:
    - name: web-app-port
      port: 3000
      protocol: TCP
      targetPort: 3000
  selector:
    app: web-app
  type: NodePort
- - -
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: web-app-deployment
    name: web-app-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - env:
            - name: DB_HOST
```

```
    value: mysql-service
  - name: DB_USER
    value: root
  - name: DB_PASSWORD
    value: password
image: bmuschko/web-app:1.0.1
name: web-app
ports:
  - containerPort: 3000
```

Generating Manifests from Other Sources

Earlier in this chapter, we learned that ConfigMaps and Secrets can be created by pointing them to a file containing the actual configuration data for it. Kustomize can help with the process by mapping the relationship between the YAML manifest of those configuration objects and their data. Furthermore, we'll want to inject the created ConfigMap and Secret in a Pod as environment variables. In this section, you will learn how to achieve this with the help of Kustomize.

The following file and directory structure contains the manifest file for the Pod and the configuration data files we need for the ConfigMap and Secret. The mandatory kustomization file lives on the root level of the directory tree:

```
.
├── config
│   ├── db-config.properties
│   └── db-secret.properties
├── kustomization.yaml
└── web-app-pod.yaml
```

In `kustomization.yaml`, you can define that the ConfigMap and Secret object should be generated with the given name. The name of the ConfigMap is supposed to be `db-config`, and the name of the Secret is going to be `db-creds`. Both of the generator attributes, `configMapGenerator` and `secretGenerator`, reference an input file used to feed in the configuration data. Any additional resources can be spelled out with the `resources` attribute. [Example 8-2](#) shows the contents of the kustomization file.

Example 8-2. A kustomization file using a ConfigMap and Secret generator

```
configMapGenerator:
- name: db-config
  files:
```

- config/db-config.properties

secretGenerator:

- **name:** db-creds
- files:**
 - config/db-secret.properties

resources:

- web-app-pod.yaml

Kustomize generates ConfigMaps and Secrets by appending a suffix to the name. You can see this behavior when creating the objects using the **apply** command. The ConfigMap and Secret can be referenced by name in the Pod manifest:

```
$ kubectl apply -k ./
configmap/db-config-t4c79h4mtt unchanged
secret/db-creds-4t9dmgtf9h unchanged
pod/web-app created
```

This naming strategy can be configured with the attribute **generatorOptions** in the kustomization file. See the [documentation](#) for more information.

Let's also try the **kustomize** subcommand. Instead of creating the objects, the command renders the processed output on the console:

```
$ kubectl kustomize ./
apiVersion: v1
data:
  db-config.properties: |-
    DB_HOST: mysql-service
    DB_USER: root
kind: ConfigMap
metadata:
  name: db-config-t4c79h4mtt
---
apiVersion: v1
data:
  db-secret.properties: REJFUEFTU1dPUkQ6IGNHRnpjM2R2Y21RPQ==
kind: Secret
metadata:
  name: db-creds-4t9dmgtf9h
type: Opaque
---
apiVersion: v1
kind: Pod
```

```

metadata:
  labels:
    app: web-app
    name: web-app
spec:
  containers:
  - envFrom:
    - configMapRef:
        name: db-config-t4c79h4mtt
    - secretRef:
        name: db-creds-4t9dmgtf9h
    image: bmuschko/web-app:1.0.1
    name: web-app
    ports:
    - containerPort: 3000
      protocol: TCP
    restartPolicy: Always

```

Adding Common Configuration Across Multiple Manifests

Application developers usually work on an application stack set comprised of multiple manifests. For example, an application stack could consist of a frontend microservice, a backend microservice, and a database. It's common practice to use the same, cross-cutting configuration for each of the manifests. Kustomize offers a range of supported fields (e.g., namespace, labels, or annotations). Refer to the [documentation](#) to learn about all supported fields.

For the next example, we'll assume that a Deployment and a Service live in the same namespace and use a common set of labels. The namespace is called **persistence** and the label is the key-value pair **team: helix**. [Example 8-3](#) illustrates how to set those common fields in the kustomization file.

Example 8-3. A kustomization file using a common field

```

namespace: persistence
commonLabels:
  team: helix
resources:
- web-app-deployment.yaml
- web-app-service.yaml

```

To create the referenced objects in the kustomization file, run the **apply** command. Make sure to create the **persistence** namespace beforehand:

```
$ kubectl create namespace persistence
```

```
namespace/persistence created
$ kubectl apply -k ./
service/web-app-service created
deployment.apps/web-app-deployment created
```

The YAML representation of the processed files looks as follows:

```
$ kubectl kustomize ./
apiVersion: v1
kind: Service
metadata:
  labels:
    app: web-app-service
    team: helix
  name: web-app-service
  namespace: persistence
spec:
  ports:
  - name: web-app-port
    port: 3000
    protocol: TCP
    targetPort: 3000
  selector:
    app: web-app
    team: helix
  type: NodePort
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: web-app-deployment
    team: helix
  name: web-app-deployment
  namespace: persistence
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
      team: helix
  template:
    metadata:
      labels:
        app: web-app
        team: helix
    spec:
      containers:
```

- env:
 - name: DB_HOST
 - value: mysql-service
 - name: DB_USER
 - value: root
 - name: DB_PASSWORD
 - value: password
- image: bmuschko/web-app:1.0.1
- name: web-app
- ports:
 - containerPort: 3000

Customizing a Collection of Manifests

Kustomize can merge the contents of a YAML manifest with a code snippet from another YAML manifest. Typical use cases include adding security context configuration to a Pod definition or setting resource boundaries for a Deployment. The kustomization file allows for specifying different patch strategies like `patchesStrategicMerge` and `patchesJson6902`. For a deeper discussion on the differences between patch strategies, refer to the documentation.

Example 8-4 shows the contents of a kustomization file that patches a Deployment definition in the file `nginx-deployment.yaml` with the contents of the file `security-context.yaml`.

Example 8-4. A kustomization file defining a patch

resources:

- nginx-deployment.yaml

patchesStrategicMerge:

- security-context.yaml

The patch file shown in **Example 8-5** defines a security context on the container-level for the Pod template of the Deployment. At runtime, the patch strategy tries to find the container named `nginx` and enhances the additional configuration.

Example 8-5. The patch YAML manifest

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-deployment

spec:

template:

spec:


```
containers:
- name: nginx
  securityContext:
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
```

The result is a patched Deployment definition, as shown in the output of the `kustomize` subcommand shown next. The patch mechanism can be applied to other files that require a uniform security context definition:

```
$ kubectl kustomize ./
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: nginx
    name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.14.2
        name: nginx
        ports:
        - containerPort: 80
        securityContext:
          fsGroup: 2000
          runAsGroup: 3000
          runAsUser: 1000
```

Key Differences Between Helm and Kustomize

On the surface, Helm and Kustomize may seem to solve the same problems. Here, we want to identify the key difference between those tools so that you can make an educated decision based on the use cases you are trying to fulfill.

Ease of use

Kustomize is bundled with the `kubectl` command line. You do not need to install another tool, nor do you have learn another templating engine.

Helm requires the installation of an executable and requires you to become familiar with its commands and workflow.

Learning curve

Kustomize builds upon the knowledge of an administrator or developer familiar with writing Kubernetes YAML manifests.

Helm has a steeper learning curse. You will have to become familiar with its package management system, the notation for its templating engine, and how to pass in user-defined values upon invocation.

Packaging

Kustomize doesn't require the end user to produce an archive file. All you need is a set of YAML manifests and the `kustomization.yaml` file which you would check into a Git repository.

Helm on the other hand requires the creation of a metadata file named `Chart.yaml`, default values represented in a file named `values.yaml`, and a set of template files in a `templates` subdirectory. To be able to distribute the Helm chart, you'll need to package it into a TAR file.

Release versioning

Kustomize solely focusing on generating the desired state in a cluster through YAML manifests. You can track changes over time by Git commit hashes or tags to indicate a version.

Helm's rigid project structure requires the definition of a chart version inside of the `Chart.yaml` file. Every time you make a change, you'd bump up the version number, often represented by semantic versioning.

Helm and Kustomize are Kubernetes tools use to automate the process of deploying objects into a cluster. Carefully analyze your technical requirements, business objectives, and the skillset of team members before deciding for either of the tools. You may even determine both tools to manage application stacks in

Kubernetes. Most Kubernetes GitOps tools, e.g. Argo CD or Flux, support Helm and Kustomize.

Summary

Helm has evolved to become a de facto tool for deploying application stacks to Kubernetes. The artifact that contains the manifest files, default configuration values, and metadata is called a chart. A team or an individual can publish charts to a chart repository. Users can discover a published chart through the Artifact Hub user interface and install it to a Kubernetes cluster.

One of the primary developer workflows when using Helm consists of finding, installing, and upgrading a chart with a specific version. You start by registering the repository containing chart files you want to consume. The `helm install` command downloads the chart file and stores it in a local cache. It also creates the Kubernetes objects described by the chart.

The installation process is configurable. A developer can provide overrides for customizable configuration values. The `helm upgrade` command lets you upgrade the version of an already installed chart. To uninstall a chart and delete all Kubernetes objects managed by the chart, run the `helm uninstall` command.

Additional tools emerged for more convenient manifest management. Kustomize is fully integrated with the `kubectl` tool chain. It helps with the generation, composition, and customization of manifests.

Exam Essentials

Assume that the Helm and Kustomize executable is preinstalled

Unfortunately, the **exam FAQ** does not mention any details about the Helm and Kustomize executable. It's fair to assume that it will be preinstalled for you and therefore you do not need to memorize installation instructions.

Become familiar with Artifact Hub

Artifact Hub provides a web-based UI for Helm charts. It's worthwhile to

explore the search capabilities and the details provided by individual charts, more specifically the repository the chart file lives in, and its configurable values. During the exam, you'll likely not be asked to navigate to Artifact Hub because its URL hasn't been listed as one of the permitted documentation pages. You can assume that the exam question will provide you with the repository URL.

Practice commands needed to consume existing Helm charts

The exam does not ask you to build and publish your own chart file. All you need to understand is how to consume an existing chart. You will need to be familiar with the `helm repo add` command to register a repository, the `helm search repo` to find available chart versions, and the `helm install` command to install a chart. You should have a basic understanding of the upgrade process for an already installed Helm chart using the `helm upgrade` command.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. In this exercise, you will use Helm to install Kubernetes objects needed for the open source monitoring solution **Prometheus**. The easiest way to install Prometheus on top of Kubernetes is with the help of the **prometheus-operator** Helm chart.

You can search for the **kube-prometheus-stack** on Artifact Hub. Add the repository to the list of known repositories accessible by Helm with the name **prometheus-community**.

Update to the latest information about charts from the respective chart repository.

Run the Helm command for listing available Helm charts and their versions. Identify the latest chart version for **kube-prometheus-stack**.

Install the the chart **kube-prometheus-stack**. List the installed

Helm chart.

List the Service named `prometheus-operated` created by the Helm chart. The object resides in the `default` namespace.

Use the `kubectl port-forward` command to forward the local port 8080 to the port 9090 of the Service. Open a browser and bring up the Prometheus dashboard.

Stop port forwarding and uninstall the Helm chart.

2. Create the directory named `manifests`. Within the directory, create two files: `pod.yaml` and `configmap.yaml`. The `pod.yaml` file should define a Pod named `nginx` with the image `nginx:1.21.1`. The `configmap.yaml` file defines a ConfigMap named `logs-config` with the key-value pair `dir=/etc/logs/traffic.log`. Create both objects with a single, declarative command.

Modify the ConfigMap manifest by changing the value of the key `dir` to `/etc/logs/traffic-log.txt`. Apply the changes. Delete both objects with a single declarative command.

Use `Kustomize` to set a common namespace `t012` for the resource file `pod.yaml`. The file `pod.yaml` defines the Pod named `nginx` with the image `nginx:1.21.1` without a namespace. Run the `Kustomize` command that renders the transformed manifest on the console.

Appendix A. Answers to Review Questions

Chapter 4, Cluster Installation and Upgrade

1. List the cluster nodes using the following command. You should see

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	2m20s	v1.32.2
minikube-m02	Ready	<none>	2m10s	v1.32.2
minikube-m03	Ready	<none>	2m3s	v1.32.2
minikube-m04	Ready	<none>	116s	v1.32.2

Schedule the Pod using the following imperative command.

```
$ kubectl run nginx --image=nginx:1.27.4-alpine
```

You can identify the node the Pod is running, as shown below. The Pod is running on the node named `minikube-m02`.

```
$ kubectl get pod nginx -o jsonpath='{.spec.nodeName}'  
minikube-m02
```

Use the `kubectl drain` command to evict all Pods on a node. Here, the node name is `minikube-m02`. You'll need to use the `--ignore-daemonsets` flag to only delete Pods that are not managed by a `DaemonSet`, and the `--force` flag to delete Pods without a controller.

```
$ kubectl drain minikube-m02 --ignore-daemonsets --force  
evicting pod default/nginx  
pod/nginx evicted  
node/minikube-m02 drained
```

2. The solution to this sample exercise requires a lot of manual steps. The following commands do not render their output.

Open an interactive shell to the control plane node using Vagrant:

```
$ vagrant ssh kube-control-plane
```

Upgrade kubeadm to version 1.32.2 and apply it:

```
$ sudo apt-mark unhold kubeadm && sudo apt-get update && sudo  
apt-get \  
install -y kubeadm=1.32.2-1.1 && sudo apt-mark hold kubeadm  
$ sudo kubeadm upgrade apply v1.32.2
```

Drain the node, upgrade the kubelet and kubectl, restart the kubelet, and uncordon the node:

```
$ kubectl drain kube-control-plane --ignore-daemonsets  
$ sudo apt-get update && sudo apt-get install -y \  
--allow-change-held-packages kubelet=1.32.2-1.1  
kubectl=1.32.2-1.1  
$ sudo systemctl daemon-reload  
$ sudo systemctl restart kubelet  
$ kubectl uncordon kube-control-plane
```

The version of the node should now say v1.32.2. Exit the node:

```
$ kubectl get nodes  
$ exit
```

Open an interactive shell to the worker node using Vagrant. Repeat all of the following steps for the other worker nodes:

```
$ vagrant ssh kube-worker-1
```

Upgrade kubeadm to version 1.32.2 and apply it to the node:

```
$ sudo apt-get update && sudo apt-get install -y \  
kubeadm=1.32.2-1.1 && sudo apt-mark hold kubeadm
```



```
--allow-change-held-packages kubeadm=1.32.2-1.1
$ sudo kubeadm upgrade node
```

Drain the node, upgrade the kubelet and `kubect1`, restart the kubelet, and uncordon the node:

```
$ kubect1 drain kube-worker-1 --ignore-daemonsets
$ sudo apt-get update && sudo apt-get install -y \
  --allow-change-held-packages kubelet=1.32.2-1.1
kubect1=1.32.2-1.1
$ sudo systemctl daemon-reload
$ sudo systemctl restart kubelet
$ kubect1 uncordon kube-worker-1
```

The version of the node should now say v1.32.2. Exit the node:

```
$ kubect1 get nodes
$ exit
```

Chapter 5, Backing Up and Restoring etcd

1. Open an interactive shell to the control plane node using Vagrant.

```
$ vagrant ssh kube-control-plane
```

The etcd Pod runs in the `kube-system` namespace. Identify the Pod by listing all Pods in the namespace. You should find the Pod named `etcd-kube-control-plane`.

```
$ kubectl get pods -n kube-system
```

The etcd Pod only runs a single container. You can render the details of the Pod using the `kubectl get` or `kubectl describe` command. To select only the container image, use the following JSONPath.

```
$ kubectl get pod etcd-kube-control-plane -n kube-system \  
-o jsonpath="{.spec.containers[0].image}"
```

The container uses the image `registry.k8s.io/etcd:3.5.16-0`. The etcd version used is 3.5.16. Write the version to the file named `etcd-version.txt`. You can view all released etcd versions in the corresponding [GitHub repository](#).

```
$ echo "3.5.16" > etcd-version.txt
```

Exit the node:

```
$ exit
```

2. The solution to this sample exercise requires a lot of manual steps. The following commands do not render their output.

Open an interactive shell to the control plane node using Vagrant.

```
$ vagrant ssh kube-control-plane
```

Determine the parameters of the Pod `etcd-kube-control-plane` by describing it. Use the correct parameter values to create a snapshot file:

```
$ kubectl describe pod etcd-kube-control-plane -n kube-system  
$ sudo ETCDCTL_API=3 etcdctl --  
cacert=/etc/kubernetes/pki/etcd/ca.crt \  
--cert=/etc/kubernetes/pki/etcd/server.crt \  
--key=/etc/kubernetes/pki/etcd/server.key snapshot save  
/opt/etcd.bak
```

Restore the backup from the snapshot file. Edit the etcd YAML manifest and change the value of `spec.volumes.hostPath.path` for the volume named `etcd-data` to `/var/bak`:

```
$ sudo ETCDCTL_API=3 etcdctl --data-dir=/var/bak snapshot  
restore \  
/opt/etcd.bak  
$ sudo vim /etc/kubernetes/manifests/etcd.yaml
```

After a short while, the Pod `etcd-kube-control-plane` should transition back into the “Running” status. Exit the node:

```
$ kubectl get pod etcd-kube-control-plane -n kube-system  
$ exit
```

Chapter 6, Authentication, Authorization, and Admission Control

1. Use the `kubectl create clusterrole` command to create the Role imperatively:

```
$ kubectl create clusterrole service-view --verb=get,list \
  --resource=services
```

If you'd rather start with the YAML file, use content shown in the file *service-view-clusterrole.yaml*:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: service-view
rules:
- apiGroups: [""]
  resources: ["services"]
  verbs: ["get", "list"]
```

Create the ClusterRole from the YAML file:

```
$ kubectl apply -f service-view-clusterrole.yaml
```

Use the `kubectl create rolebinding` command to create the RoleBinding imperatively:

```
$ kubectl create rolebinding ellasmith-service-view --
user=ellasmith \
  --clusterrole=service-view -n development
```

The declarative approach of the RoleBinding could look like the one in the file *ellasmith-service-view-rolebinding.yaml*:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ellasmith-service-view
  namespace: development
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: service-view
subjects:
- kind: User
  name: ellasmith
  apiGroup: rbac.authorization.k8s.io

```

Create the RoleBinding from the YAML file:

```
$ kubectl apply -f ellasmith-service-view-rolebinding.yaml
```

Use the `kubectl create clusterrole` command to create the ClusterRole imperatively:

```
$ kubectl create clusterrole combined \
  --aggregation-rule="rbac.cka.cncf.com/aggregate=true"
```

If you'd rather start with the YAML file, use content shown in the file *combined-clusterrole.yaml*:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: combined
aggregationRule:
  clusterRoleSelectors:
  - matchLabels:
    rbac.cka.cncf.com/aggregate: "true"

```

```
rules: []
```

Create the ClusterRole from the YAML file:

```
$ kubectl apply -f combined-clusterrole.yaml
```

Rendering the rules shows an empty list as no ClusterRoles were selected by label selection:

```
$ kubectl describe clusterrole combined
Name:          combined
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  -----  -

```

Use the `kubectl create clusterrole` command to create the ClusterRole imperatively. The command does not provide an option for adding a label. Therefore, generating the YAML manifest using the `dry-run` option is a good start:

```
$ kubectl create clusterrole deployment-modify \
  --verb=create,delete,patch,update --resource=deployments \
  --dry-run=client -o yaml > deployment-modify-
clusterrole.yaml
```

Modify the content shown in the file *deployment-modify-clusterrole.yaml*:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: deployment-modify
  labels:
```

```

    rbac.cka.cncf.com/aggregate: "true"
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["create", "delete", "patch", "update"]

```

Create the ClusterRole from the YAML file:

```
$ kubectl apply -f deployment-modify-clusterrole.yaml
```

Rendering the rules for the Clusterrole combined lists the rules defined by the ClusterRole deployment-modify:

```
$ kubectl describe clusterrole combined
Name:          combined
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources            Non-Resource URLs  Resource Names  Verbs
  -----
  deployments.apps    []                  []              [create
\
  delete patch update]
```

Listing Services is allowed for the user `ellasmith` in the namespace `development`. This is taken care by RoleBinding `ellasmith-service-view`:

```
$ kubectl auth can-i list services --as=ellasmith --
namespace=development
yes
```

Write the output the file `list-services-ellasmith.txt`.

```
$ echo 'yes' > list-services-ellasmith.txt
```

Watching Deployments is not allowed for the user `ellasmith` in the namespace `production`. The ClusterRole named `deployment-modify` only allows the verbs `create`, `delete`, `patch`, and `update`.

```
$ kubectl auth can-i watch deployments --as=ellasmith --  
namespace=production  
no
```

Write the output the file *watch-deployments-ellasmith.txt*.

```
$ echo 'no' > watch-deployments-ellasmith.txt
```

Deployments are allowed to be created, deleted, patched, and updated.

2. First, create the namespace named `apps`. Then, we'll create the ServiceAccount:

```
$ kubectl create namespace apps  
$ kubectl create serviceaccount api-access -n apps
```

Alternatively, you can use the declarative approach. Create the namespace from the definition in the file `apps-namespace.yaml`:

```
apiVersion: v1  
kind: Namespace  
metadata:  
  name: apps
```

Create the namespace from the YAML file:

```
$ kubectl apply -f apps-namespace.yaml
```

Create a new YAML file called `api-serviceaccount.yaml` with the following contents:


```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api-access
  namespace: apps
```

Run the `apply` command to instantiate the ServiceAccount from the YAML file:

```
$ kubectl create -f api-serviceaccount.yaml
```

Use the `create clusterrole` command to create the ClusterRole imperatively:

```
$ kubectl create clusterrole api-clusterrole --
verb=watch,list,get \
--resource=pods
```

If you'd rather start with the YAML file, use content shown in the file `api-clusterrole.yaml`:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: api-clusterrole
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["watch", "list", "get"]
```

Create the ClusterRole from the YAML file:

```
$ kubectl apply -f api-clusterrole.yaml
```

Use the `create clusterrolebinding` command to create the

ClusterRoleBinding imperatively.

```
$ kubectl create clusterrolebinding api-clusterrolebinding \
  --serviceaccount=apps:api-access --clusterrole=api-
clusterrole
```

The declarative approach of the ClusterRoleBinding could look like the one in the file `api-clusterrolebinding.yaml`:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: api-clusterrolebinding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: api-clusterrole
subjects:
- apiGroup: ""
  kind: ServiceAccount
  name: api-access
  namespace: apps
```

Create the ClusterRoleBinding from the YAML file:

```
$ kubectl apply -f api-clusterrolebinding.yaml
```

Execute the `run` command to create the Pods in the different namespaces. You will need to create the namespace `rm` before you can instantiate the Pod `disposable`:

```
$ kubectl run operator --image=nginx:1.21.1 --restart=Never \
  --port=80 --serviceaccount=api-access -n apps
$ kubectl create namespace rm
$ kubectl run disposable --image=nginx:1.21.1 --restart=Never
```

```
\
-n rm
```

The following YAML manifest shows the `rm` namespace definition stored in the file `rm-namespace.yaml`:

```
apiVersion: v1
kind: Namespace
metadata:
  name: rm
```

The YAML representation of those Pods stored in the file `api-pods.yaml` could look as follows:

```
apiVersion: v1
kind: Pod
metadata:
  name: operator
  namespace: apps
spec:
  serviceAccountName: api-access
  containers:
  - name: operator
    image: nginx:1.21.1
    ports:
    - containerPort: 80
```

```
apiVersion: v1
kind: Pod
metadata:
  name: disposable
  namespace: rm
spec:
  containers:
  - name: disposable
```

```
image: nginx:1.21.1
```

Create the namespace and Pods from the YAML files:

```
$ kubectl create -f rm-namespace.yaml
$ kubectl create -f api-pods.yaml
```

Determine the API server endpoint and the Secret access token of the ServiceAccount. You will need this information for making the API calls:

```
$ kubectl config view --minify -o \
  jsonpath='{.clusters[0].cluster.server}'
https://192.168.64.4:8443
$ kubectl get secret $(kubectl get serviceaccount api-access -
n apps \
  -o jsonpath='{.secrets[0].name}') -o
jsonpath='{.data.token}' -n apps \
  | base64 --decode
eyJhbGciOiJSUzI1NiIsImtpZCI6Ii1hOUhI...
```

Open an interactive shell to the Pod named operator:

```
$ kubectl exec operator -it -n apps -- /bin/sh
```

Emit API calls for listing all Pods and deleting the Pod disposable living in the `rm` namespace. You will find that while the `list` operation is permitted, the `delete` operation isn't:

```
# curl https://192.168.64.4:8443/api/v1/namespaces/rm/pods --
header \
"Authorization: Bearer
eyJhbGciOiJSUzI1NiIsImtpZCI6Ii1hOUhI..." \
--insecure
{
```

```

        "kind": "PodList",
        "apiVersion": "v1",
        ...
    }
# curl -X DELETE https://192.168.64.4:8443/api/v1/namespaces \
/rm/pods/disposable --header \
"Authorization: Bearer
eyJhbGciOiJSUzI1NiIsImtpZCI6Ii1hOUhI..." \
--insecure
{
    "kind": "Status",
    "apiVersion": "v1",
    "metadata": {

    },
    "status": "Failure",
    "message": "pods \"disposable\" is forbidden: User \
\"system:serviceaccount:apps:api-access\" cannot delete \
resource \"pods\" in
API group \"\" in the namespace \"rm\"",
    "reason": "Forbidden",
    "details": {
        "name": "disposable",
        "kind": "pods"
    },
    "code": 403
}

```

Chapter 7, Custom Resource Definitions (CRDs) and Operators

1. Create the CRD from the provided URL:

```
$ kubectl apply -f https://raw.githubusercontent.com/mongodb/mongodb-kubernetes-operator/master/config/crd/bases/mongodbcommunity.mongodb.com_mongodbcommunity.yaml
customresourcedefinition.apiextensions.k8s.io/mongodbcommunity
.\
mongodbcommunity.mongodb.com created
```

You can find the installed CRD named `mongodbcommunity.mongodbcommunity.mongodb.com` with the following command:

```
$ kubectl get crds
NAME                                                                CREATED AT
mongodbcommunity.mongodbcommunity.mongodb.com 2023-12-18T23:44:04Z
```

One way to inspect the schema of the CRD is to use the `kubectl describe` command:

```
$ kubectl describe crds
mongodbcommunity.mongodbcommunity.mongodb.com
```

You will find that the output of the command is very lengthy. Looking through the details, you will see that the type is called `MongoDBCommunity`. The CRD offers a lot of properties that can be set when instantiating an object of this type. See the documentation of the operator for more information.

2. Create the object from the YAML manifest file:

```
$ kubectl apply -f backup-resource.yaml
customresourcedefinition.apiextensions.k8s.io/backups.example.com created
```

You can interact with CRD using the following command. Make sure to spell out the full name of the CRD, `backups.example.com`:

```
$ kubectl get crd backups.example.com
NAME                                CREATED AT
backups.example.com                2023-05-24T15:11:15Z
$ kubectl describe crd backups.example.com
...
```

Create the YAML manifest in file *backup.yaml* that uses the CRD kind Backup:

```
apiVersion: example.com/v1
kind: Backup
metadata:
  name: nginx-backup
spec:
  cronExpression: "0 0 * * *"
  podName: nginx
  path: /usr/local/nginx
```

Create the object from the YAML manifest file:

```
$ kubectl apply -f backup.yaml
backup.example.com/nginx-backup created
```

You can interact with the object using the built-in `kubectl` commands for any other Kubernetes API primitive:

```
$ kubectl get backups
```

```
NAME          AGE
```

```
nginx-backup  24s
```

```
$ kubectl describe backup nginx-backup
```

```
...
```


Chapter 8, Helm and Kustomize

1. Add the Helm chart repository using the provided URL. The name assigned to the URL is `prometheus-community`:

```
$ helm repo add prometheus-community https://prometheus-  
community.\  
github.io/helm-charts  
"prometheus-community" has been added to your repositories
```

Update the chart information with the following command:

```
$ helm repo update prometheus-community  
Hang tight while we grab the latest from your chart  
repositories...  
...Successfully got an update from the "prometheus-community"  
\  
chart repository  
Update Complete. *Happy Helming!*
```

You can search published chart versions in the the repository named `prometheus-community`:

```
$ helm search hub prometheus-community  
URL                                     CHART  
VERSION    ...  
https://artifacthub.io/packages/helm/prometheus... 70.3.0  
...
```

Install the latest version of the chart `kube-prometheus-stack`:

```
$ helm install prometheus prometheus-community/kube-  
prometheus-stack  
NAME: prometheus
```

```
LAST DEPLOYED: Wed Mar 26 14:14:53 2025
NAMESPACE: default
STATUS: deployed
REVISION: 1
...
```

The installed charts can be listed with the following command:

```
$ helm list
NAME            NAMESPACE    REVISION    UPDATED     ...
prometheus      default       1           2025-03-26  ...
```

One of the objects created by the chart is the Service named **prometheus-operated**. This Service exposes the Prometheus dashboard on port 9090:

```
$ kubectl get service prometheus-operated
NAME                                TYPE           CLUSTER-IP    EXTERNAL-IP
...
prometheus-operated                ClusterIP      None           <none>      ...
```

Set up port forwarding from port 8080 to port 9090:

```
$ kubectl port-forward service/prometheus-operated 8080:9090
Forwarding from 127.0.0.1:8080 -> 9090
Forwarding from [::1]:8080 -> 9090
```

Open a browser and enter the URL <http://localhost:8080/>. You will be presented with the Prometheus dashboard.

localhost

 Prometheus Alerts Graph Status ▾ Help



☐ Use local time

☐ Enable query history

☒ Enable autocomplete

☒ Enable highlighting

☒ Enable linter

Q

Expression (press Shift+Enter for newlines)

Execute

Table

Graph

<

Evaluation time

>

No data queried yet

Remove Panel

Add Panel

Uninstall the chart with the following command:

```
$ helm uninstall prometheus
release "prometheus" uninstalled
```

2. Navigate to the folder containing the `manifests` directory. Create all objects contained in the `manifests` directory using the recursive `apply` command:

```
$ kubectl apply -f manifests/ -R
configmap/logs-config created
pod/nginx created
```

Modify the value of the key `dir` in the file `configmap.yaml` using an editor. Then update the live object of the ConfigMap using the following command:

```
$ vim manifests/configmap.yaml
$ kubectl apply -f manifests/configmap.yaml
configmap/logs-config configured
```

Delete all objects that have been created from the `manifests` directory using the recursive `delete` command:

```
$ kubectl delete -f manifests/ -R
configmap "logs-config" deleted
pod "nginx" deleted
```

Create the file `kustomization.yaml`. It should define the common attribute for the namespace and reference the resource with the file `pod.yaml`. The following YAML file shows its contents:

```
namespace: t012
resources:
- pod.yaml
```

Run the following `kustomize` command to render the transformed manifest as console output:

```
$ kubectl kustomize ./
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: t012
spec:
  containers:
  - image: nginx:1.21.1
    name: nginx
```

Chapter 9, Pods and Namespaces

1. You can either use the imperative approach or the declarative approach. First, we'll look at creating the namespace with the imperative approach:

```
$ kubectl create namespace j43
```

Create the Pod:

```
$ kubectl run nginx --image=nginx:1.17.10 --port=80 --  
namespace=j43
```

Alternatively, you can use the declarative approach. Create a new YAML manifest in the file called *namespace.yaml* with the following contents:

```
apiVersion: v1  
kind: Namespace  
metadata:  
  name: j43
```

Create the namespace from the YAML manifest:

```
$ kubectl apply -f namespace.yaml
```

Create a new YAML manifest in the file *nginx-pod.yaml* with the following contents:

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: nginx  
spec:
```

```
containers:
- name: nginx
  image: nginx:1.17.10
  ports:
  - containerPort: 80
```

Create the Pod from the YAML manifest:

```
$ kubectl apply -f nginx-pod.yaml --namespace=j43
```

You can use the command-line option `-o wide` to retrieve the IP address of the Pod:

```
$ kubectl get pod nginx --namespace=j43 -o wide
```

The same information is available if you query for the Pod details:

```
$ kubectl describe pod nginx --namespace=j43 | grep IP:
```

You can use the command-line options `--rm` and `-it` to start a temporary Pod. The following command assumes that the IP address of the Pod named `nginx` is `10.1.0.66`:

```
$ kubectl run busybox --image=busybox:1.36.1 --restart=Never -
-rm -it \
-n j43 -- wget -O- 10.1.0.66:80
```

To download the logs, use a simple `logs` command:

```
$ kubectl logs nginx --namespace=j43
```

Editing the live object is forbidden. You will receive an error message if you try to add the environment variables:

```
$ kubectl edit pod nginx --namespace=j43
```

You will have to re-create the object with a modified YAML manifest, but first you'll have to delete the existing object:

```
$ kubectl delete pod nginx --namespace=j43
```

Edit the existing YAML manifest in the file *nginx-pod.yaml*:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.17.10
    ports:
    - containerPort: 80
    env:
    - name: DB_URL
      value: postgresql://mydb:5432
    - name: DB_USERNAME
      value: admin
```

Apply the changes:

```
$ kubectl apply -f nginx-pod.yaml --namespace=j43
```

Use the `exec` command to open an interactive shell to the container:

```
$ kubectl exec -it nginx --namespace=j43 -- /bin/sh
# ls -l
```

2. Combine the command-line options `-o yaml` and `--dry-run=client` to write the generated YAML to a file. Make sure to escape the double-quote characters of the string rendered by the `echo`

command:

```
$ kubectl run loop --image=busybox:1.36.1 -o yaml --dry-run=client \
  --restart=Never -- /bin/sh -c 'for i in 1 2 3 4 5 6 7 8 9
10; \
  do echo "Welcome $i times"; done' \
  > pod.yaml
```

Create the Pod from the YAML manifest:

```
$ kubectl apply -f pod.yaml --namespace=j43
```

The status of the Pod will say **Completed**, as the executed command in the container does not run in an infinite loop:

```
$ kubectl get pod loop --namespace=j43
```

The container command cannot be changed for existing Pods. Delete the Pod so you can modify the manifest file and re-create the object:

```
$ kubectl delete pod loop --namespace=j43
```

Change the YAML manifest content:

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: loop
    name: loop
spec:
  containers:
  - args:
```

```
- /bin/sh
- -c
- while true; do date; sleep 10; done
image: busybox:1.36.1
name: loop
resources: {}
dnsPolicy: ClusterFirst
restartPolicy: Never
status: {}
```

Create the Pod from the YAML manifest:

```
$ kubectl apply -f pod.yaml --namespace=j43
```

You can describe the Pod events by grepping for the term:

```
$ kubectl describe pod loop --namespace=j43 | grep -C 10
Events:
```

You can simply delete the namespace, which will delete all objects within the namespace:

```
$ kubectl delete namespace j43
```

Chapter 11, ConfigMaps and Secrets

1. Create the ConfigMap and point to the text file upon creation:

```
$ kubectl create configmap app-config --from-  
file=application.yaml  
configmap/app-config created
```

The ConfigMap defines a single key-value pair. The key is the name of the YAML file, and the value is the contents of *application.yaml*:

```
$ kubectl get configmap app-config -o yaml  
apiVersion: v1  
data:  
  application.yaml: |-  
    dev:  
      url: http://dev.bar.com  
      name: Developer Setup  
    prod:  
      url: http://foo.bar.com  
      name: My Cool App  
kind: ConfigMap  
metadata:  
  creationTimestamp: "2023-05-22T17:47:52Z"  
  name: app-config  
  namespace: default  
  resourceVersion: "7971"  
  uid: 00cf4ce2-ebec-48b5-a721-e1bde2aabd84
```

Execute the run command in combination with the `--dry-run` flag to generate the file for the Pod:

```
$ kubectl run backend --image=nginx:1.23.4-alpine -o yaml \  
--dry-run=client --restart=Never > pod.yaml
```

The final YAML manifest should look similar to the following code snippet:

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: backend
    name: backend
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: backend
    volumeMounts:
    - name: config-volume
      mountPath: /etc/config
  volumes:
  - name: config-volume
    configMap:
      name: app-config
```

Create the Pod by pointing the `apply` command to the YAML manifest:

```
$ kubectl apply -f pod.yaml
pod/backend created
```

Log into the Pod and navigate to the directory `/etc/config`. You will find the file `application.yaml` with the expected YAML content:

```
$ kubectl exec backend -it -- /bin/sh
/ # cd /etc/config
/etc/config # ls
application.yaml
/etc/config # cat application.yaml
```

```
dev:
  url: http://dev.bar.com
  name: Developer Setup
prod:
  url: http://foo.bar.com
  name: My Cool App
/etc/config # exit
```

2. It's easy to create the Secret from the command line:

```
$ kubectl create secret generic db-credentials --from-
literal=\
db-password=password
secret/db-credentials created
```

The imperative command automatically Base64-encodes the provided value of the literal. You can render the details of the Secret object from the command line. The assigned value to the key `db-password` is `cGFzc3dk`:

```
$ kubectl get secret db-credentials -o yaml
apiVersion: v1
data:
  db-password: cGFzc3dk
kind: Secret
metadata:
  creationTimestamp: "2023-05-22T16:47:33Z"
  name: db-credentials
  namespace: default
  resourceVersion: "7557"
  uid: 2daf580a-b672-40dd-8c37-a4adb57a8c6c
type: Opaque
```

Execute the `run` command in combination with the `--dry-run` flag to generate the file for the Pod:

```
$ kubectl run backend --image=nginx:1.23.4-alpine -o yaml \
  --dry-run=client --restart=Never > pod.yaml
```

Edit the YAML manifest and create an environment that reads the key from the Secret while assigning a new name for it.

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: backend
    name: backend
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: backend
    env:
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: db-password
```

Create the Pod by pointing the `apply` command to the YAML manifest:

```
$ kubectl apply -f pod.yaml
pod/backend created
```

You can find the environment variable in Base64-decoded form by shelling into the container and running the `env` command:

```
$ kubectl exec -it backend -- env
DB_PASSWORD=passwd
```

Chapter 11, Deployments and ReplicaSets

1. Create the YAML manifest for a Deployment in the file *nginx-deployment.yaml*. The label selector should match the labels assigned to the Pod template. The following code snippet shows the contents of the YAML manifest file:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    tier: backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: v1
  template:
    metadata:
      labels:
        app: v1
    spec:
      containers:
        - image: nginx:1.23.0
          name: nginx
```

Create the deployment by pointing it to the YAML manifest. Check on the Deployment status:

```
$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx created
$ kubectl get deployment nginx
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
------	-------	------------	-----------	-----

```
nginx    3/3      3          3          10s
```

Set the new image and check the revision history:

```
$ kubectl set image deployment/nginx nginx=nginx:1.23.4
deployment.apps/nginx image updated
$ kubectl rollout history deployment nginx
deployment.apps/nginx
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
$ kubectl rollout history deployment nginx --revision=2
deployment.apps/nginx with revision #2
Pod Template:
  Labels:      app=v1
              pod-template-hash=5bd95c598
Containers:
  nginx:
    Image:      nginx:1.23.4
    Port:       <none>
    Host Port:  <none>
    Environment: <none>
    Mounts:      <none>
    Volumes:      <none>
```

Add the change cause to the current revision by annotating the Deployment object:

```
$ kubectl annotate deployment nginx kubernetes.io/change-
cause=\
"Pick up patch version"
deployment.apps/nginx annotated
```

The revision change cause can be inspected by rendering the rollout history:


```
$ kubectl rollout history deployment nginx
deployment.apps/nginx
REVISION  CHANGE-CAUSE
1          <none>
2          Pick up patch version
```

Now, scale the Deployment to 5 replicas. You should find 5 Pods controlled by the Deployment:

```
$ kubectl scale deployment nginx --replicas=5
deployment.apps/nginx scaled
$ kubectl get pod -l app=v1
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-5bd95c598-25z4j	1/1	Running	0	3m39s
nginx-5bd95c598-46mlt	1/1	Running	0	3m38s
nginx-5bd95c598-bszvp	1/1	Running	0	48s
nginx-5bd95c598-dwr8r	1/1	Running	0	48s
nginx-5bd95c598-kjrvf	1/1	Running	0	3m37s

Roll back to revision 1. You will see the new revision. Inspecting the revision should show the image `nginx:1.23.0`:

```
$ kubectl rollout undo deployment/nginx --to-revision=1
deployment.apps/nginx rolled back
$ kubectl rollout history deployment nginx
deployment.apps/nginx
REVISION  CHANGE-CAUSE
2          Pick up patch version
3          <none>
$ kubectl rollout history deployment nginx --revision=3
deployment.apps/nginx with revision #3
Pod Template:
  Labels:      app=v1
              pod-template-hash=f48dc88cd
  Containers:
```

nginx:

Image: nginx:1.23.0

Port: <none>

Host Port: <none>

Environment: <none>

Mounts: <none>

Volumes: <none>

Chapter 12, Scaling Workloads

1. Define the Deployment in the file *nginx-deployment.yaml*, as shown:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.23.4
          name: nginx
          resources:
            requests:
              cpu: "0.5"
              memory: "500Mi"
            limits:
              memory: "500Mi"
```

Create the Deployment object from the manifest file:

```
$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx created
```

Ensure that all Pods controlled by the Deployment transition into

“Running” status:

```
$ kubectl get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	1/1	1	1	49s

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-5bbd9746c-9b4np	1/1	Running	0	24s

Next, define the HorizontalPodAutoscaler with the given resource thresholds in the file *nginx-hpa.yaml*. The final manifest is shown here:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx
  minReplicas: 3
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 75
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 60
```

Create the HorizontalPodAutoscaler object from the manifest file:

```
$ kubectl apply -f nginx-hpa.yaml
horizontalpodautoscaler.autoscaling/nginx-hpa created
```

When you inspect the HorizontalPodAutoscaler object, you will find that the number of replicas will be scaled up to the minimum number 3 even though the Deployment defines only a single replica. At the time of running this command, Pods are not using a significant amount of CPU and memory. That's why the current metrics show 0%:

```
$ kubectl get hpa nginx-hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
nginx-hpa	Deployment/nginx	0%/60%, 0%/75%	3	8	3	2m19s

Appendix B. Exam Review Guide

This book covers all objectives relevant to the exam and more. The tables in this appendix map the exam objective to the corresponding book chapter. Furthermore, you will also find a reference to the Kubernetes documentation. Some foundational objectives important to the exam, such as Pods and namespaces, have not been listed explicitly in the curriculum; however, the book does cover them. You can use the mapping as a reference to review specific objectives in more detail.

Cluster Architecture, Installation & Configuration

Exam Objective	Chapter	Reference Documentation	Tutorial
Manage role based access control (RBAC)	Chapter 6	Using RBAC Authorization, Role Based Access Control Good Practices	N/A
Prepare underlying infrastructure for installing a Kubernetes cluster	Chapter 4	N/A	N/A
Create and manage Kubernetes	Chapter 4	kubeadm	Creating a cluster with kubeadm, Upgrading

clusters using kubeadm			kubeadm clusters
Manage the lifecycle of Kubernetes clusters	Chapter 5	etcd	Backing up an etcd cluster, Restoring an etcd cluster
Implement and configure a highly-available control plane	Chapter 4	kubeadm, Options for Highly Available Topology	Creating Highly Available Clusters with kubeadm
Use Helm and Kustomize to install cluster components	Chapter 8	Helm, Kustomize	Declarative Management of Kubernetes Objects Using Kustomize, Managing Secrets using Kustomize
Understand extension interfaces (CNI, CSI, CRI, etc.)	Chapter 4	Infrastructure extensions, Compute, Storage, and Networking Extensions	N/A
Understand CRDs, install and configure operators	Chapter 7	Custom Resources, Operator pattern	Use Custom Resources

Workloads & Scheduling

Exam Objective	Chapter	Reference Documentation	Tutorial
Understand application deployments and how to perform rolling update and rollbacks	[Link to Come], [Link to Come]	Pods, Deployments, ReplicaSet	Performing a Rolling Update
Use ConfigMaps and Secrets to configure applications	[Link to Come]	ConfigMaps, Secrets	Configure a Pod to Use a ConfigMap, Managing Secrets using kubectl, Managing Secrets using Configuration File
Configure workload autoscaling	[Link to Come]	Scaling a Deployment	Running Multiple Instances of Your App, Horizontal Pod Autoscaling
Understand the primitives used to create robust, self-healing, application deployments	[Link to Come]	ReplicaSet	N/A
Configure Pod admission and scheduling (limits,	N/A	Resource Management for Pods and	N/A

node affinity, etc.)

Containers,
Assigning Pods to
Nodes

Servicing & Networking

Exam Objective	Chapter	Reference Documentation	Tutorial
Understand connectivity between Pods	N/A	DNS for Services and Pods	N/A
Define and enforce Network Policies	N/A	Network Policies	Declare Network Policy
Understand ClusterIP, NodePort, LoadBalancer service types and endpoints	N/A	Service	Connecting Applications with Services
Use the Gateway API to manage Ingress traffic	N/A	Gateway API	N/A
Know how to use Ingress controllers and Ingress resources	N/A	Ingress, Ingress Controllers	Set up Ingress on Minikube with the NGINX Ingress Controller
Understand and use	N/A	CoreDNS	N/A

Storage

Exam Objective	Chapter	Reference Documentation	Tutorial
Implement storage classes and dynamic volume provisioning	N/A	Persistent Volumes, Storage Classes	N/A
Configure volume types, access modes and reclaim policies	N/A	Persistent Volumes	N/A
Manage persistent volumes and persistent volume claims	N/A	PersistentVolumeClaims	Configure a Pod to Use a PersistentVolume for Storage

Troubleshooting

Exam Objective	Chapter	Reference Documentation	Tutorial
Troubleshoot clusters and nodes	N/A	Troubleshooting Clusters	N/A

Troubleshoot cluster components	N/A	Troubleshooting Clusters	N/A
Monitor cluster and application resource usage	N/A	Metrics Server, kubectl top	N/A
Manage and evaluate container output streams	N/A	Pod and container logs	N/A

About the Author

Benjamin Muschko is a software engineer, consultant, and trainer with more than 20 years of experience in the industry. He's passionate about project automation, testing, and continuous delivery. Ben is an author, a frequent speaker at conferences, and an avid open source advocate. He holds the CKAD, CKA and CKS certifications.

Software projects sometimes feel like climbing a mountain. In his free time, Ben loves hiking **Colorado's 14ers** and enjoys conquering long-distance trails.